

딥러닝

과정 목표

◆ 딥러닝 기초

- 인공지능의 이해
- 딥러닝 개념 이해
- 딥러닝 예제 실습

환경 설정 1. 가상환경 만들기

- ✓ 현재 존재하는 가상환경 목록 확인
 - `conda env list`
- ✓ 가상환경 생성
 - `conda create -n ml-dl-nlp python=3.10`
- ✓ 생성된 가상환경으로 들어가기
 - `conda activate ml-dl-nlp`
- ✓ 가상환경 나오기
 - `conda deactivate`
- ✓ 가상환경 지우기
 - `conda env remove --name 가상환경이름`

환경 설정 2. 설치 라이브러리

- ✓ `python -m pip install --upgrade pip`
- ✓ `pip install jupyter numpy==1.23.5 pandas==1.5.3
matplotlib==3.7.0 seaborn==0.12.2 scikit-learn==1.2.1`
- ✓ `pip install tensorflow==2.10.0`

환경 설정 3.

가상환경을 jupyter에서 선택가능한 커널로 등록

- ✓ 가상환경에 들어가서 jupyter 설치
 - `conda install ipykernel`
- ✓ Jupyter에서 선택가능한 커널로 등록
 - `python -m ipykernel install --user --name ml-dl-nlp --display-name "ml-dl-nlp(ipykernel)"`
- ✓ Jupyter 커널 목록 확인
 - `jupyter kernelspec list`
- ✓ Jupyter 커널 목록에서 삭제
 - `jupyter kernelspec remove display이름`
- ✓ Jupyter lab 실행
 - `Jupyter lab --notebook-dir=d:\Wai\source\W05_deepLearning`

인공지능의 이해

(인공지능, 기계학습, 딥러닝)

인공지능, 기계학습, 딥러닝

Artificial Intelligence

인공지능

사고나 학습 등 인간이 가진
지적 능력을 컴퓨터를 통해
구현하는 기술



Machine Learning

머신러닝

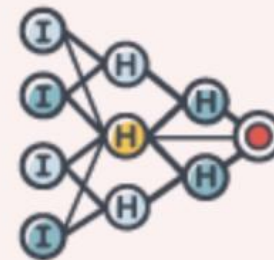
컴퓨터가 스스로 학습하여
인공지능의 성능을
향상 시키는 기술 방법



Deep Learning

딥러닝

인간의 뉴런과 비슷한
인공신경망 방식으로
정보를 처리



인공지능이란?

Artificial intelligence

From Wikipedia, the free encyclopedia

[출처:

https://en.wikipedia.org/wiki/Artificial_intelligence]

"AI" redirects here. For other uses, see [AI \(disambiguation\)](#) and [Artificial intelligence \(disambiguation\)](#).

Artificial intelligence (AI, also **machine intelligence**, **MI**) is [intelligence](#) demonstrated by [machines](#), in contrast to the **natural intelligence** (NI) displayed by humans and other animals. In [computer science](#) AI research is defined as the study of "[intelligent agents](#)": any device that perceives its environment and takes actions that maximize its chance of successfully achieving its goals.^[1] Colloquially, the term "artificial intelligence" is applied when a machine mimics "cognitive" functions that humans associate with other [human minds](#), such as "learning" and "problem solving".^[2]



[구글 번역]

[<https://translate.google.com>]

인공 지능 (인공 지능, 기계 지능, MI)은 사람이나 다른 동물에 의해 표시되는 자연 지능 (NI)과 달리 기계가 시연하는 지능입니다. 컴퓨터 과학에서 인공 지능 연구는 "지능형 에이전트"의 연구로 정의됩니다. 즉, 환경을 인식하고 목표를 성공적으로 달성 할 수있는 기회를 최대화하는 장치입니다. 말하자면 "인공 지능"이라는 용어는 인간이 "학습"및 "문제 해결"과 같은 다른 인간의 마음과 관련시키는 "인지"기능을 모방 할 때 적용됩니다.



수정 제안하기

인공지능

프로 9단 이세돌

인간 대 인공지능의 대결

1967년	체스	체스 프로그램 '맥핵' vs 철학자 후버트 드레퓔스	맥핵 승
1996년	체스	IBM 슈퍼컴퓨터 '딥블루' vs 체스 세계챔피언 개리 카스파로프	카스파로프 승 (3승 2무 1패)
1997년	체스	IBM 슈퍼컴 '딥퍼블루' vs 카스파로프	딥퍼블루 승 (2승 3무 1패)
2011년	퀴즈	IBM 슈퍼컴 '왓슨' vs 퀴즈 챔피언 제닝스·루터	왓슨 우승
2013년	장기	장기 프로그램 '어웨이크' vs 일본 프로기사 연합	어웨이크 승 (3승 1무 1패)
2015년	포커	포커 프로그램 '클라우드코' vs 프로 포커선수 4명	프로 선수 승리
2015년	바둑	구글 '알파고' vs 프로 바둑기사 판후이(2단)	알파고 승 (5승 무패)
2016년 3월	바둑	알파고 vs 프로 9단 이세돌	?



그래픽=양인성 기자

인공지능

알파고 세기의 대결 전적

vs 이세돌(2016년 3월 9~15일)

이세돌	알파고	1 : 4	1국	알파고 불계승
			2국	알파고 불계승
			3국	알파고 불계승
			4국	이세돌 불계승
			5국	알파고 불계승

vs 커제(2017년 5월 23~27일)

커제	알파고	0 : 3	1국	알파고 1집반 승
			2국	알파고 불계승
			3국	알파고 불계승

인공지능(AI) 기술 앞세워 의료분야 진출하는 기업들

구글

당뇨망막병증과 노인성 황반변성증 등의 조기 진단법 찾는 의학연구 프로젝트 진행. 빅데이터로 100만 명 진료기록 분석

IBM

암을 진단하는 '왓슨 온톨로지', 150만 명 이상의 환자 기록 등을 분석. 지난해 12월 가천대길병원이 이를 도입해 인공지능 암센터 개소

애플

애플워치를 통해 사용자 건강 데이터 수집 목표

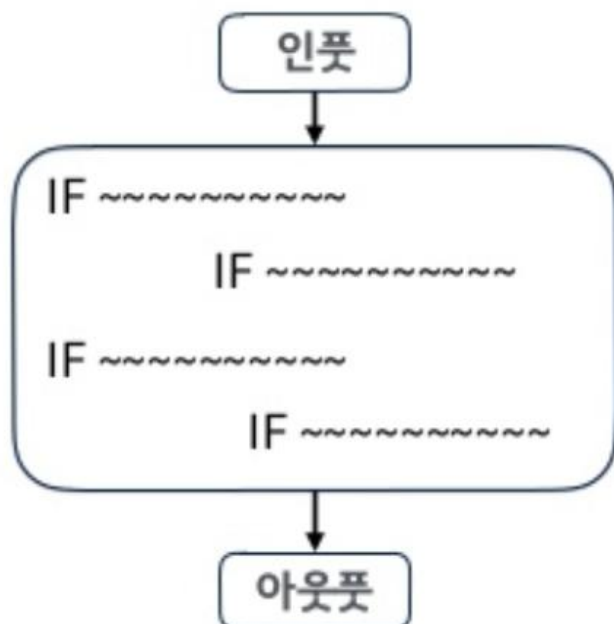
페이스북

운동기록 활동 데이터를 추적 관리하는 모바일 애플리케이션 무브스(Moves) 인수, 빅데이터 수집과 딥러닝 기술 개발

기계 학습

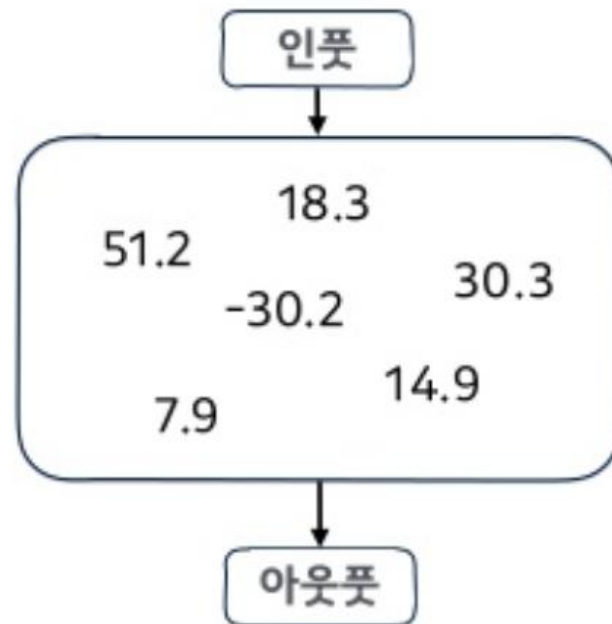
기존의 코딩:

조건을 라인바이라인으로 **긴 코드**로 써내려 가는 일

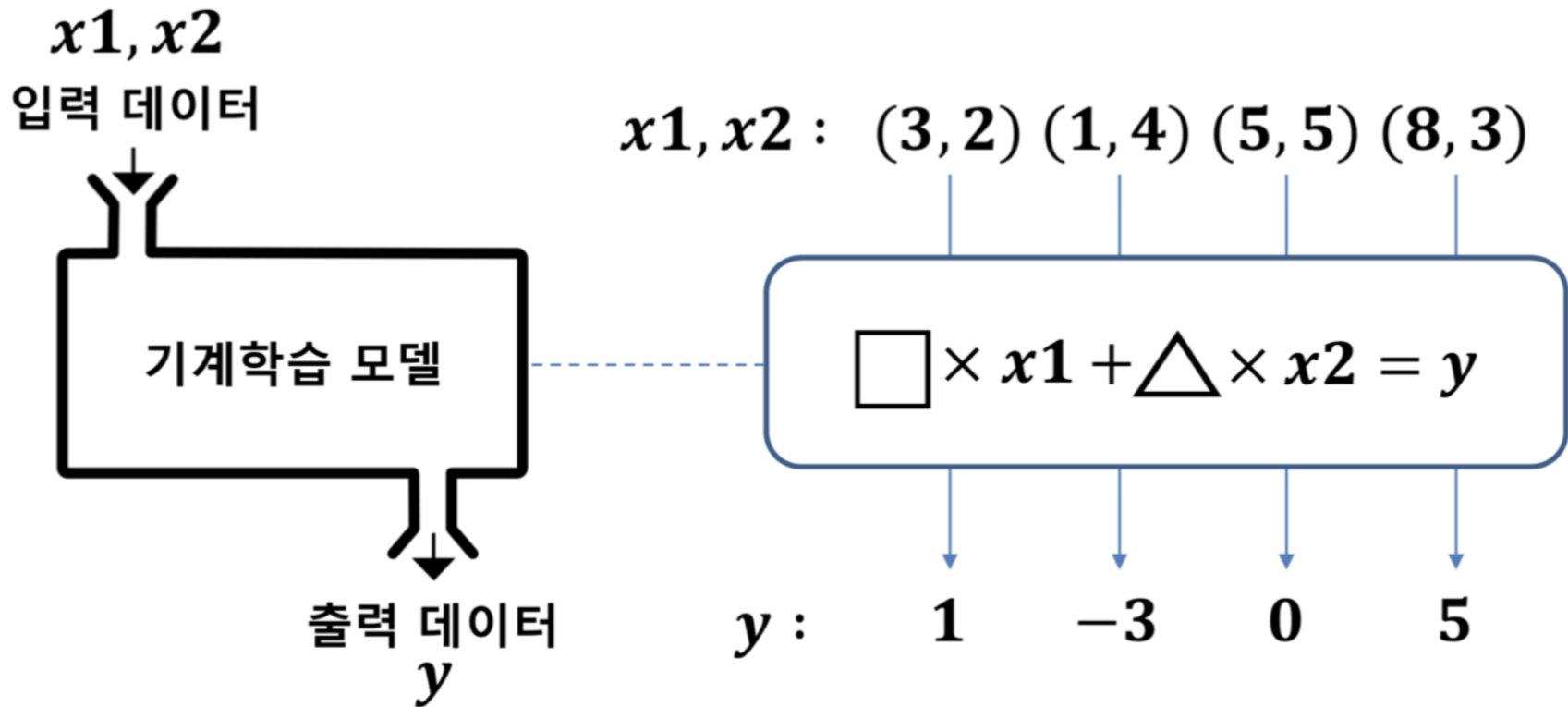


앞으로의 코딩:

조건을 학습 **모델의 여러 가중치로 변환**하는 일



기계 학습



| $\square$ 와 $\triangle$ 를 가중치(weight)라고 하며 기계학습은 이러한 가중치를 컴퓨터가 스스로 '학습(Learning)'을 통해 찾아내도록 하는 것

$$\square = 1$$

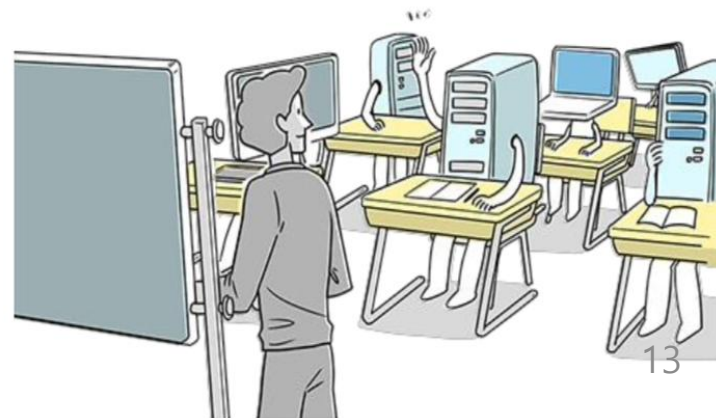
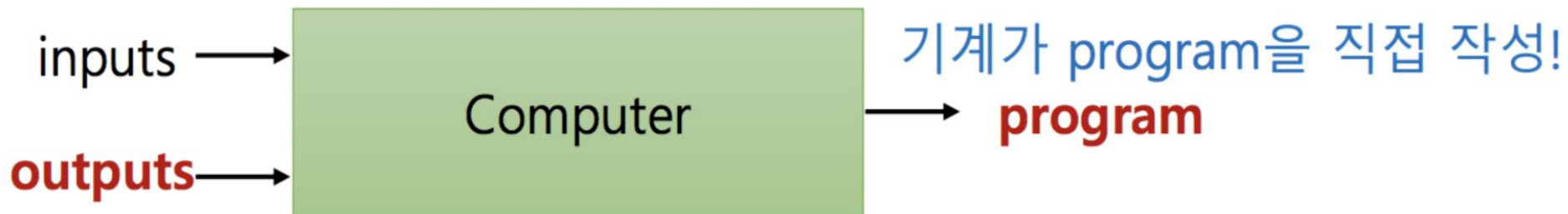
$$\triangle = -1$$

기계 학습

일반적인 programming



기계 학습



기계학습



IF 바다에 살고,
IF 이빨이 날카롭고,
IF 무섭게 생겼다.



애도 상어,
재도 상어,
나도 상어,
상어들은 이렇구나..

AI가 인간보다 잘하는 분야 1 - 보드 게임 (2016)

Go 4:1



AlphaGo beats L. Sedols in 2016

Chess 2:1



Komodo beasts H. Nakamura in 2016

[출처: <https://goo.gl/XodPw>]

AI가 인간보다 잘하는 분야 2 - 이미지 분류 (2016)

Human Performance

95%



A close up of a child holding a stuffed animal



Two pizzas sitting on top of a stove top oven



A man flying through the air while riding a skateboard

A.I. Performance

97%



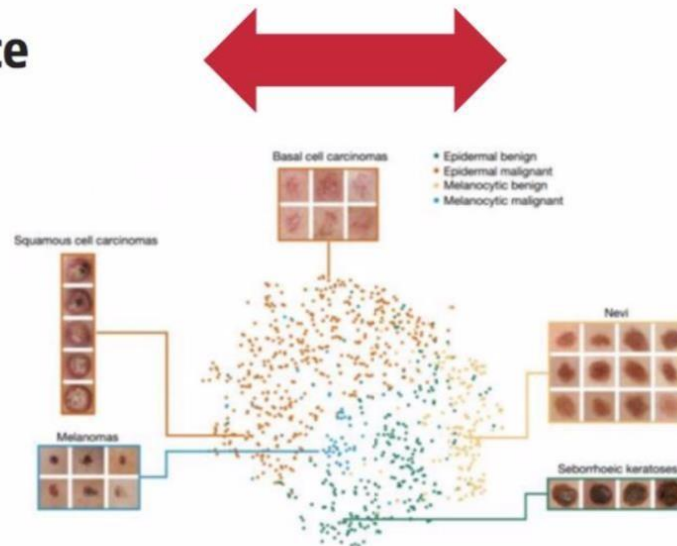
AI가 인간보다 잘하는 분야 3 – 피부암 진단 (2016)

Pathologist Performance

96,5%

A.I. Performance

97,1%



Pathologist + A.I.

99,5%

[출처: <http://www.nature.com/nature/journal/v542/n7639/full/>]

AI가 인간보다 잘하는 분야 4 – 유방암 진단 (2017)

Pathologist Performance

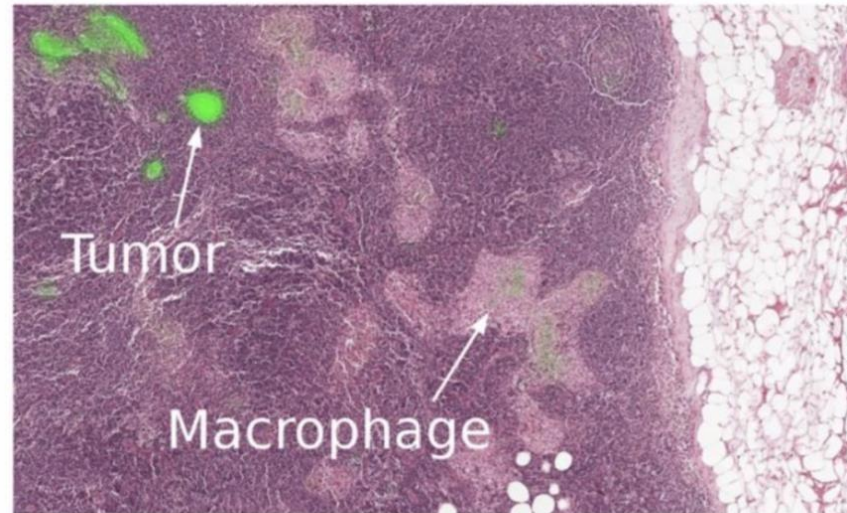
73%

The pathologist ended up spending 30 hours on this task on 130 slides



A.I. Performance

92%



A closeup of a lymph node biopsy.

[출처: <https://research.googleblog.com/2017/03/assisting-path>]

AI가 인간보다 잘하는 분야 5 - 얼굴 인식 (2016)

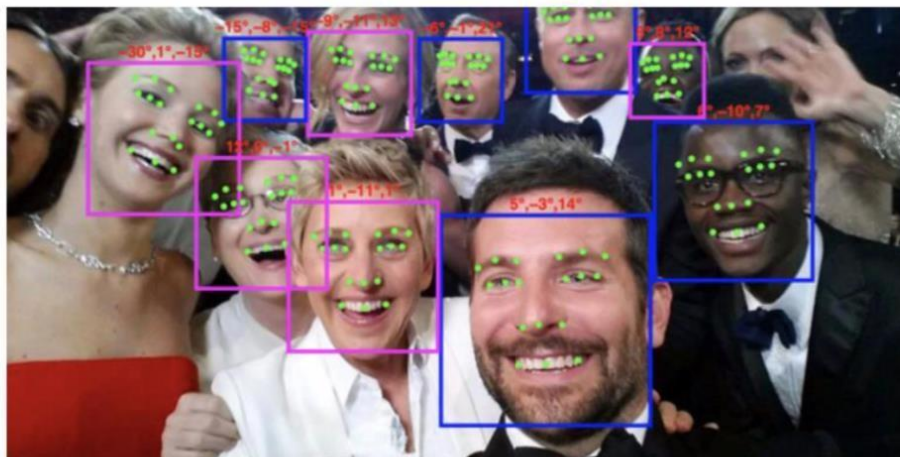
Human Performance

97,5%



A.I. Performance

97,7%



[출처: <https://arxiv.org/pdf/1603.0>]

AI가 인간보다 잘하는 분야 6 – 입모양 읽기 기술 (2016)

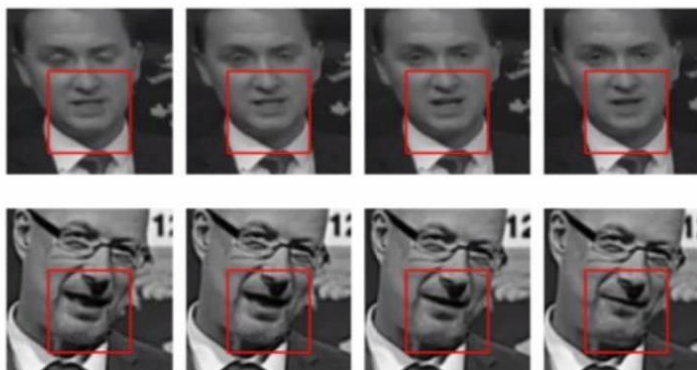
Human Performance

41,3%



A.I. Performance

57,9%



컴퓨팅

구글 인공지능, '독순술'도 깨쳤다

입모양 읽기 기술...사람 전문가보다 훨씬 정확



임유경 기자



입력 : 2016.11.24.09:09



수정 : 2016.11.24.09:09

[출처: <https://arxiv.org/pdf/1611.05>]

AI가 인간보다 잘하는 분야 7 - 음성 인식 (2016)

Human Performance

[제목 없음]

41,3%



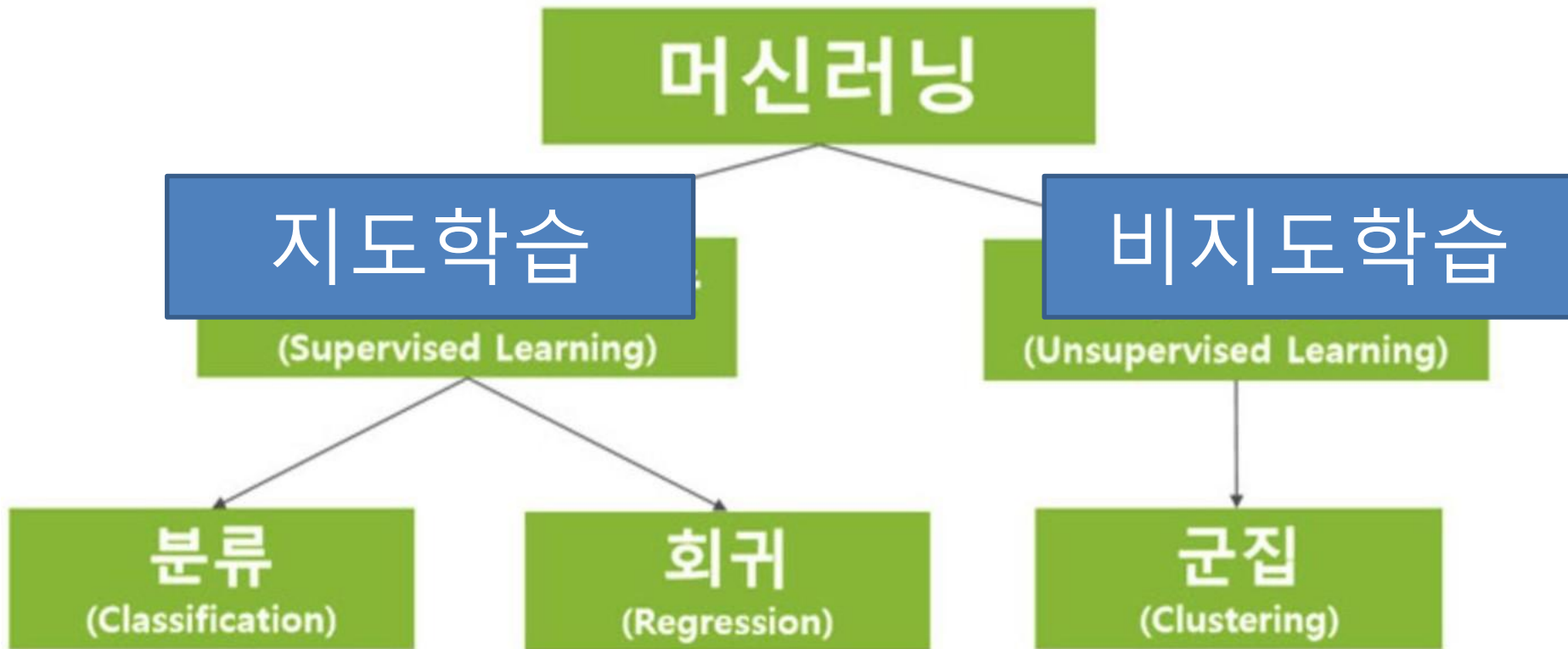
A.I. Performance

57,9%



[출처: <https://arxiv.org/pdf/1610.05>]

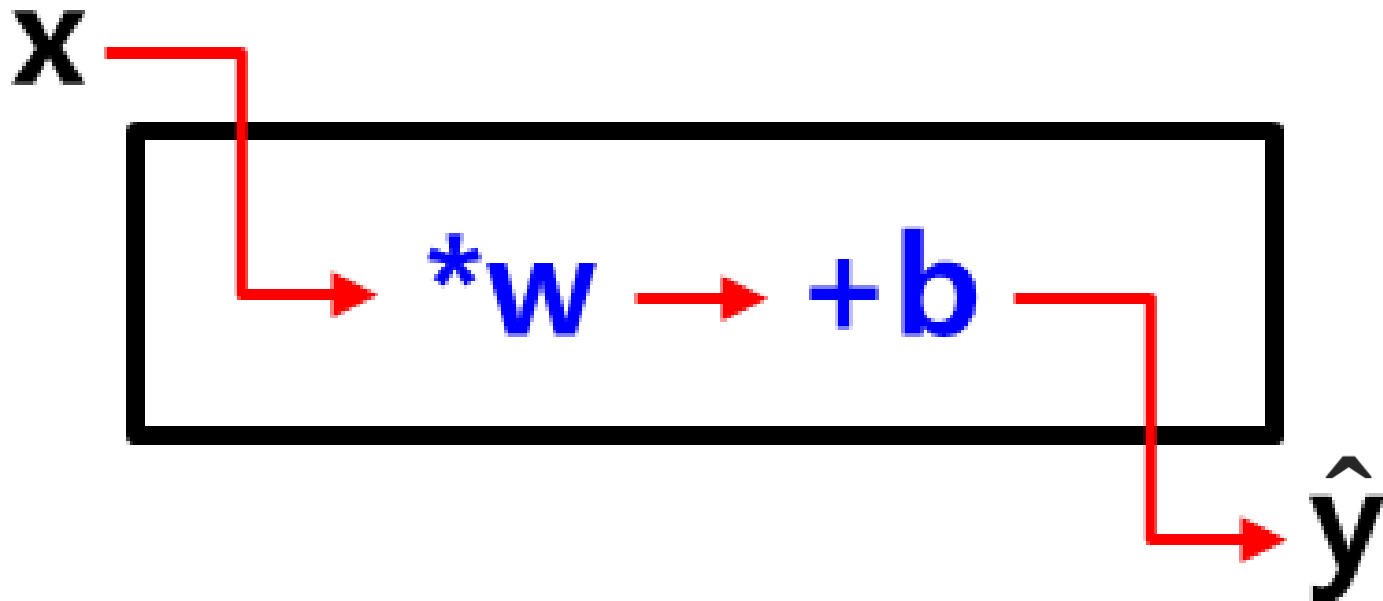
기계 학습



Machine Learning

Function

- $\hat{y} = \mathbf{w}x + \mathbf{b}$



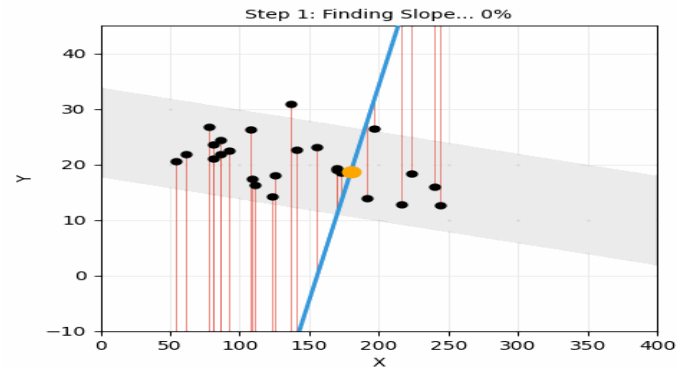
Machine Learning

Error Function() for Regression Analysis

- Error Function(): **실제값과 예측값의 차이**(Error)를 비교하는 지표

- y : 실제값, $\hat{y}$: 예측값
- $\text{Error}(y, \hat{y}) = (y - \hat{y})^2$

- Ex) **Model**: $\hat{y} = 2x + 1$



y	x	$\hat{y}$	Error
2	1		
4	3		
6	5		
8	7		
10	9		

* y 는 지도학습의 Output(Label), x 는 Input Data

Machine Learning

Mean Squared Error for Regression Analysis

- 변경된 w 와 b 에 대한 **Error 비교**
 - **Mean Squared Error(MSE)**는 Error의 평균
 - $MSE(y, \hat{y}) = \text{mean}((y - \hat{y})^2) = \text{mean}((y - wx - b)^2)$
- Ex) **Model**: $\hat{y} = 2x + 1$ vs. $\hat{y} = 1.5x + 1.5$

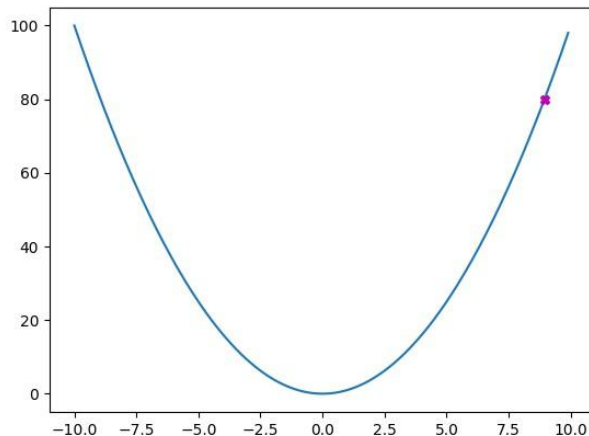
y	x	$\hat{y}_1$	Error_1	$\hat{y}_2$	Error_2
2	1				
4	3				
6	5				
8	7				
10	9				

Machine Learning

Parameters

- Error Function() 값을(MSE)을 **최소로 하는** Parameter(w, b) 값을 찾는 것이 목적
 - $\hat{y} = wx + b$
 - Parameter: Error(w, b)
- Ex) $MSE(2, 1) = 33 > MSE(1.5, 1.5) = 11$

Error

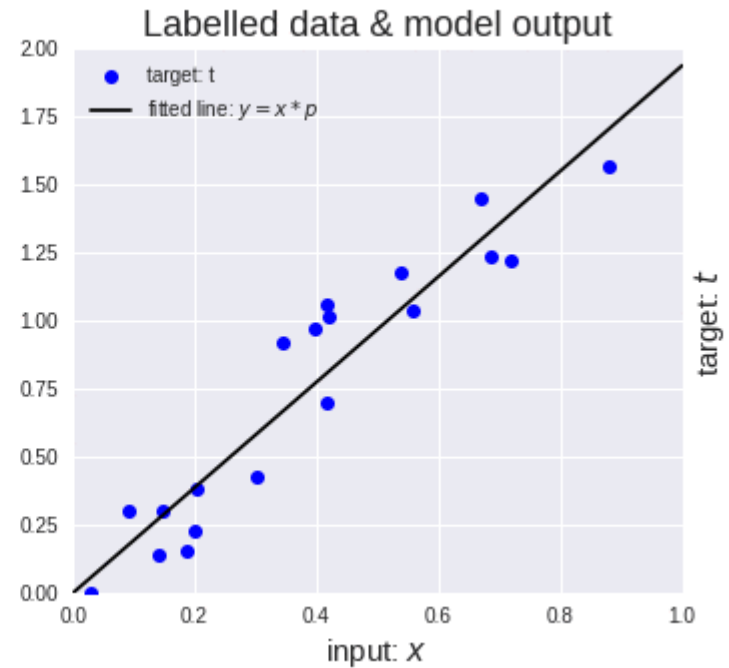
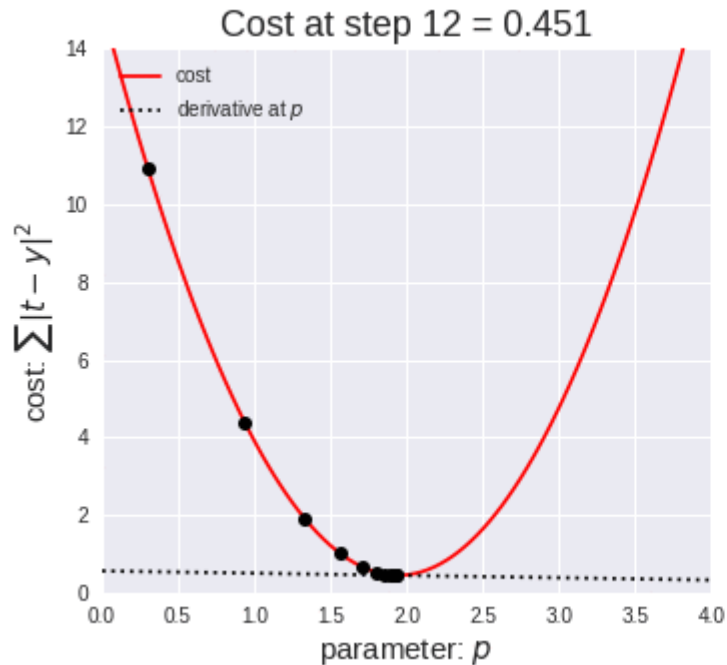


w or b

Machine Learning

Gradient Descent

- How to work?



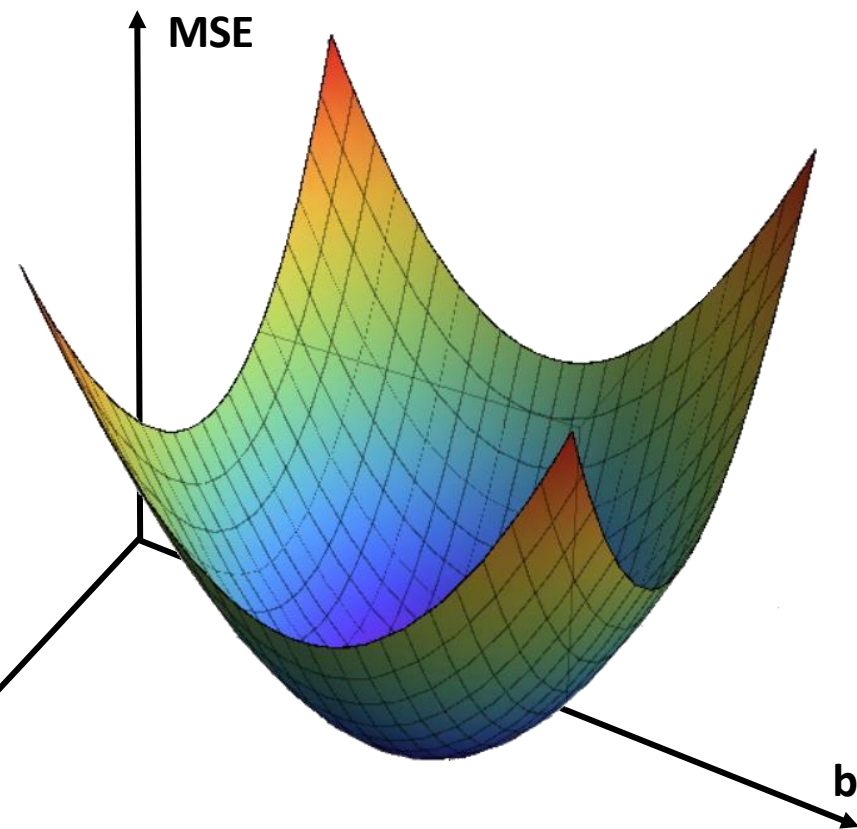
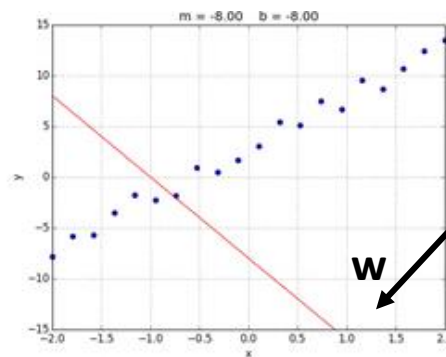
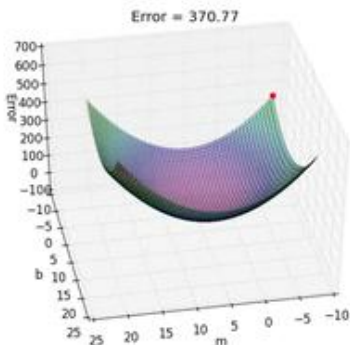
Machine Learning

편미분(Partial Derivative) 1/2

○ $\frac{\partial \text{Error}}{\partial w}$: Parameter 값이 **변경될 때** MSE값은 **어떻게 변화**되는가?

- ∂w or ∂b : Parameter 변화량
- ∂Error : Error Function() 변화량

○ $\frac{\partial \text{Error}}{\partial w}$ & $\frac{\partial \text{Error}}{\partial b}$



Machine Learning

편미분(Partial Derivative) 2/2

- How to calculate?

- $\text{Error}() = (y - \hat{y})^2 = (y - wx - b)^2$

- $\frac{\partial \text{Error}()}{\partial w} = (y - wx - b)^2 = 2(y - wx - b) * (-x)$

$$= (y - \hat{y}) * (-2x)$$

- $\frac{\partial \text{Error}()}{\partial b} = (y - wx - b)^2 = 2(y - wx - b) * (-1)$

$$= (y - \hat{y}) * (-2)$$

Machine Learning

Gradient Descent

○ 경사하강법

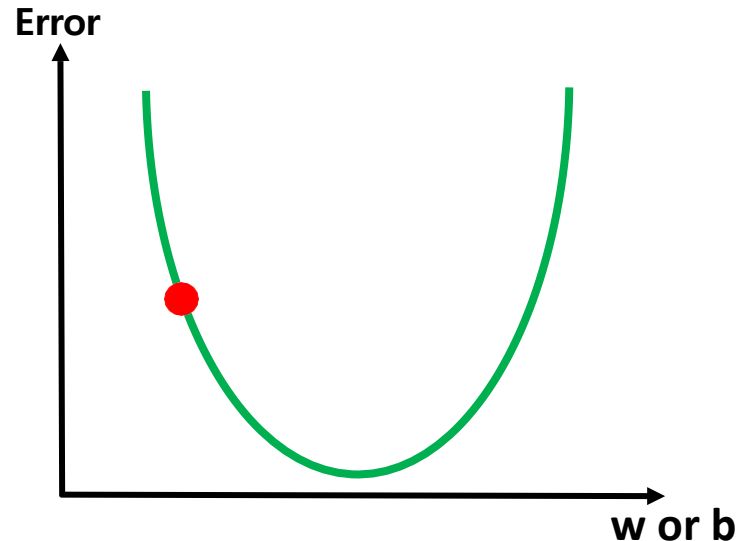
- $$\mathbf{W} = \mathbf{W} - \gamma \frac{\partial Error}{\partial \mathbf{W}}$$

- γ : Learning Rate or Step Size

- $\frac{\partial Error}{\partial \mathbf{W}}$ & $\frac{\partial Error}{\partial \mathbf{b}}$: Parameter(w, b) 값을 변경하면서 학습을 반복

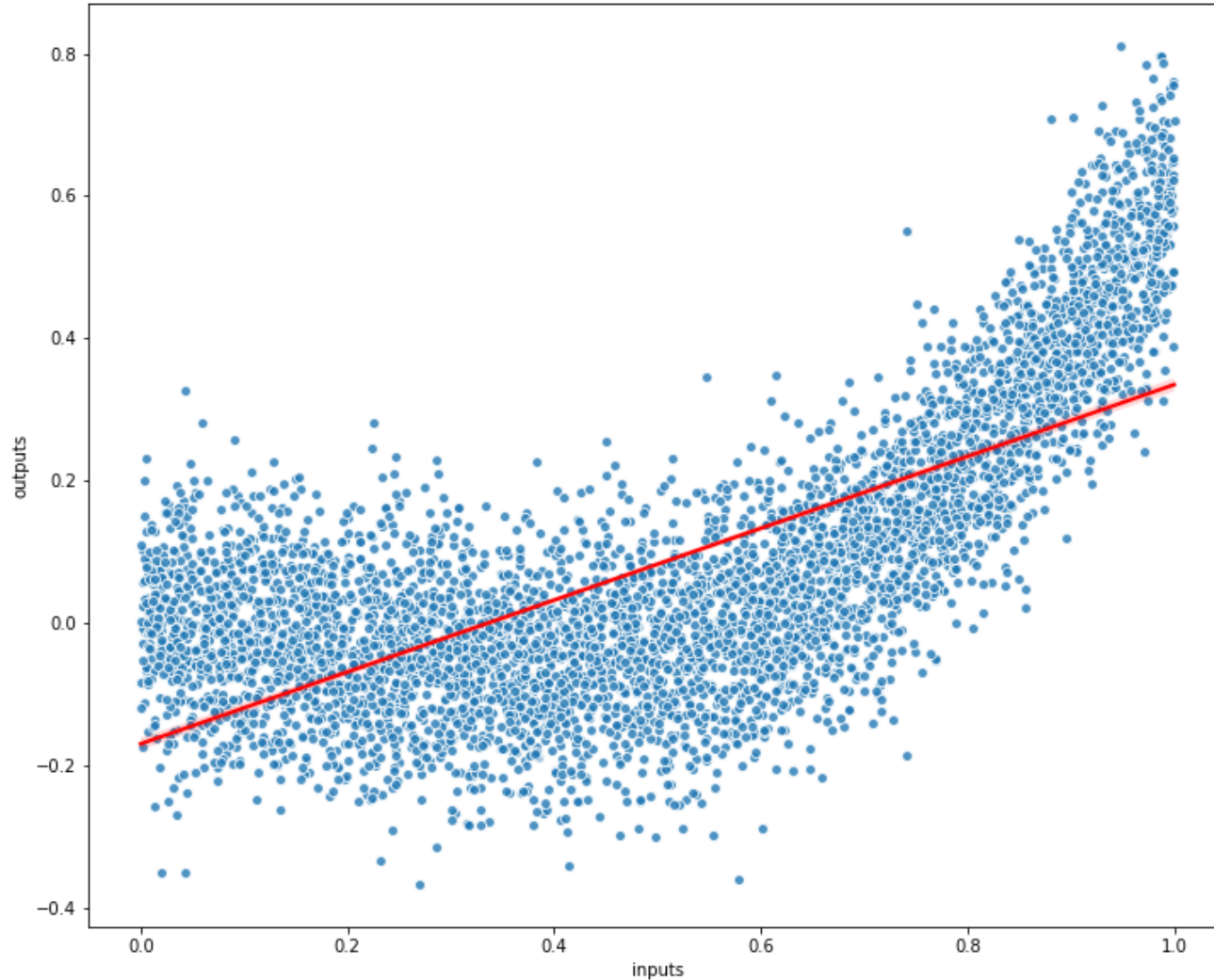
○ Learning 또는 Training

- 오차값 제곱 평균(MSE)을 최소로 만드는 Parameter(w, b) 값을 찾는 과정



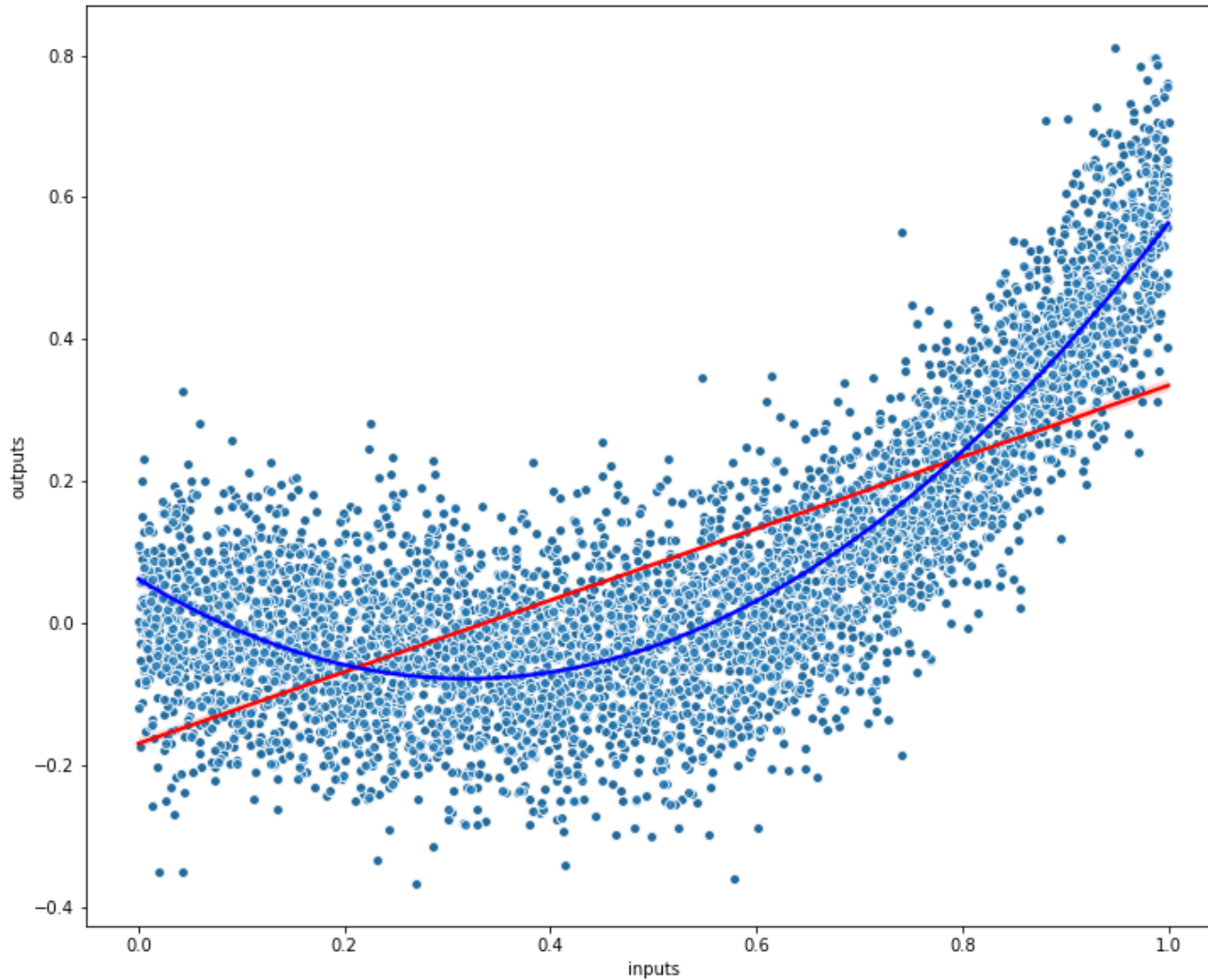
Model Validation

Enough vs. Not enough



Model Validation

First-Order Model vs. High-Order Model



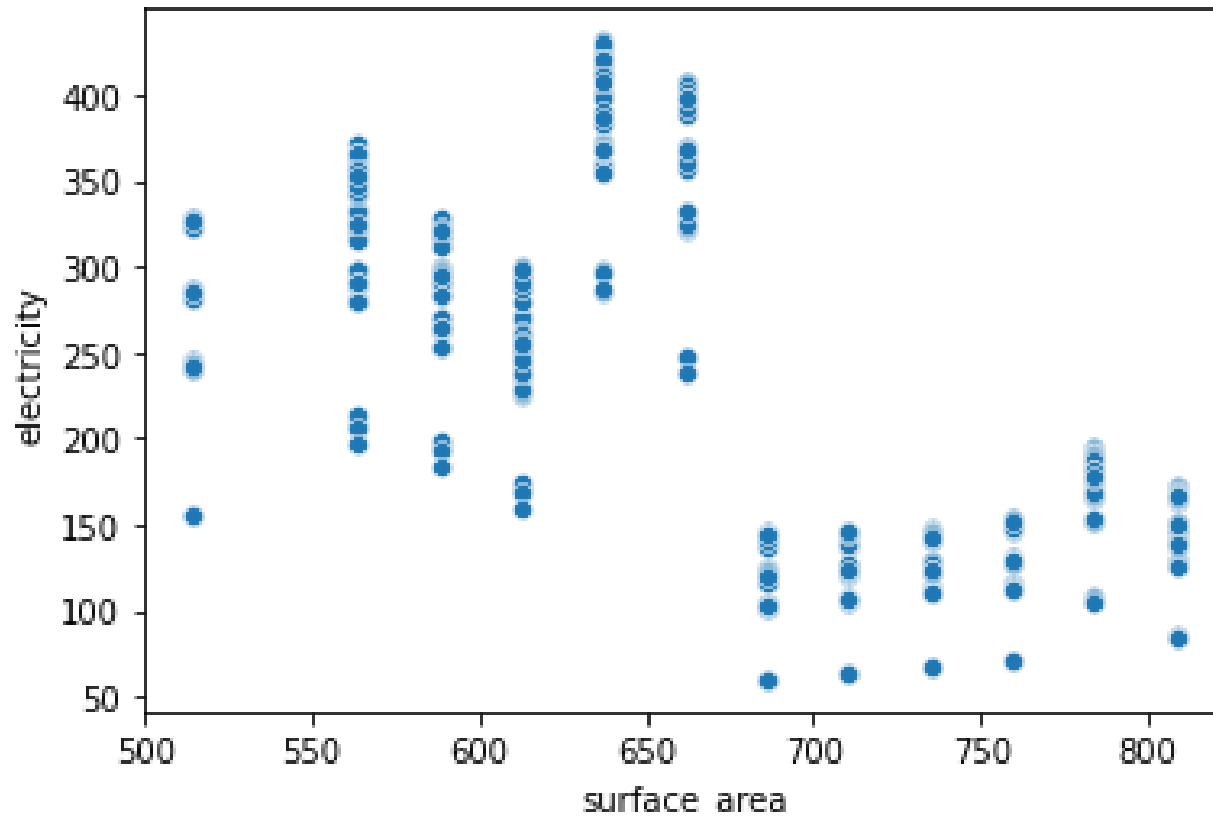
Model Capacity

Model Validation



Model Capacity 1/6

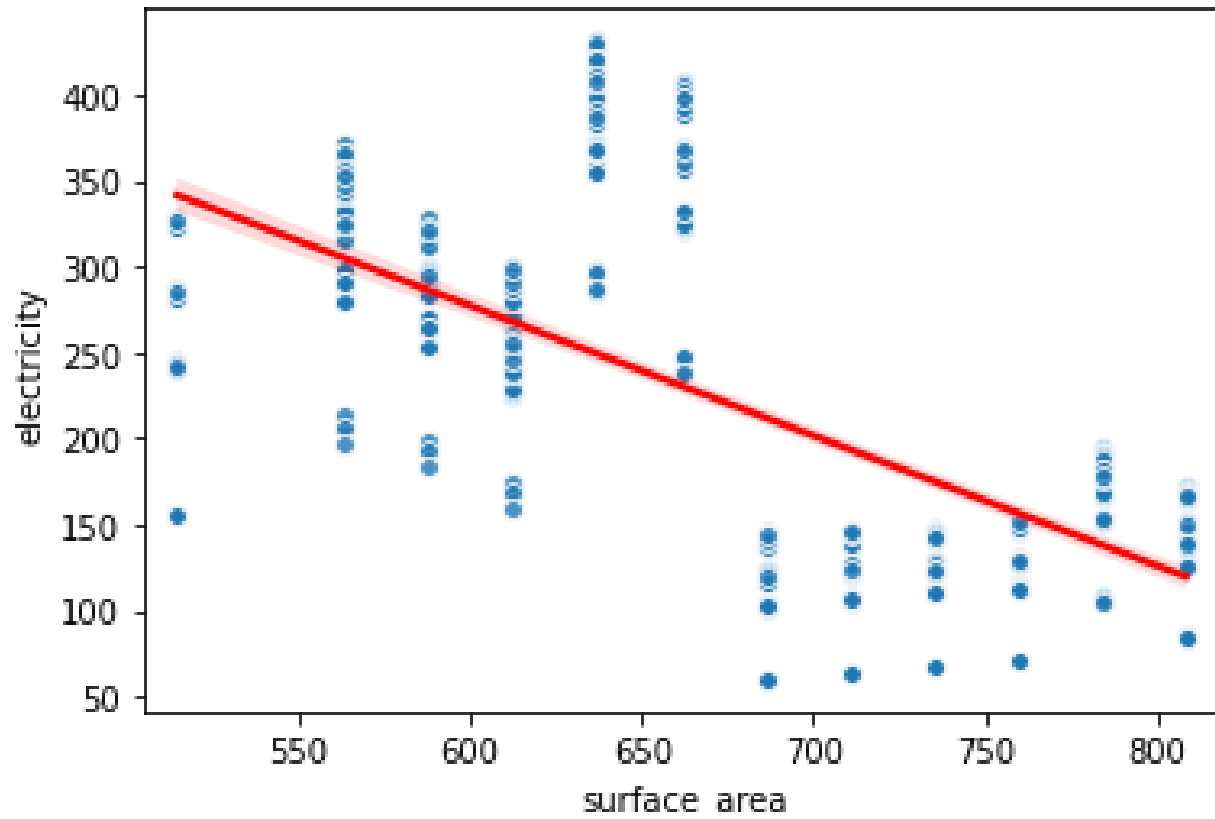
- Data 산점도



Model Validation

Model Capacity 2/6

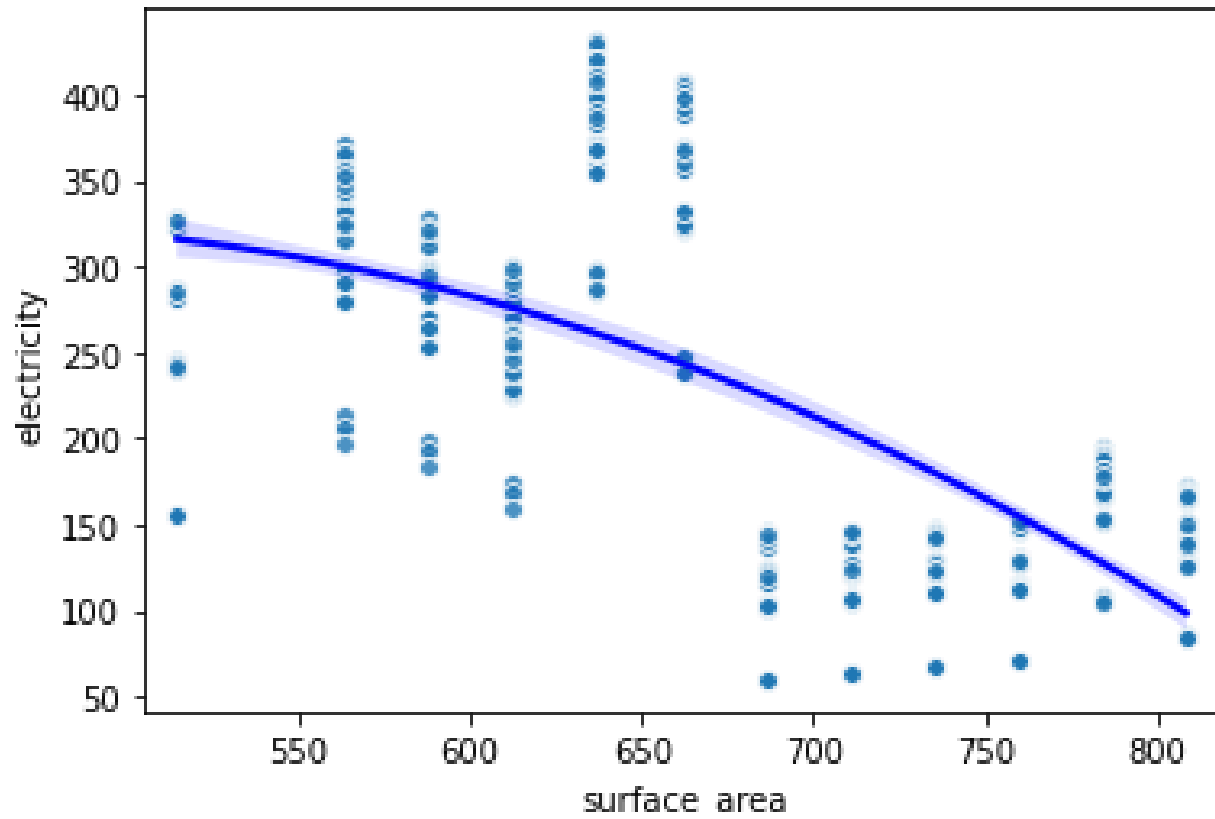
- 1차 모델 시각화



Model Validation

Model Capacity 3/6

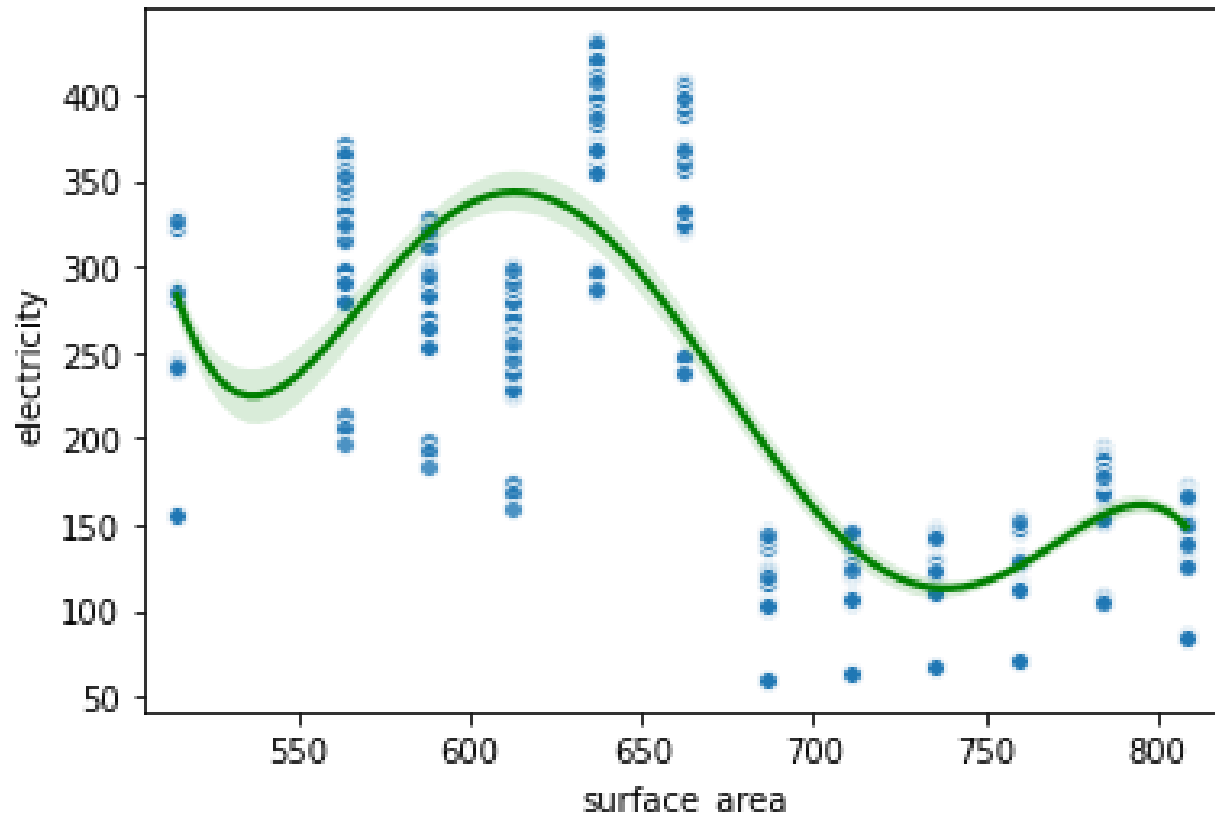
- 2차 모델 시각화



Model Validation

Model Capacity 4/6

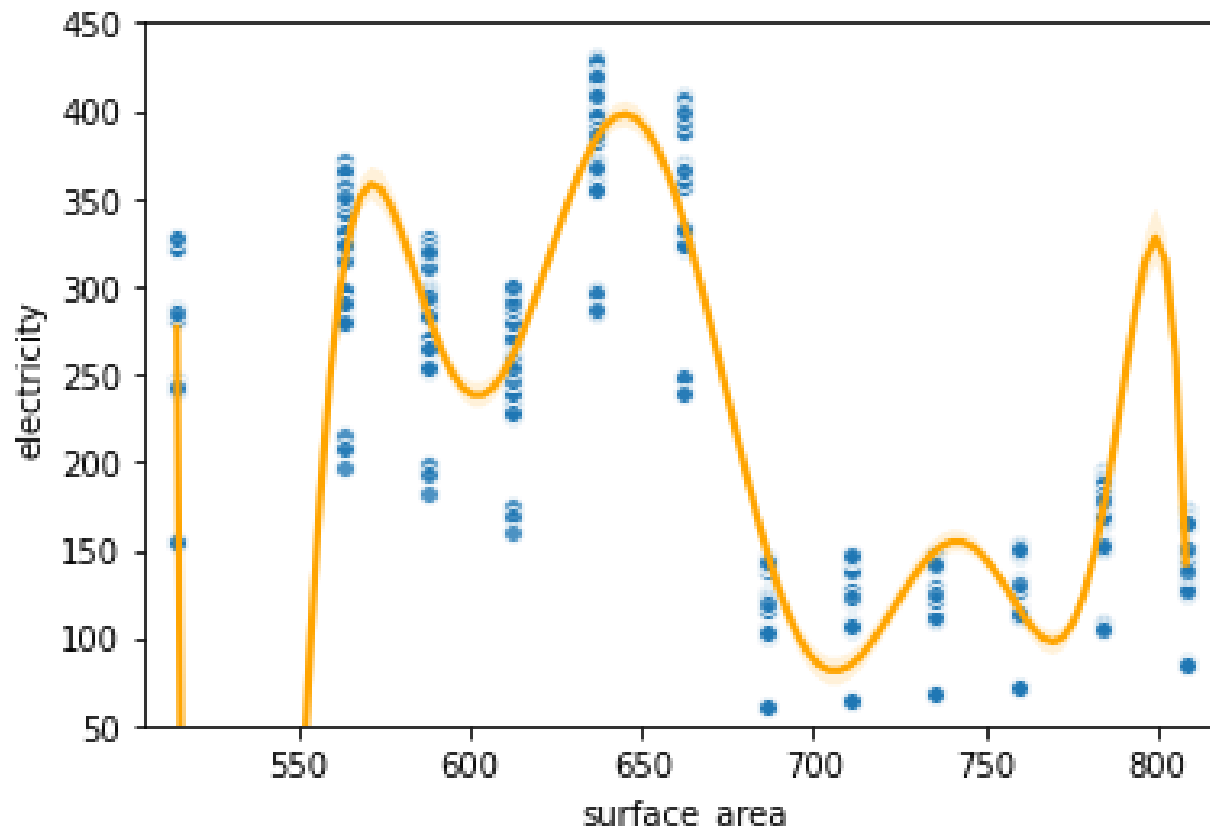
- 5차 모델 시각화



Model Validation

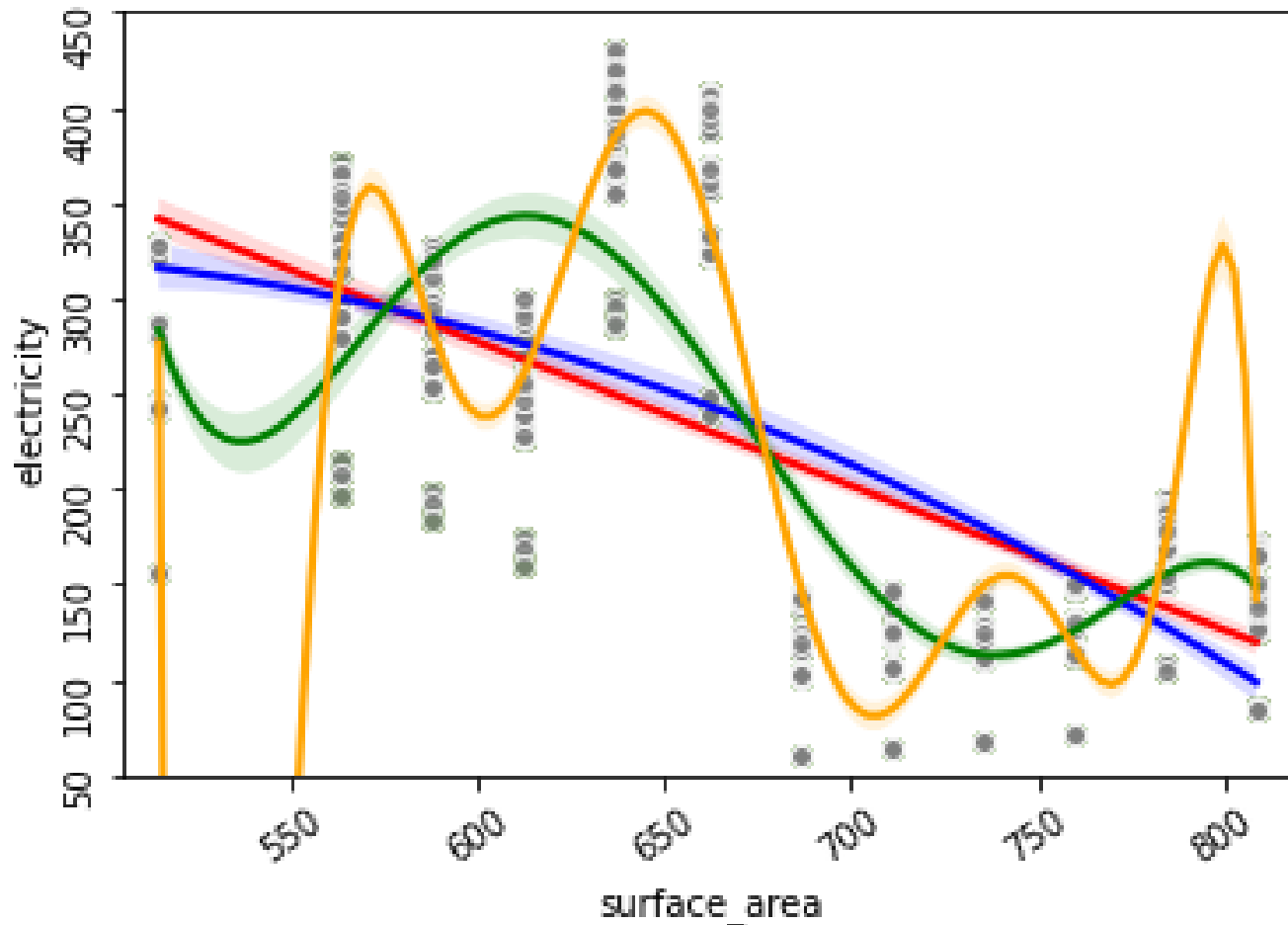
Model Capacity 5/6

- 9차 모델 시각화



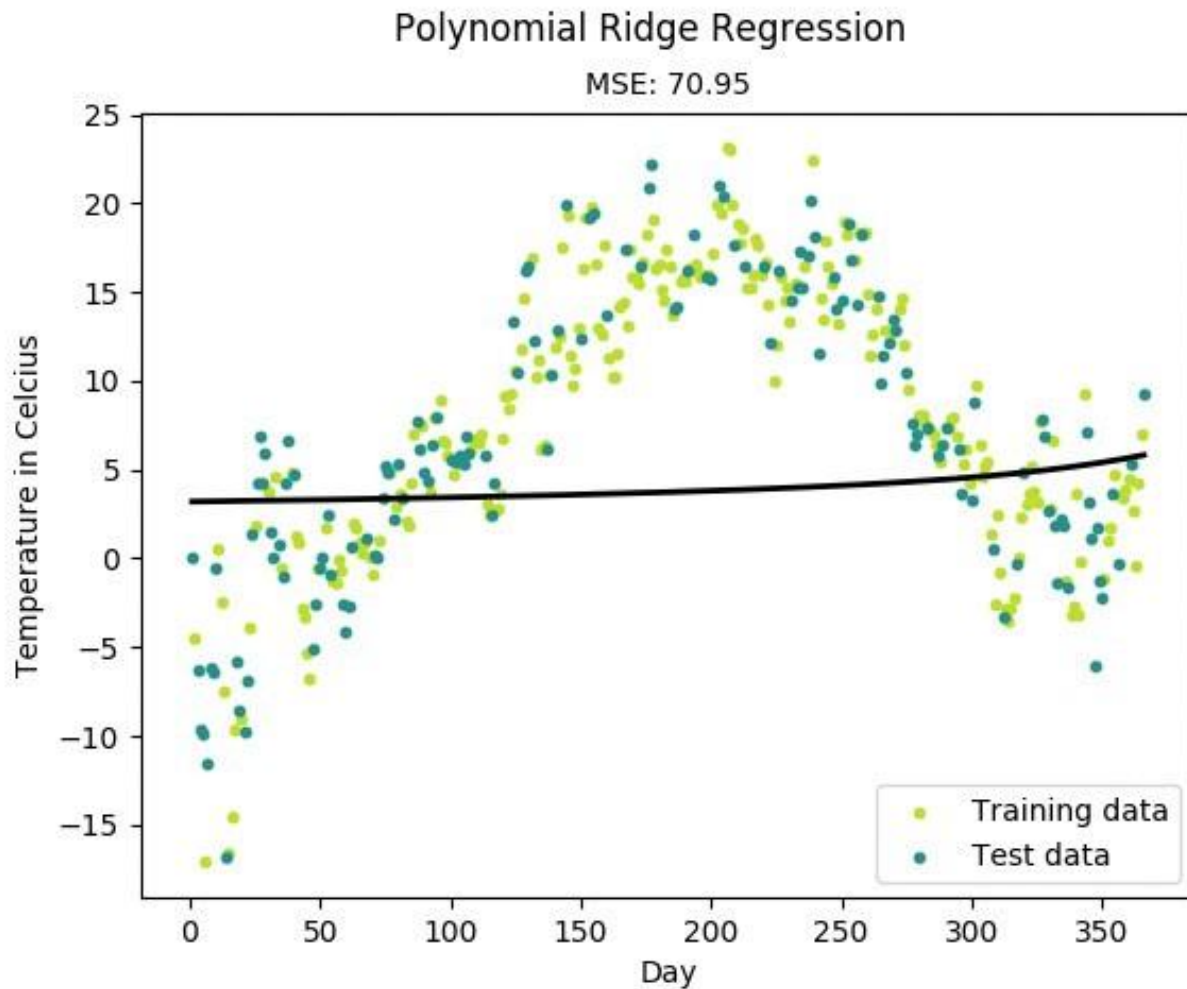
Model Validation

Model Capacity 6/6



Model Validation

Polynomial vs. MSE



Model Validation

scikit-learn

- Open-Source Machine Learning Library
- Simple and efficient tools for predictive data analysis
- <https://scikit-learn.org/stable>
- 주요 기능
 - Data Preprocessing
 - Model Selection
 - Regression Modeling (수치 예측)
 - Classification Modeling (분류)
 - Clustering Modeling (군집)
 - Dimensionality Reduction (차원 축소)



Training Error

Model Validation

Training Error^{1/2}

- 기본적으로 학습(Learning)은 Training Error를 최소화하는 방향으로 진행
- Training Error
 - Training Data에 Model을 적용하여 확인한 실제값과 예측값의 차이(Error)
 - $\text{mean}((y - \hat{y})^2)$
- 여러 개의 Model을 생성 후 각각의 Training Error(MSE)를 비교
 - 실무 데이터 적용을 위한 최적 Model 선택(Model Selection)
 - 하지만 Model 학습과 평가에 같은 데이터를 사용하면 부작용 발생 가능

Training Error ^{2/2}

- 3개 모델 Training Error 비교

```
[ ] print('1차 모델 : ', TR_Err_1)
    print('5차 모델 : ', TR_Err_5)
    print('9차 모델 : ', TR_Err_9)
```

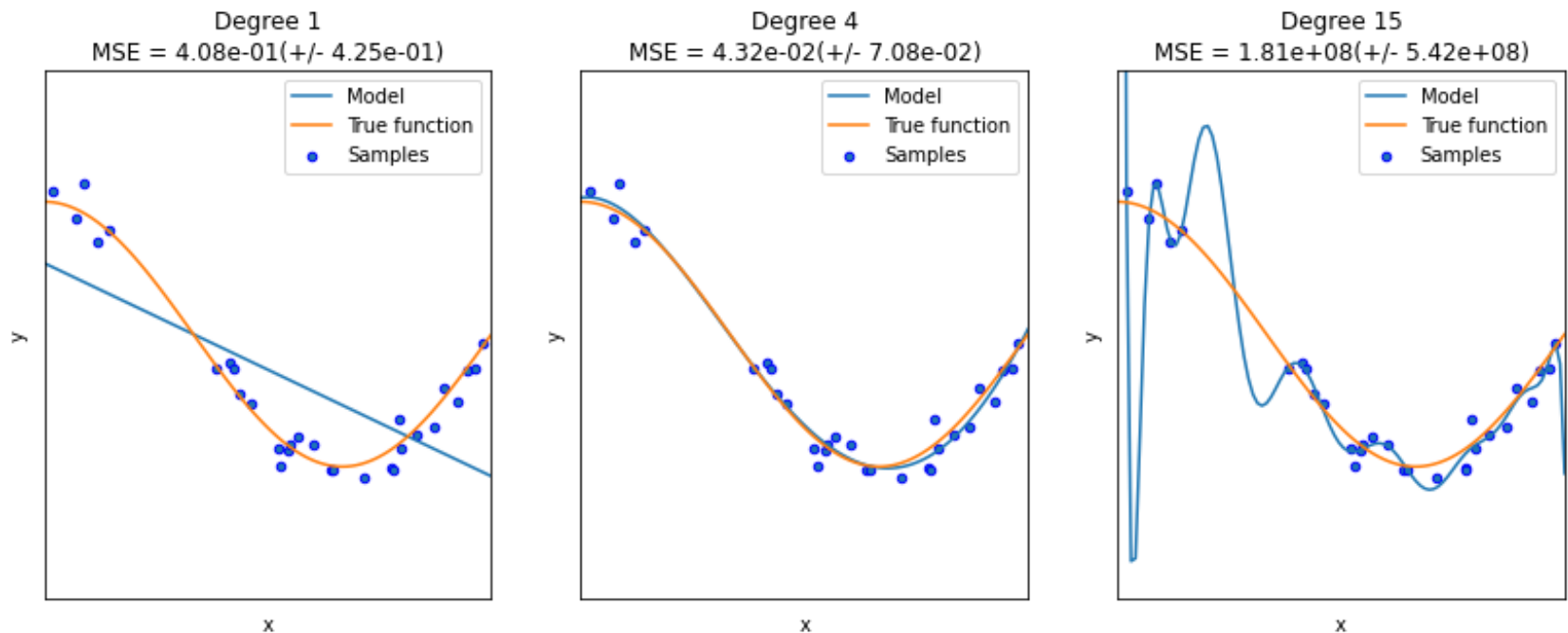
```
☞ 1차 모델 : 5763.983779426344
   5차 모델 : 4177.726328606005
   9차 모델 : 4086.7199908149655
```

Overfitting

Model Validation

Overfitting

- 학습 결과 **Training Data에만 최적화된 모델** 생성
 - 모델 생성 시 사용하지 않은 **Data에서 급격한 성능저하 발생**
- 부적합(Under Fit) vs. 최적합(Ideal Fit) vs. 과적합(Over Fit)



Testing Error

Model Validation

Testing Error^{1/5}

- 예측 모델 학습(모델 생성) 후 반드시 평가(모델 평가) 필요
- Testing Error 활용으로 모델 평가 객관성 측면 향상
 - Training Data: 학습(모델 생성)을 위해 제공되는 데이터
 - Testing Data: 학습 결과를 평가(모델 평가)하기 위해 제공되는 데이터
- 전체 데이터 크기에 따라 일반적으로 8:2 또는 7:3 등의 비율로 분할

Training Data

Testing Data

지도 학습(Supervised Learning)

- 컴퓨터에게 어떤 것이 맞는 답인지를 지정해 줌
- 목적 값(Target Value)이 있음
- 컴퓨터는 지정해 준 답과 비슷한 것을 판단해서 맞는 것이 무엇인지 판단함
 - 판단을 하기 위해 수많은 데이터를 활용하여 학습함
- 지도학습 방법은 기술적으로 분류(Classification), 예측(Regression)으로 구분됨

지도 학습(Supervised Learning)

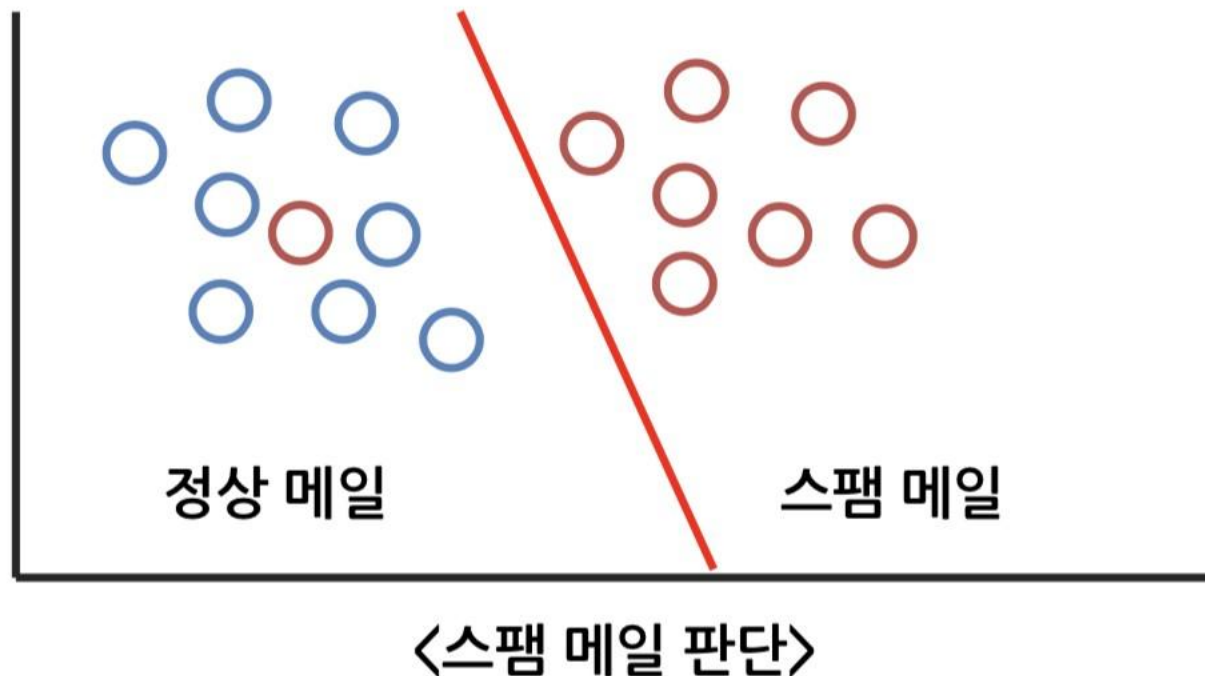
1) 분류

- 영어로는 Classification으로 불림
- 이전까지의 학습한 데이터를 기초로 컴퓨터가 결과를 판별하는 방법
- 목적 값(Target Value)의 연속성이 없이 몇 가지 값으로 분류됨
- 스팸 메일을 골라야 할 때, 컴퓨터는 학습된 데이터를 기초로 판단함
→ 스팸이다 / 스팸이 아니다

지도 학습(Supervised Learning)

2) 분류(Classification)의 예 - 스팸 메일 판단하기

- 수많은 스팸 메일을 컴퓨터가 학습하게 하고 학습한 데이터에 의해 컴퓨터는 일정 조건을 만들고 그에 맞게 판단함



지도 학습(Supervised Learning)

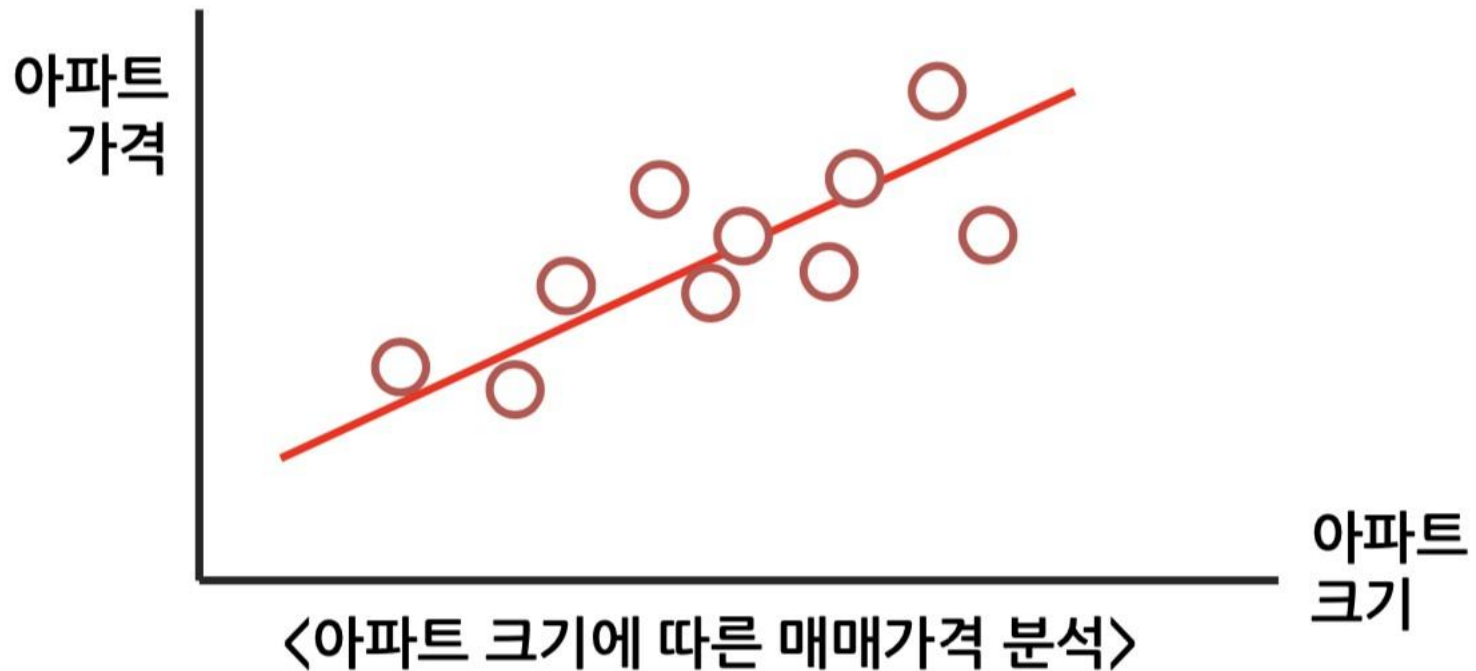
3) 예측(Regression)

- 일종의 회귀분석
- $F(x) = x + 2$ 와 같이 단순한 선형 회귀분석으로 예측할 수 있음
- 목적 값(Target Value)의 연속성이 있음
- 아파트 크기에 따른 매매 가격은 연속성이 있음
→ 20평 2억, 30평 3억, 40평 4억...

지도 학습(Supervised Learning)

4) 예측(Regression)의 예 - 크기에 따른 아파트 매매 가격 분석

- 수많은 아파트 매매 가격 정보를 컴퓨터가 분석하고 학습한 데이터에 의해 조건(아파트 크기)에 따라 그 결과(가격)을 보여줌



1. 기존의 프로그램 방식

- 회귀분석(linear regression) 설명 사이트 :

https://gbhat.com/machine_learning/linear_regression.html

- 오차함수 : MSE(오차제곱평균), RMSE(루트를 취하기 때문에 MSE의 단점이 어느정도 해소. 이상치에 덜 민감), MAE(mean absolute error ; 절대값평균)

-

<https://jysden.medium.com/%EC%96%B8%EC%A0%9C-mse-mae-rmse%EB%A5%BC-%EC%82%AC%EC%9A%A9%ED%95%98%EB%8A%94%EA%B0%80-c473bd831c62>

![image](https://miro.medium.com/v2/resize:fit:720/format:webp/1*XRXgMqrr5rq-V1rW7SkrtA.png)

- 경사하강법 설명
 - <https://www.mql5.com/ko/articles/11200>

지도 학습(Supervised Learning)

◆ 지도학습 모델 실습

1. 분석 데이터 준비 및 계획

1) 무엇을 분석할 것인가?

2) 분석을 위한 준비

3) 데이터 준비

2. 데이터 분석 및 결과 요약

지도 학습 - KNN

- kNN(k Nearest Neighbors) : 최근접 이웃 알고리즘

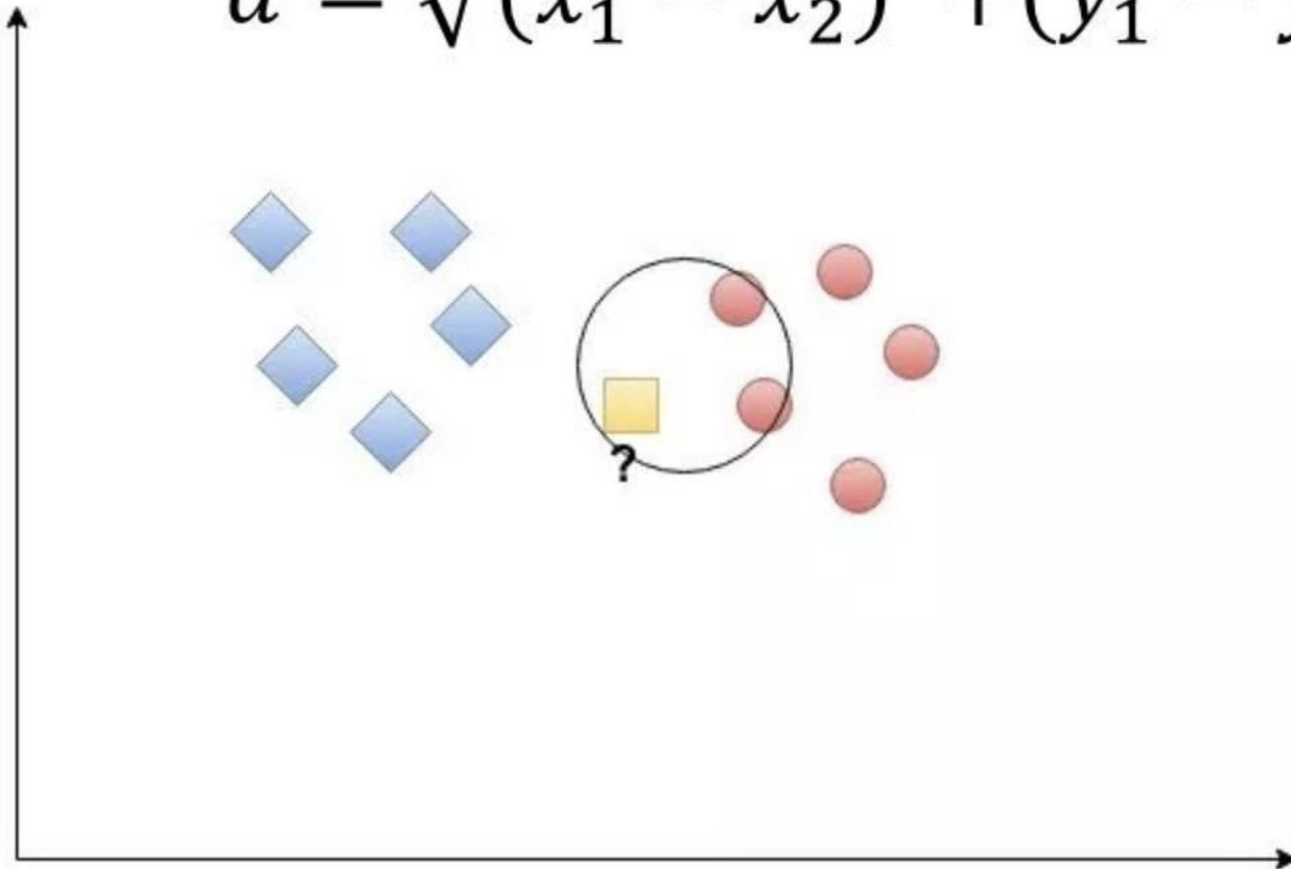
➡ 새로운 데이터의 분류를 알기 위해 사용



k 값에 의해 결정된 분류를 새로운 데이터의 분류로 확정

지도 학습 - KNN

$$d = \sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2}$$



지도 학습 - KNN

1) kNN의 장점

1 정확도가 높음

- 기존 분류 체계 값을 모두 검사하여 비교하므로 정확함

2 오류 데이터가 결과값에 크게 영향을 미치지 않음

- 비교하여 가까운 상위 k개의 데이터만 활용하므로 오류 데이터는 비교 대상에서 제외됨

3 데이터에 대한 가정이 없음

- 기존의 데이터를 기반으로 하므로, 가정이 아닌 실 데이터 기반으로 비교함

지도 학습 - KNN

2) kNN의 단점

1 느림

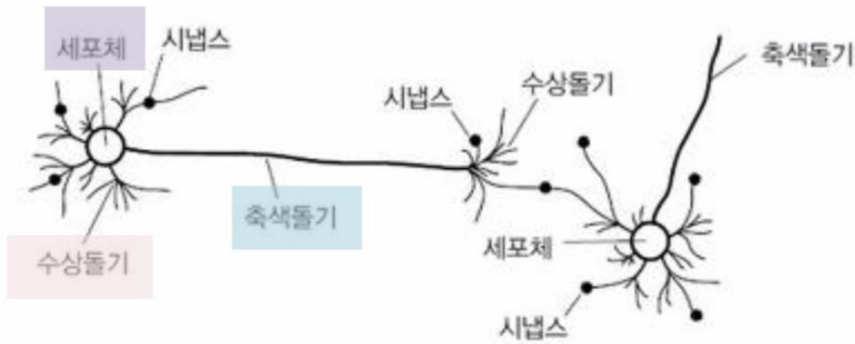
- 기존 모든 데이터를 비교해야 함
- 기존 데이터가 많을수록 더욱 느려짐
- 처리시간이 계속 증가함

2 많은 메모리를 사용함

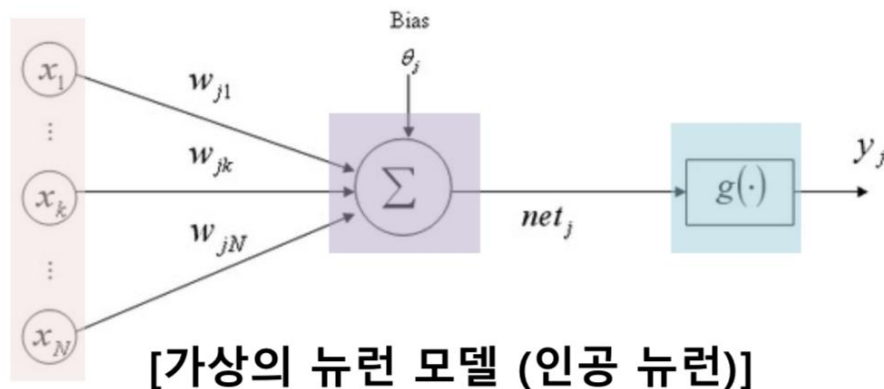
- 기존 데이터를 모두 활용해야 함
- 데이터 활용을 위해 메모리를 많이 사용함
- 데이터의 양에 따라 정확도는 올라가지만 고사양의 하드웨어가 필요함

딥러닝

사람의 뇌세포와 가상의 모델



[생물학적 뉴런(신경세포)]



[가상의 뉴런 모델 (인공 뉴런)]

생물학적 신경망	인공 뉴런
Dendrites (수상돌기)	Inputs
Cell Body (세포체 or Soma)	Neuron
Axon (축색돌기)	Outputs

뉴런



인간의 뉴런(neuron)

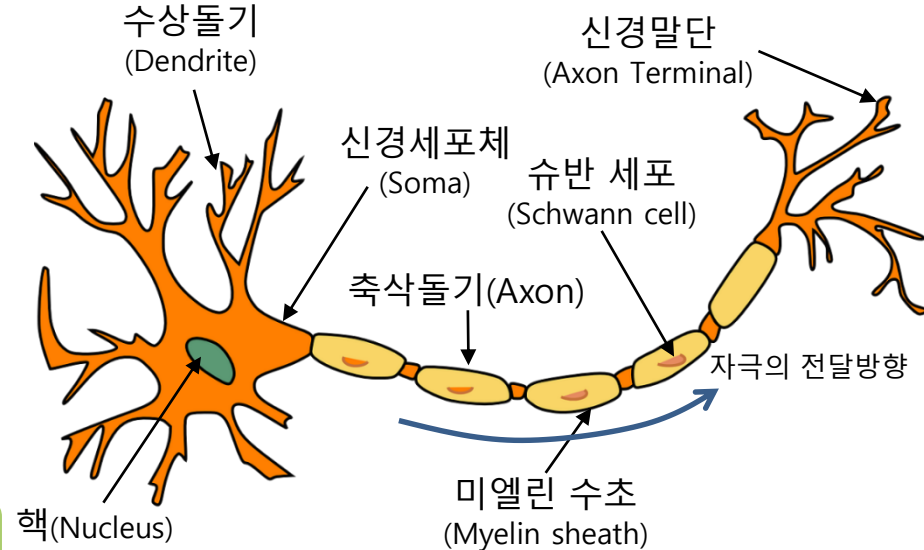
- ▶ 시냅스(synapse)를 통해 뉴런간 신호를 전달
- ▶ 뉴런은 수상돌기(dendrite)를 통해 입력 신호를 받음
- ▶ 입력 신호가 특정크기(threshold) 이상인 경우에만 활성화 되어 축삭돌기(axon)을 통해 다음 뉴런으로 전달



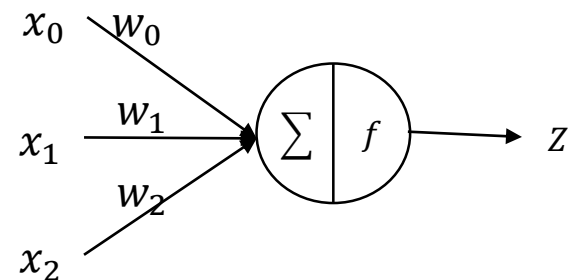
인공 뉴런(노드; node)

- ▶ 각 노드는 가중치가 있는 입력 신호를 받음
- ▶ 받은 신호는 모두 더한 후, 활성화 함수(activation function)를 적용함
- ▶ 활성화 함수의 값이 특정 값 이상인 경우에만 다음 노드의 입력값으로 전달

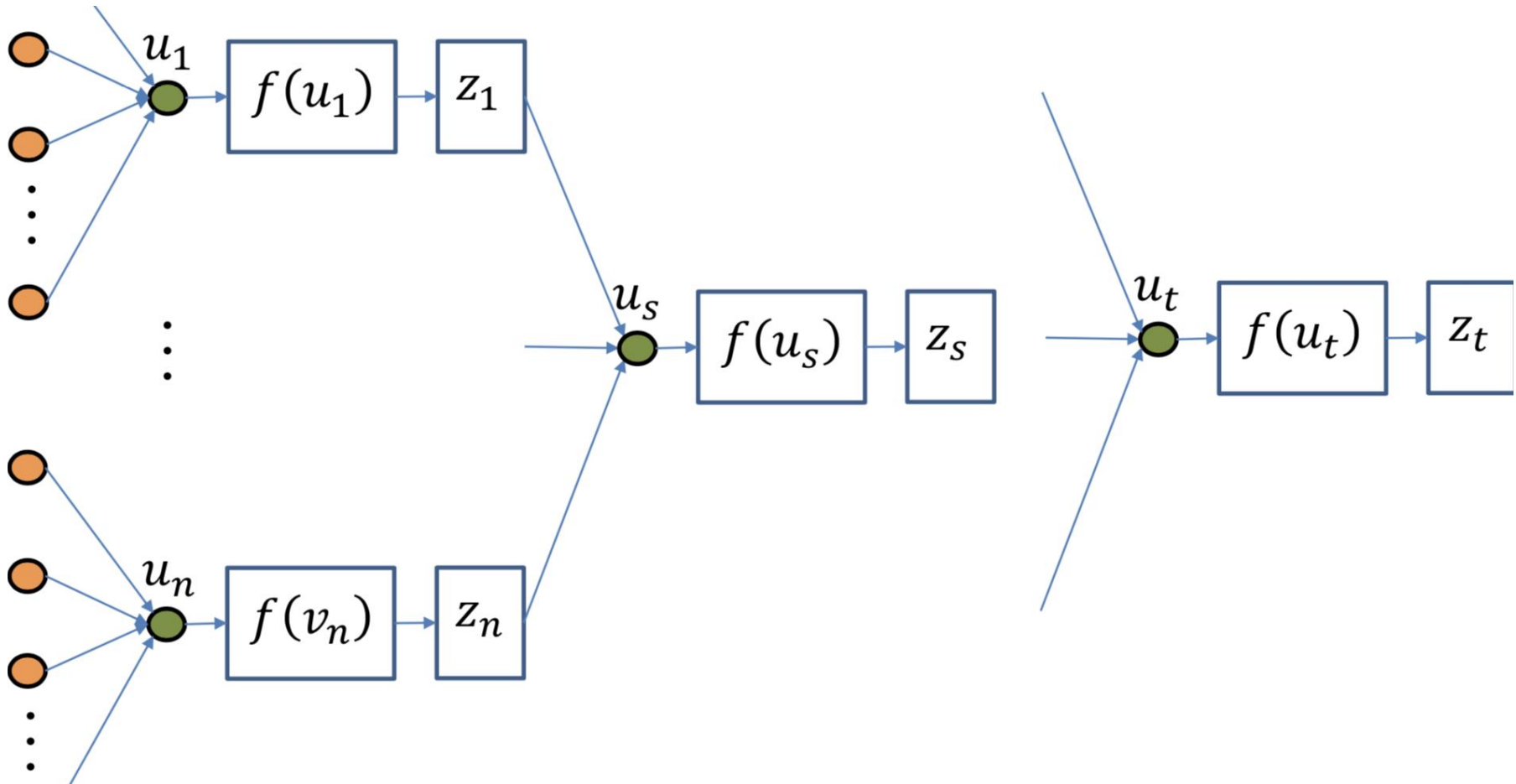
인간의 뉴런 구조



인공 뉴런의 구조

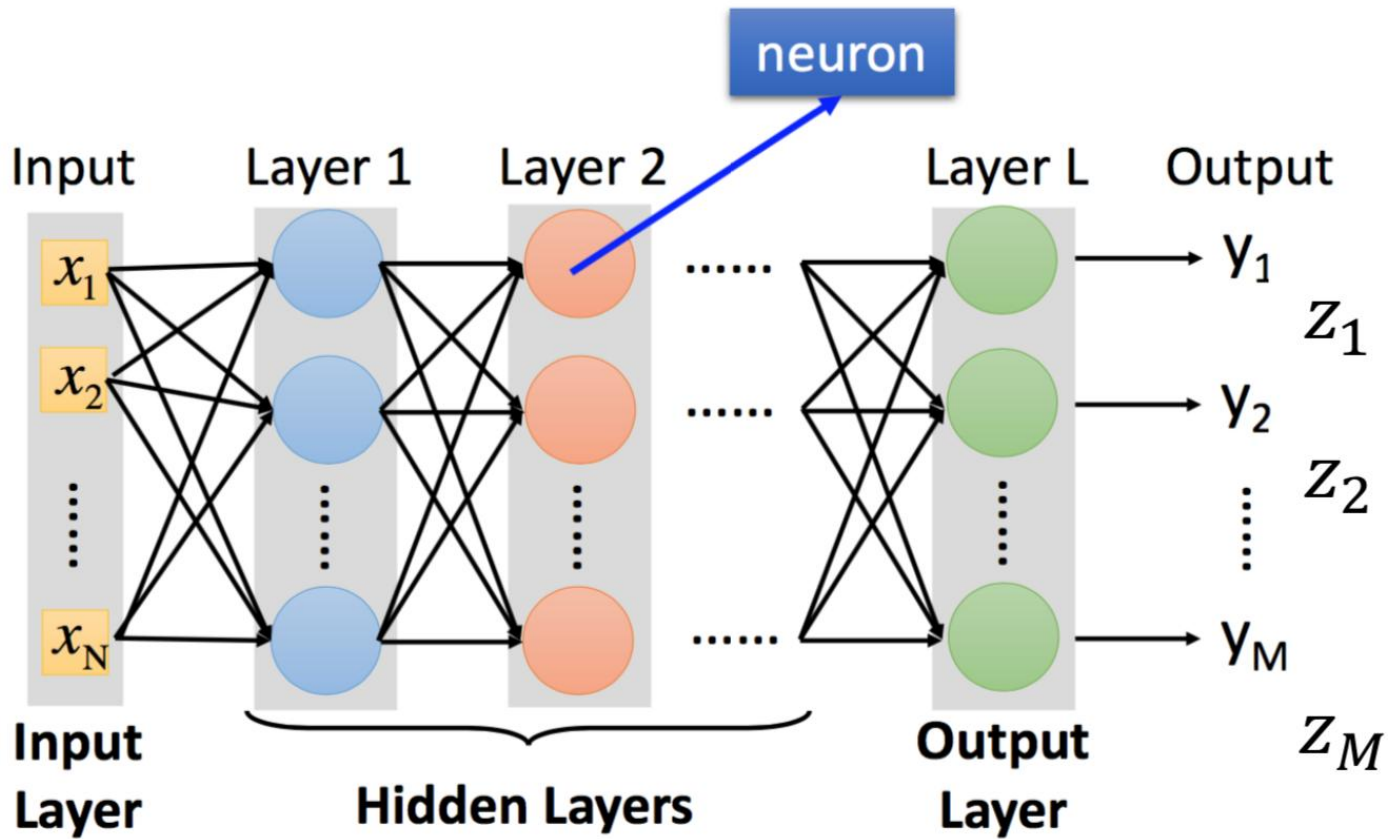


딥러닝 - 인공신경망



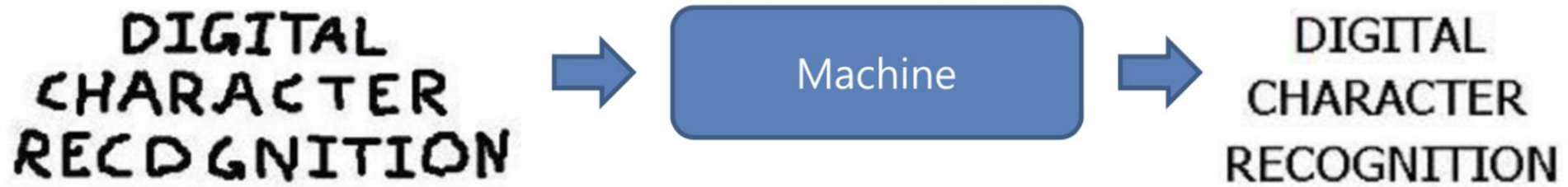
- 각 노드는 가중치가 있는 입력 신호를 받음
- 입력 신호는 모두 더한 후 이 신호가 전달될지 말지 활성화 함수 적용
- 활성화함수의 값이 특정 값 이상인 경우에만 다음 노드의 입력값으로 전달

딥러닝 - 인공신경망



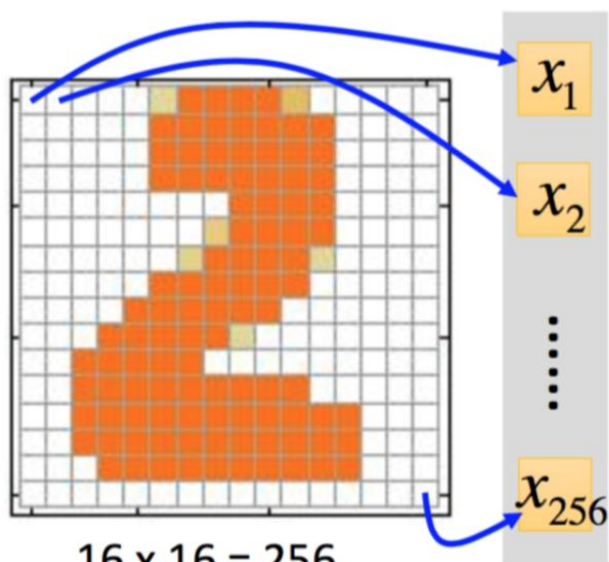
Deep Learning에서 'Deep' 의 의미:
매우 많은 Hidden Layers

딥러닝 예제 (손글씨 인식)



딥러닝 예제 (손글씨 인식)

Input

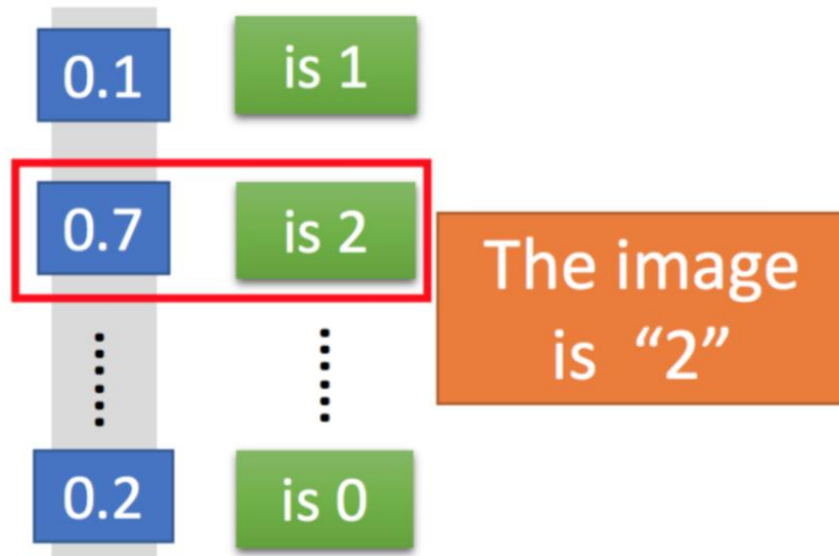


$16 \times 16 = 256$

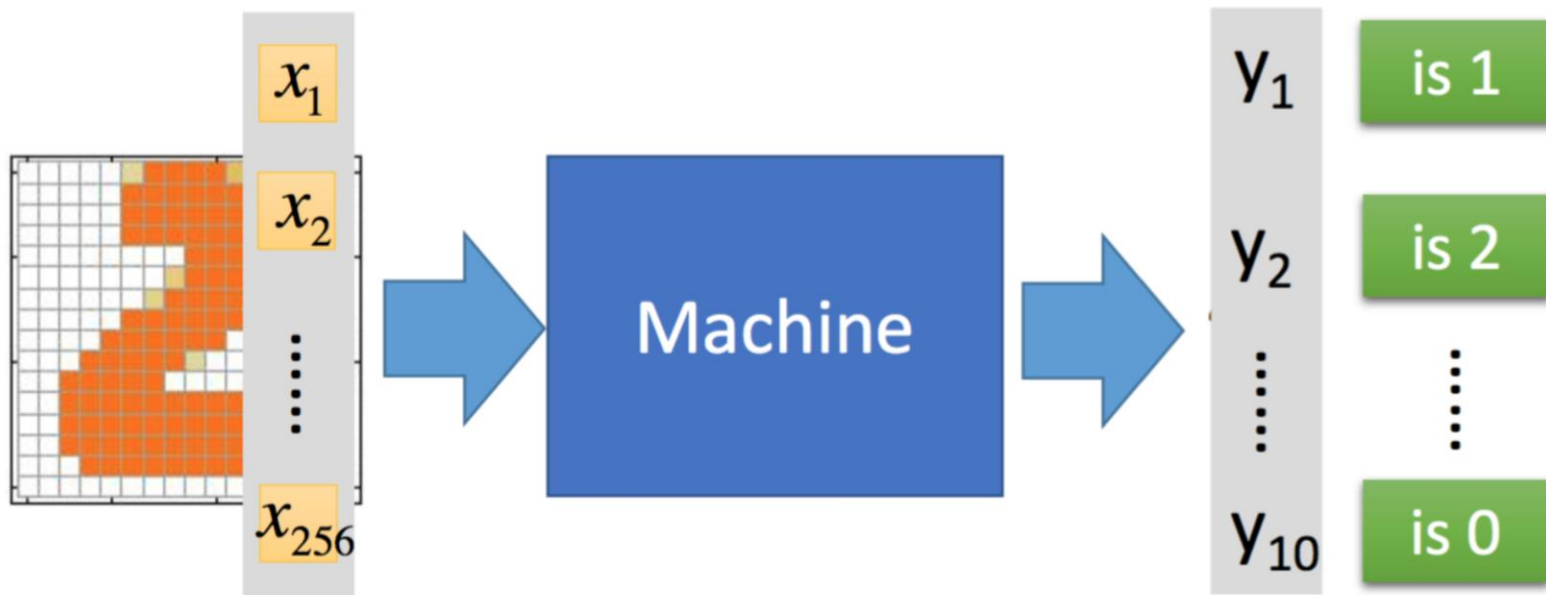
Ink $\rightarrow 1$

No ink $\rightarrow 0$

Output



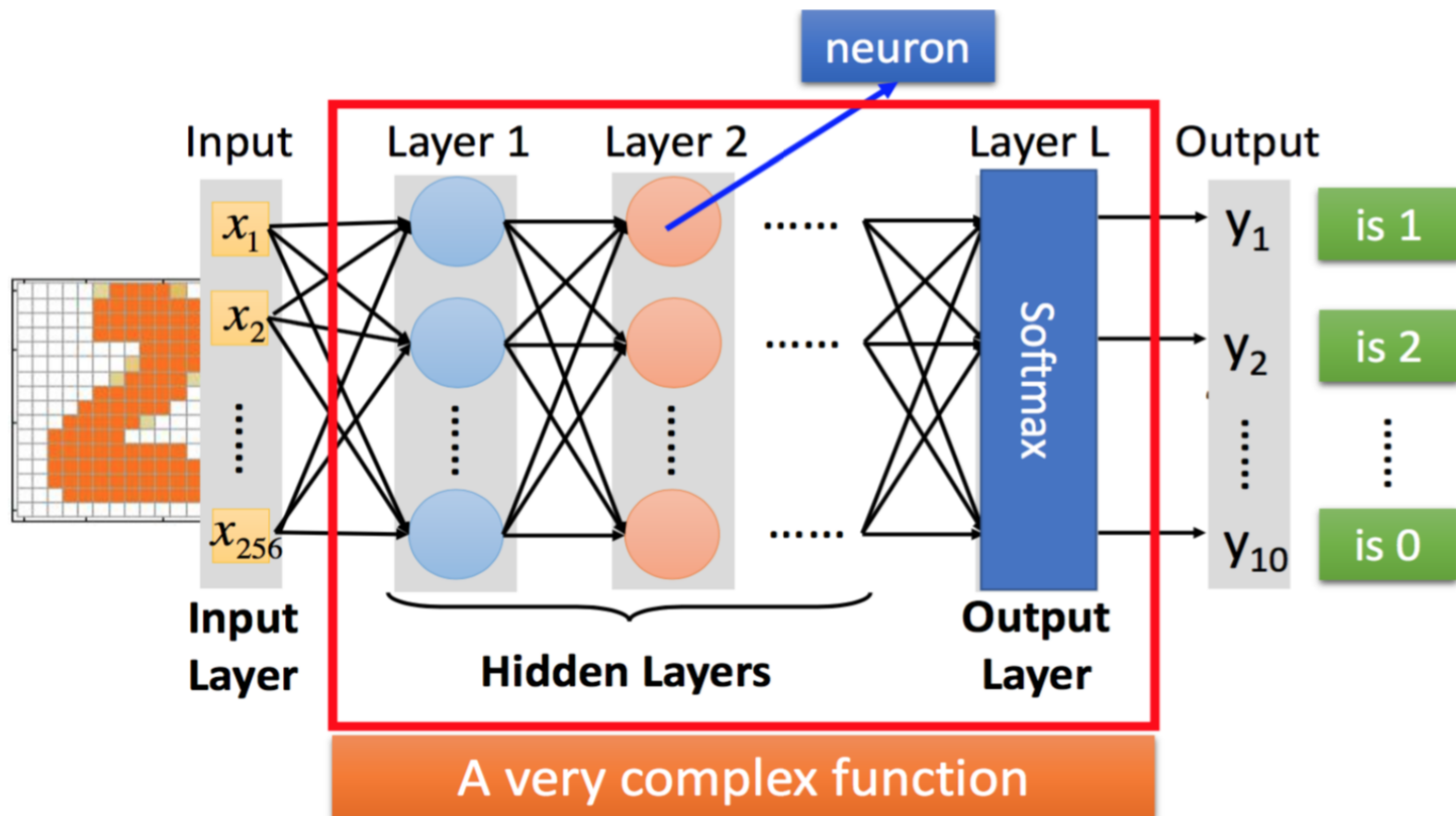
딥러닝 예제 (손글씨 인식)



Input:
256-dim vector

output:
10-dim vector

딥러닝 예제 (손글씨 인식)



딥러닝 예제 (손글씨 인식)

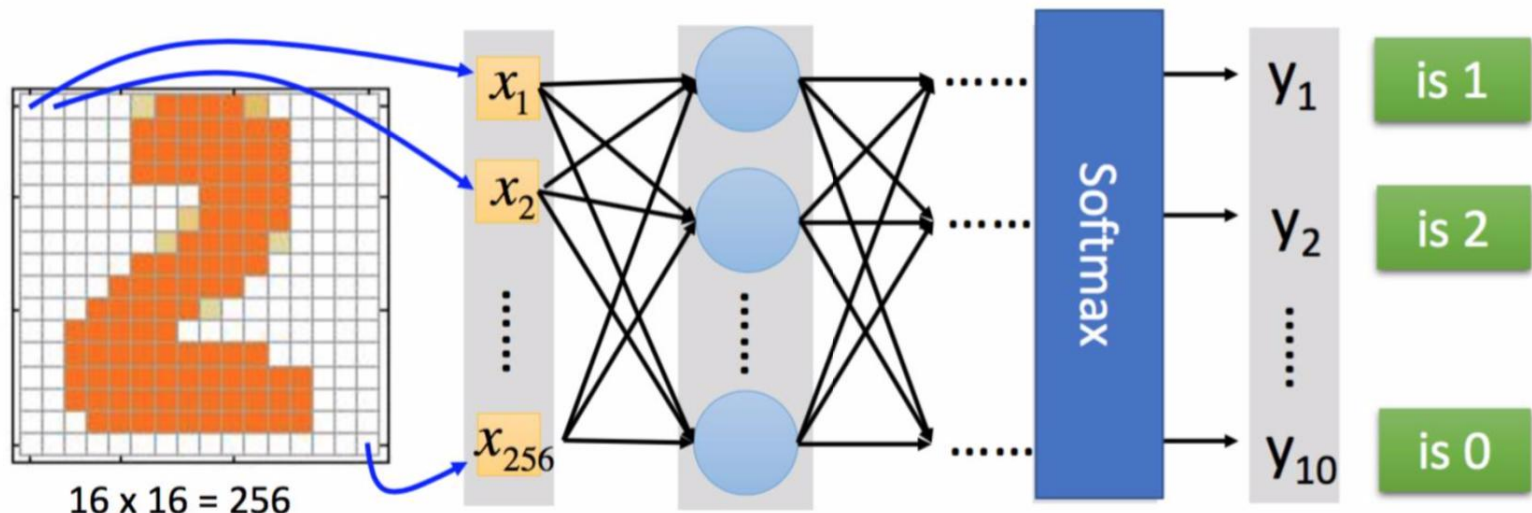
학습 데이터 준비

이미지 데이터와 각 이미지에 대한 레이블링

{2→2, 5→5, 4→8, 0→0, 2→2, 7→7, 5→5, 1→1,
3→3, 0→0, 3→3, 9→9, 6→6, 2→2, 8→8, 2→2,
0→0, 6→6, 6→6, 1→1, 1→1, 7→7, 8→8, 5→5,
0→0, 4→4, 7→7, 6→6, 0→0, 2→2, 5→5,
3→3, 1→1, 5→5, 6→6, 7→7, 5→5, 4→4, 1→1,
9→9, 3→3, 6→6, 8→8, 0→0, 9→9, 3→3,
0→0, 3→3, 7→7, 4→4, 4→4, 3→3, 8→8, 0→0,
4→4, 1→1, 3→3, 7→7, 6→6, 4→4, 7→7, 2→2,
7→7, 2→2, 5→5, 2→2, 0→0, 9→9, 8→8, 9→9,
8→8, 1→1, 6→6, 4→4, 8→8, 5→5, 8→8,
0→0, 6→6, 7→7, 4→4, 5→5, 8→8, 4→4,
3→3, 1→1, 5→5, 1→1, 9→9, 9→9, 9→9, 2→2,
4→4, 7→7, 3→3, 1→1, 9→9, 2→2, 9→9, 6→6}]

딥러닝 예제 (손글씨 인식)

학습 목표 설정



딥러닝 예제 (손글씨 인식)

이상적인 학습 목표

The learning target is

Input:  → y_1 이 가장 큰 값을 가져야 함

Input:  → y_2 가 가장 큰 값을 가져야 함

1
0
0
.
.
.
0

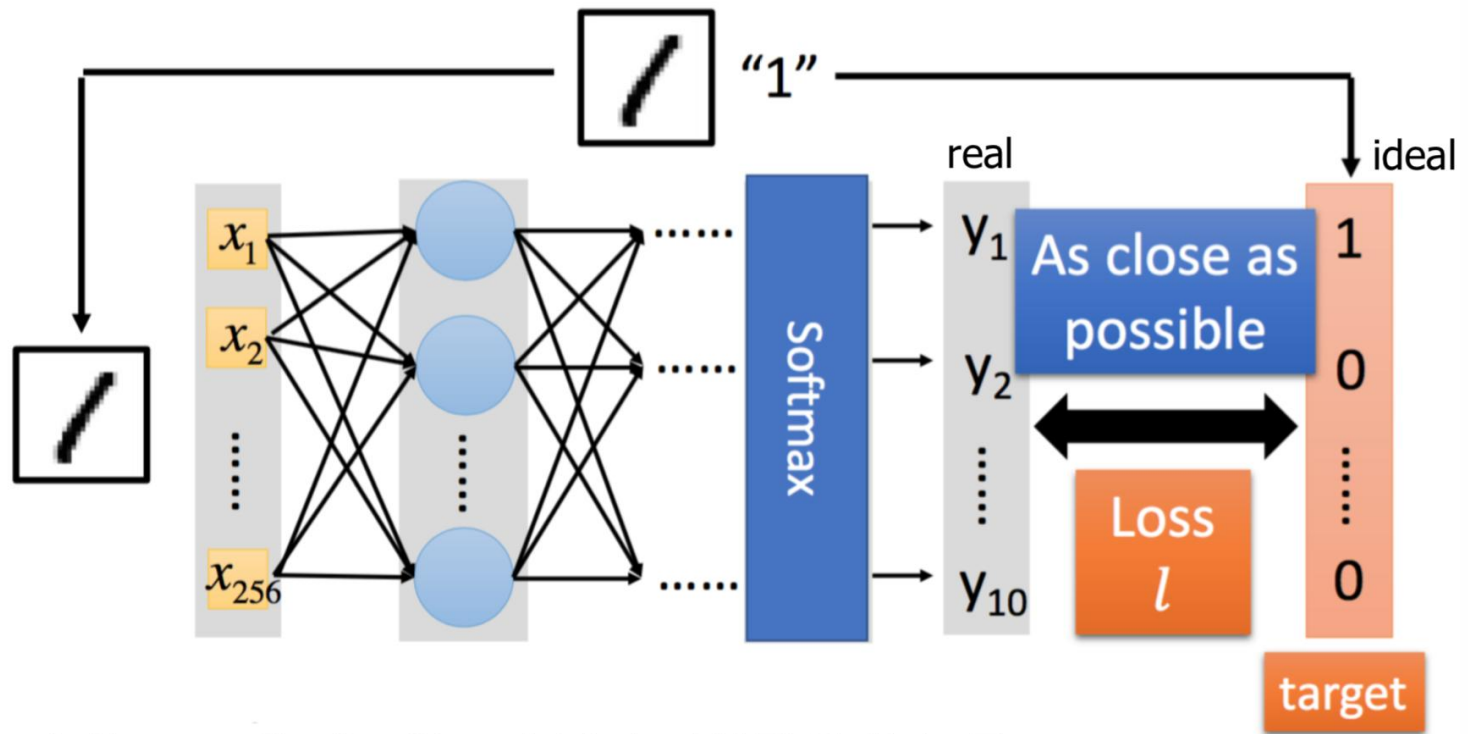
10-dim
Vector

0
1
0
.
.
.
0

10-dim
Vector

딥러닝 예제 (손글씨 인식)

이상적인 학습 목표와 손실(Loss)

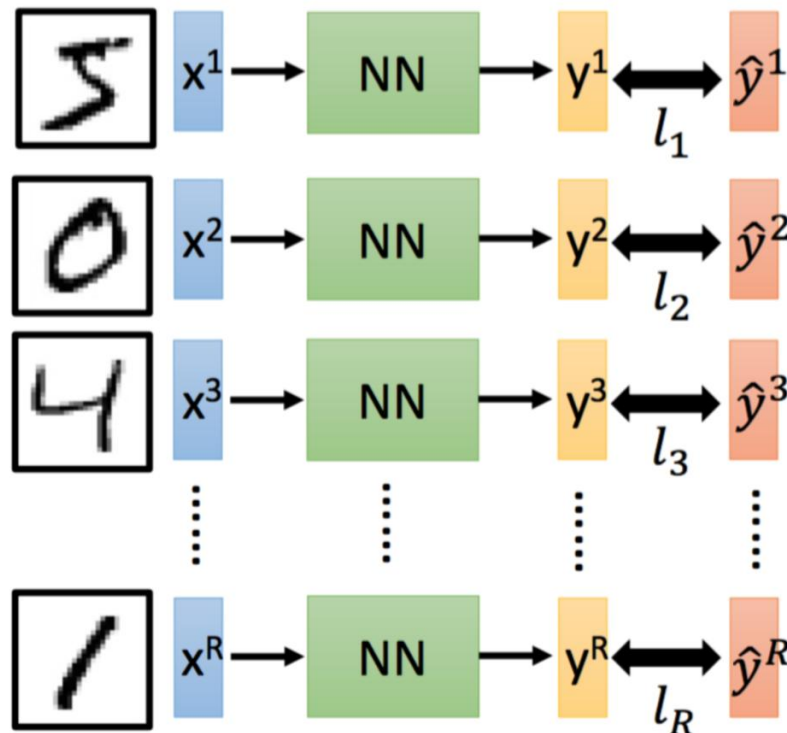


손실(Loss)은 네트워크 출력과 이상적인 학습 목표 사이의 차이값

딥러닝 예제 (손글씨 인식)

이상적인 학습 목표와 손실(Loss)

For all training data ...



Total Loss:

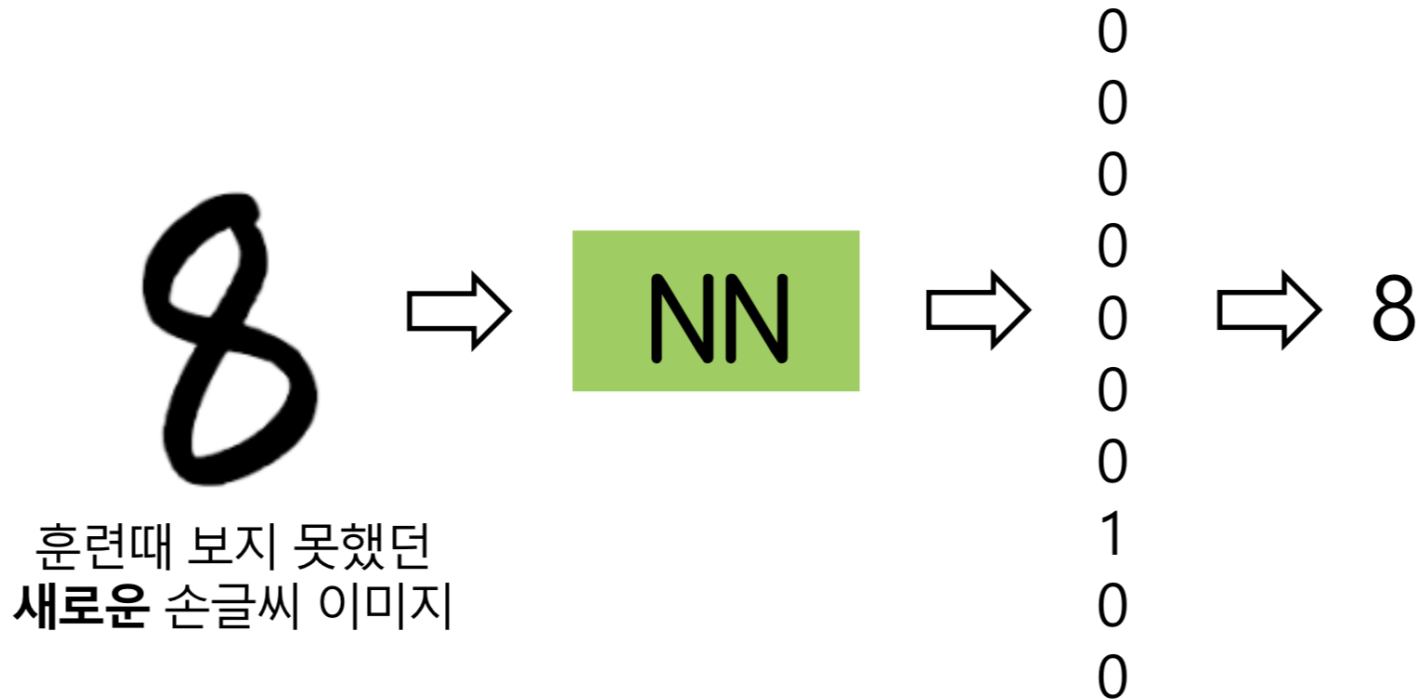
$$L = \sum_{r=1}^R l_r$$

학습 목표

Total Loss (L)을 최소화
할 수 있는 신경망 내부
의 전체 Weight와 Bias
구하기

딥러닝 예제 (손글씨 인식)

훈련이 완료된 신경망 모델을 실전에 활용



딥러닝 데이터 셋

(Part 2)

딥러닝 용어

◆ 옵티마이저(Optimizer)

- Gradient Decent
- Adam
- RMSProp

◆ 활성화 함수(Activation Function)

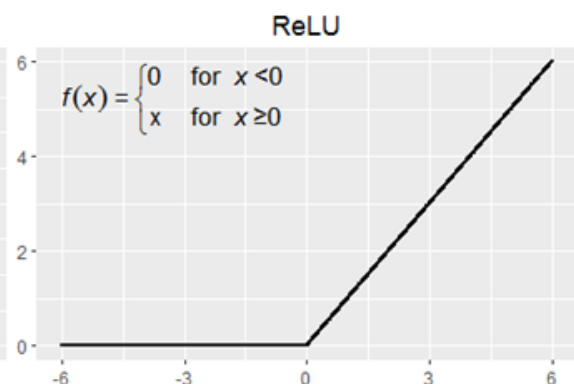
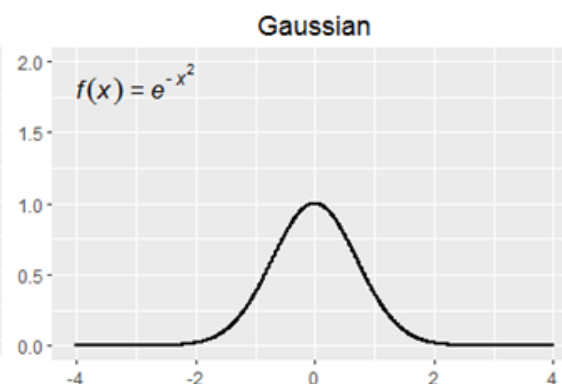
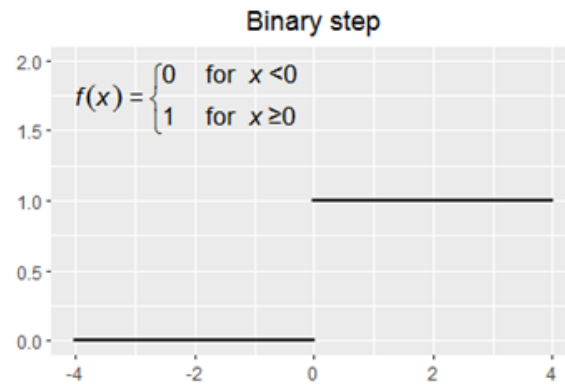
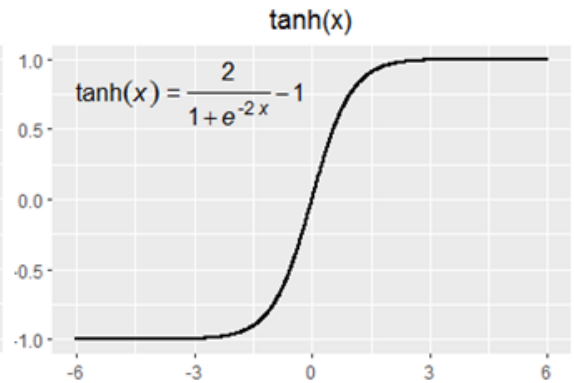
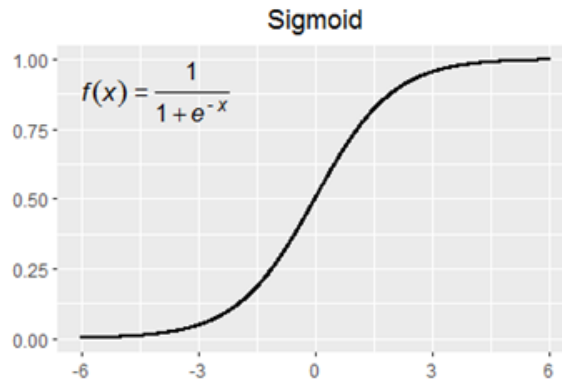
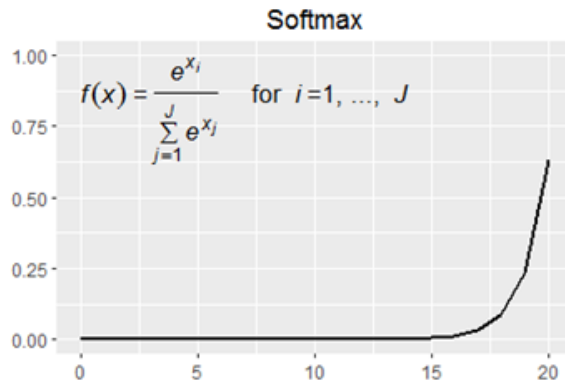
- ReLU
- Sigmoid
- Softmax

◆ Batch 학습

◆ DropOut

◆ LearningRate

활성화 함수(activation function)



텐서플로우 2.10

1. 데이터셋 생성
 - 데이터 전처리
2. 모델 구성
 - 모델 구성(레이어 추가)
3. 모델 학습 과정 설정 ; 어떻게 학습할지 설정
4. 모델 학습
5. 학습 과정 살펴보기
 - 손실, 정확도 등으로 학습 상황 파악 및 판단
6. 모델 평가하기
7. 모델 사용하기

데이터셋

- ◇ 경우 1
 - 예측불가..!

훈련셋

시험셋

모의
고사
1회

모의
고사
2회

모의
고사
3회

모의
고사
4회

모의
고사
5회

작년
수능
문제

올해
수능
문제

데이터셋

◇ 경우 2

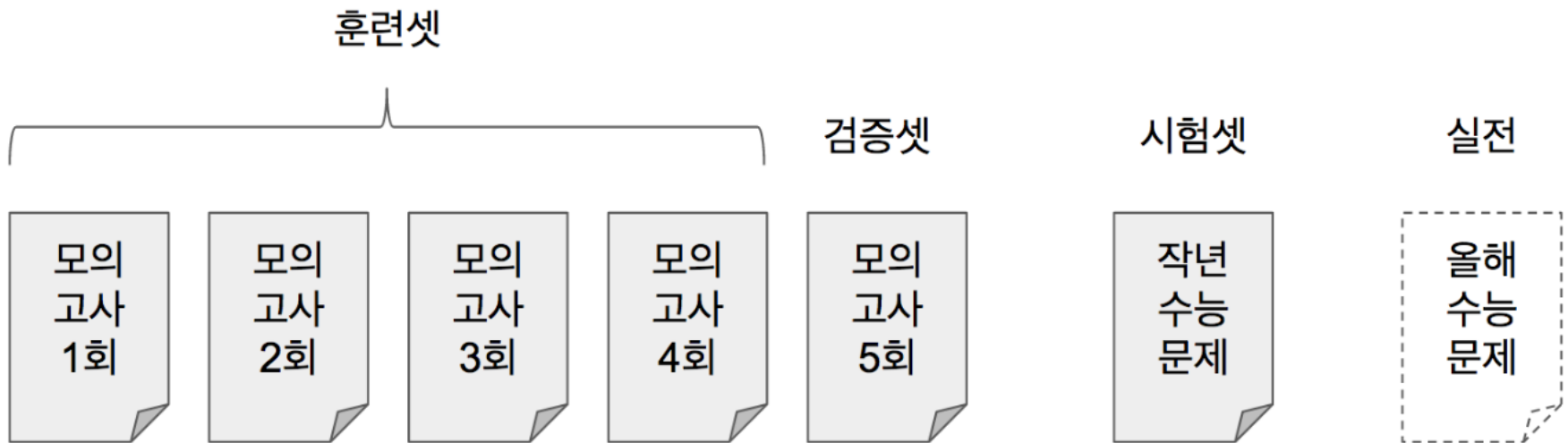
- 작년 수능문제는 훈련 하면 안 됨!



데이터셋

◇ 경우 3

- 검증을 통해 적절한 학습 방법을 찾아간다!



데이터셋

◇ 경우 4

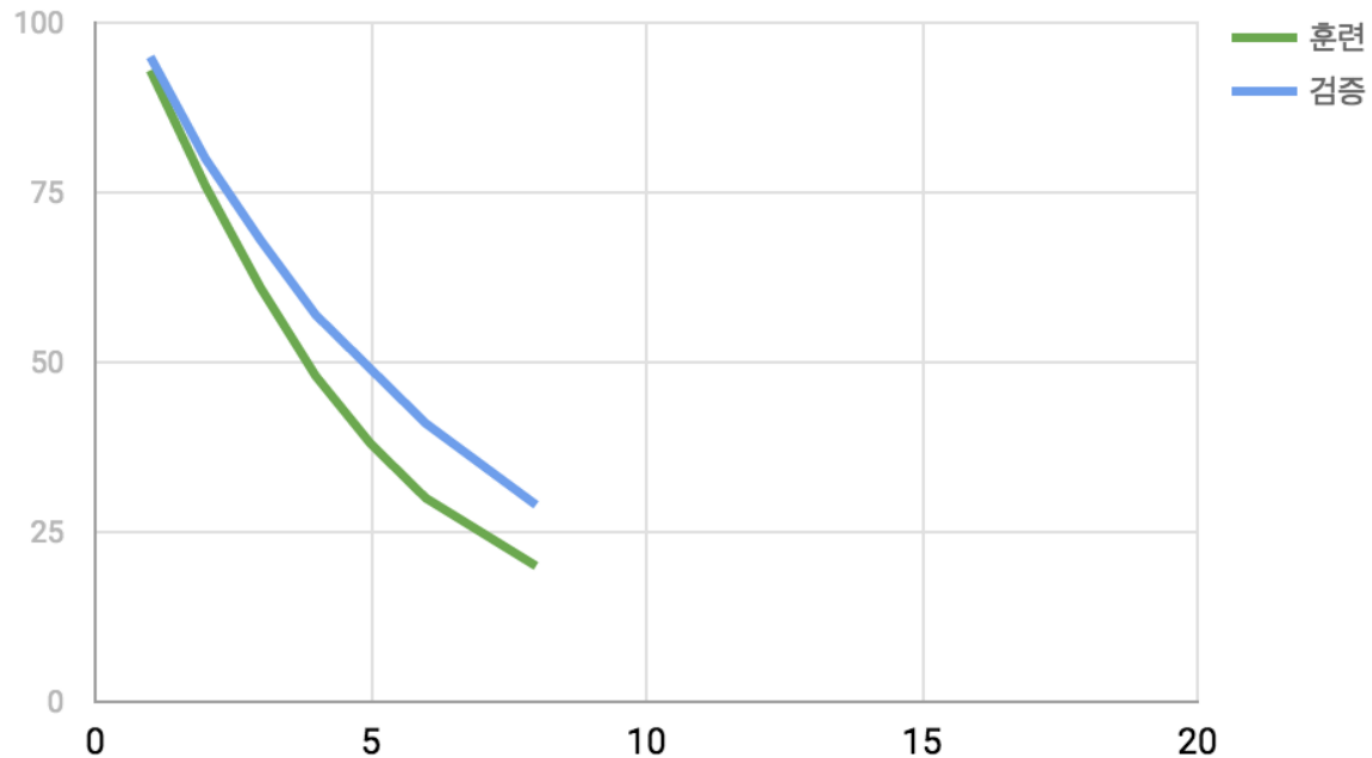
– 보다 안정적으로!



데이터셋

◇ 학습 결과

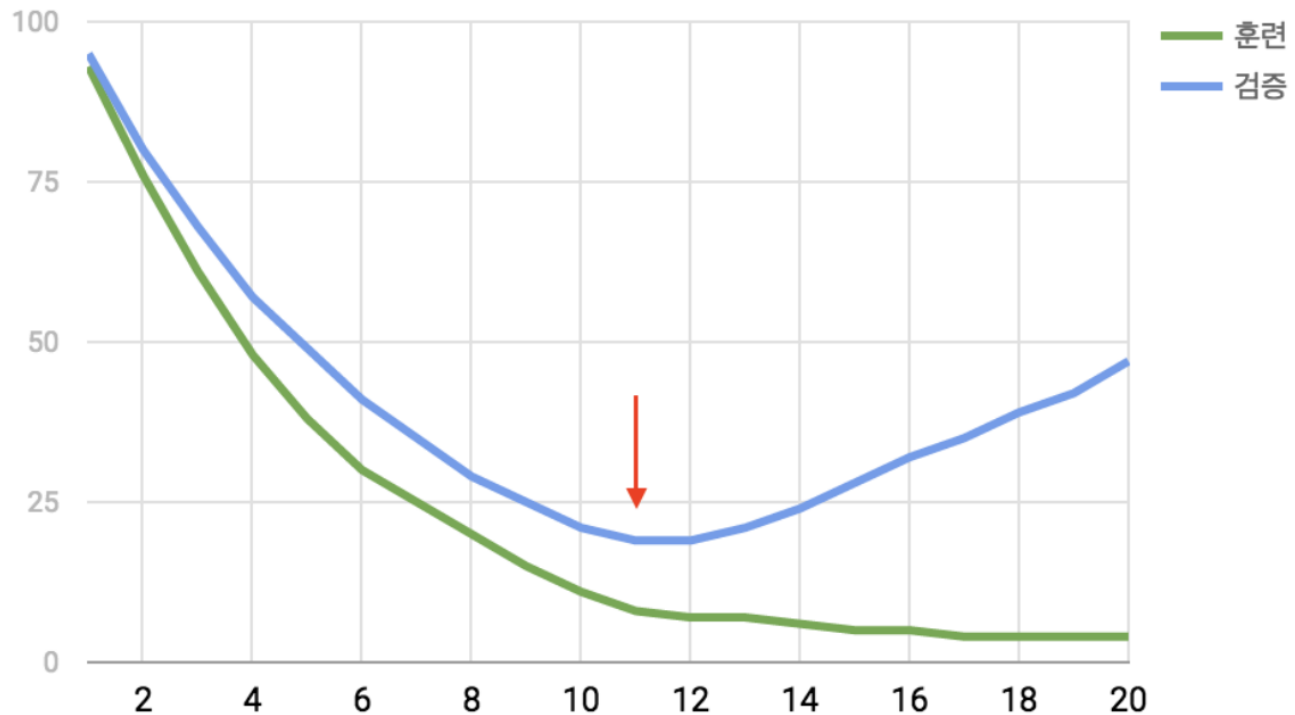
– 학습이 아직 덜 된 것 같다..!



데이터 셋

◆ 학습 결과

- 이 때 학습을 멈추면 최고! (오버피팅=과적합)



학습 과정 이야기

◆ 주요 인자

- x : 입력 데이터
- y : 라벨 값
- `batch_size` : 몇 개의 샘플로 가중치를 갱신할 것인지 지정
- `epochs` : 학습 반복 횟수

문제지

문제1	문제2	문제3	문제4	문제5	문제6	...	문제100
-----	-----	-----	-----	-----	-----	-----	-------

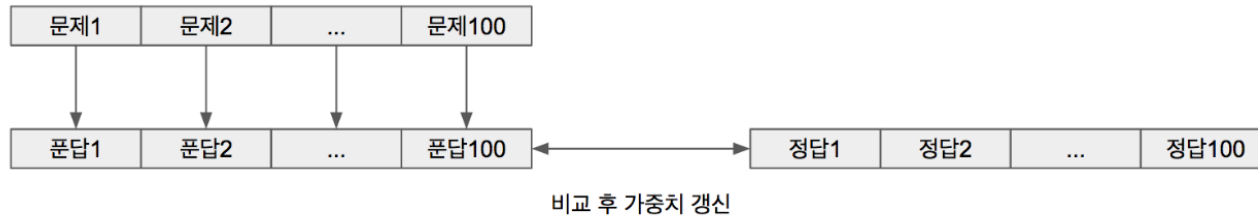
해답지

정답1	정답2	정답3	정답4	정답5	정답6	...	정답100
-----	-----	-----	-----	-----	-----	-----	-------

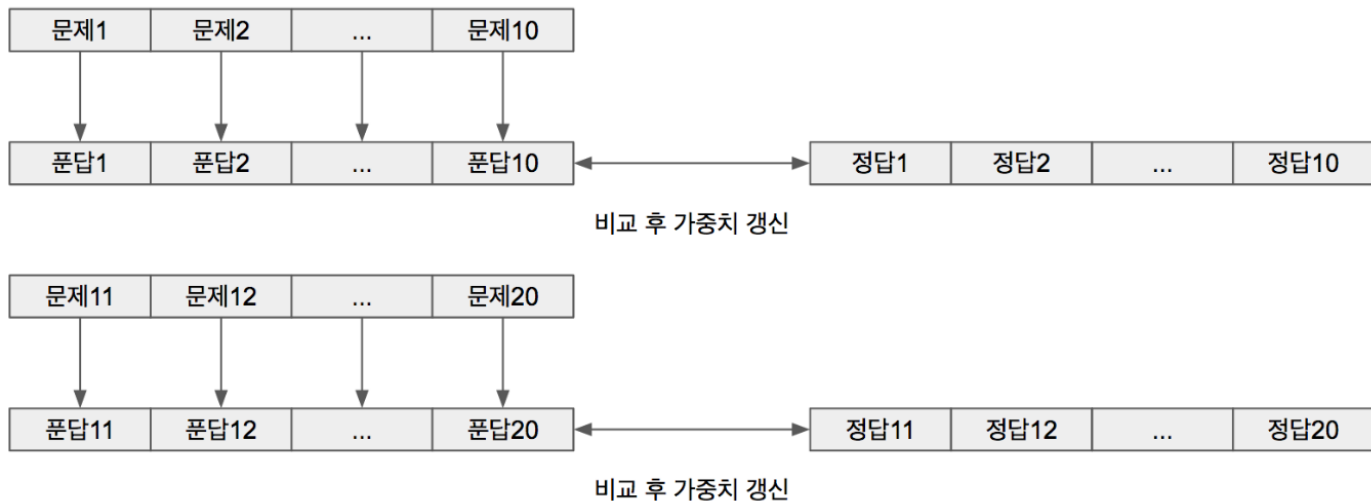
학습 과정 이야기

◆ 배치 사이즈

– 배치 100



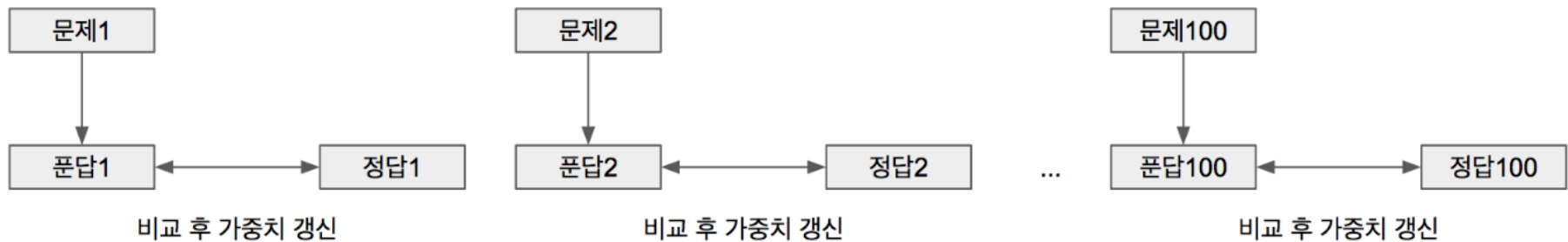
– 배치 10



학습 과정 이야기

◇ 배치 사이즈

– 배치 1



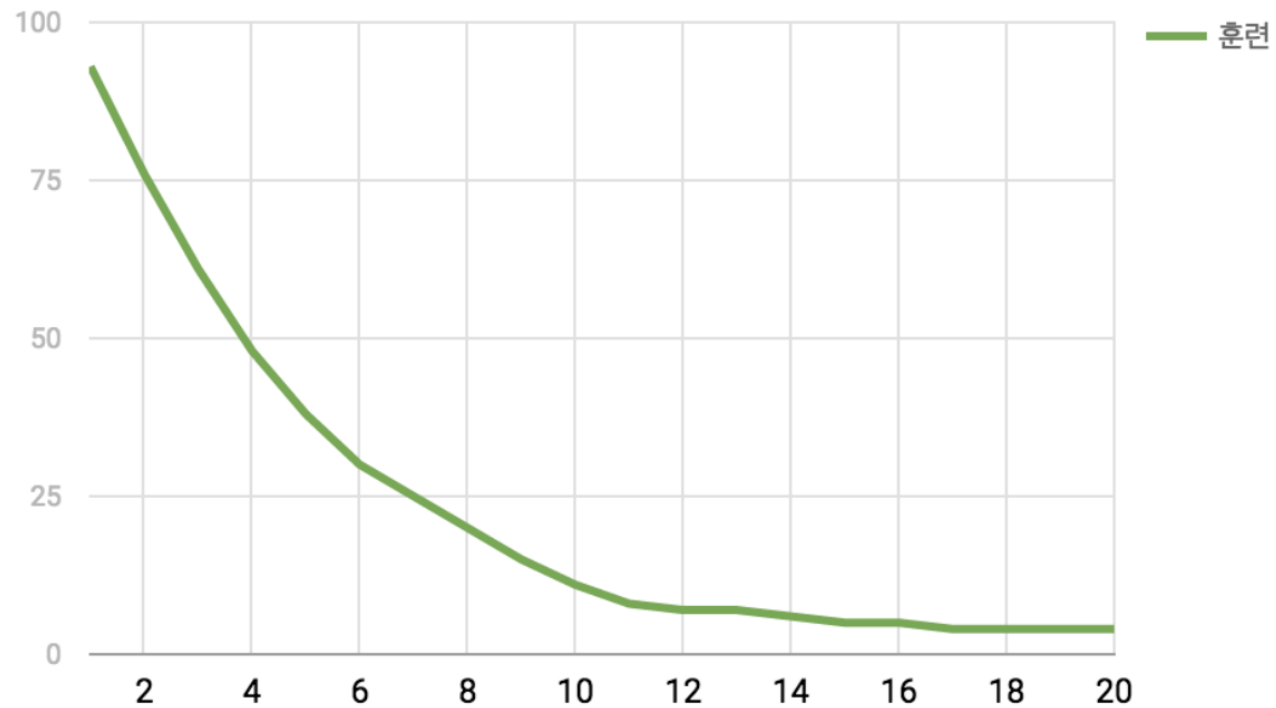
– 배치 사이즈 큰 것과 작은 것 중 뭐가 더 좋을까?

- 사람이 100문제를 다 풀고 정답을 맞춰 보는 것
- 사람이 1문제마다 풀면서 정답을 맞춰 보는 것

학습 과정 이야기

◆ 에포크(epchos) : 반복횟수

- 에포크를 무조건 늘리면 좋을까?



학습 과정 살펴보기

◆ 히스토리 기능

– `Model.fit()`

- 매 에포크 마다의 훈련 손실값 (loss) **loss가 acc보다 중요**
- 매 에포크 마다의 훈련 정확도 (acc)
- 매 에포크 마다의 검증 손실값 (val_loss)
- 매 에포크 마다의 검증 정확도 (val_acc)

학습 조기 종료 시키기

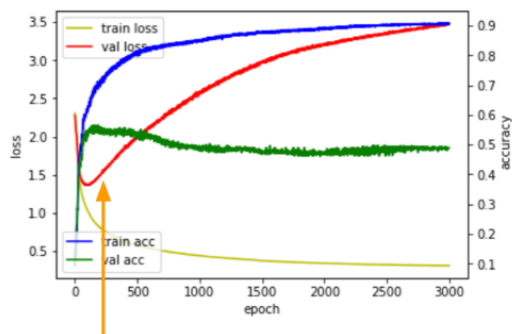
◆ EarlyStopping

patience는 실수했을 때 봐준다는 의미

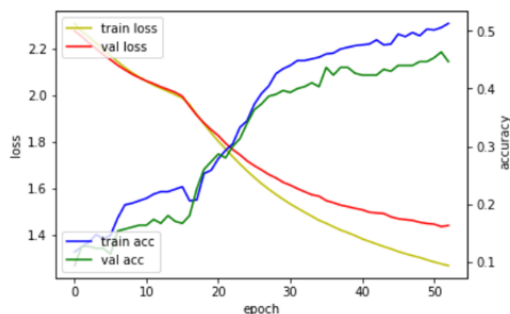
```
keras.callbacks.EarlyStopping(monitor='val_loss', min_delta=0, patience=0, verbose=0, mode='auto')
```

- monitor : 관찰하고자 하는 항목입니다. 'val_loss'나 'val_acc'가 주로 사용됩니다.
- min_delta : 개선되고 있다고 판단하기 위한 최소 변화량을 나타냅니다. 만약 변화량이 min_delta보다 적은 경우에는 개선이 없다고 판단합니다.
- patience : 개선이 없다고 바로 종료하지 않고 개선이 없는 에포크를 얼마나 기다려 줄 것인 가를 지정합니다. 만약 10이라고 지정하면 개선이 없는 에포크가 10번째 지속될 경우 학습을 종료합니다.
- verbose : 얼마나 자세하게 정보를 표시할 것인가를 지정합니다. (0, 1, 2)
- mode : 관찰 항목에 대해 개선이 없다고 판단하기 위한 기준을 지정합니다. 예를 들어 관찰 항목이 'val_loss'인 경우에는 감소되는 것이 멈출 때 종료되어야 하므로, 'min'으로 설정됩니다.
 - auto : 관찰하는 이름에 따라 자동으로 지정합니다.
 - min : 관찰하고 있는 항목이 감소되는 것을 멈출 때 종료합니다.
 - max : 관찰하고 있는 항목이 증가되는 것을 멈출 때 종료합니다.

과적합 발생

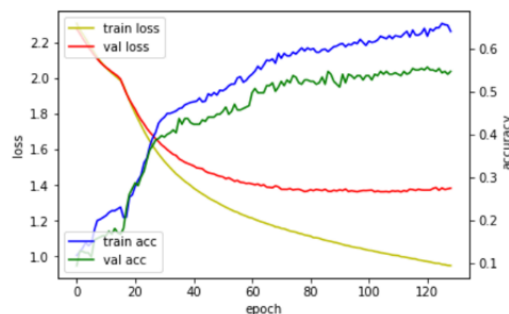


성급한 조기종료



EarlyStopping()

적절한 조기종료



EarlyStopping(patience = 20)

평가 이야기

◆ 성능평가지표 (정확도, 정밀도, 재현율)

$$(Accuracy) = \frac{TP + TN}{TP + FN + FP + TN}$$

$$(Precision) = \frac{TP}{TP + FP}$$

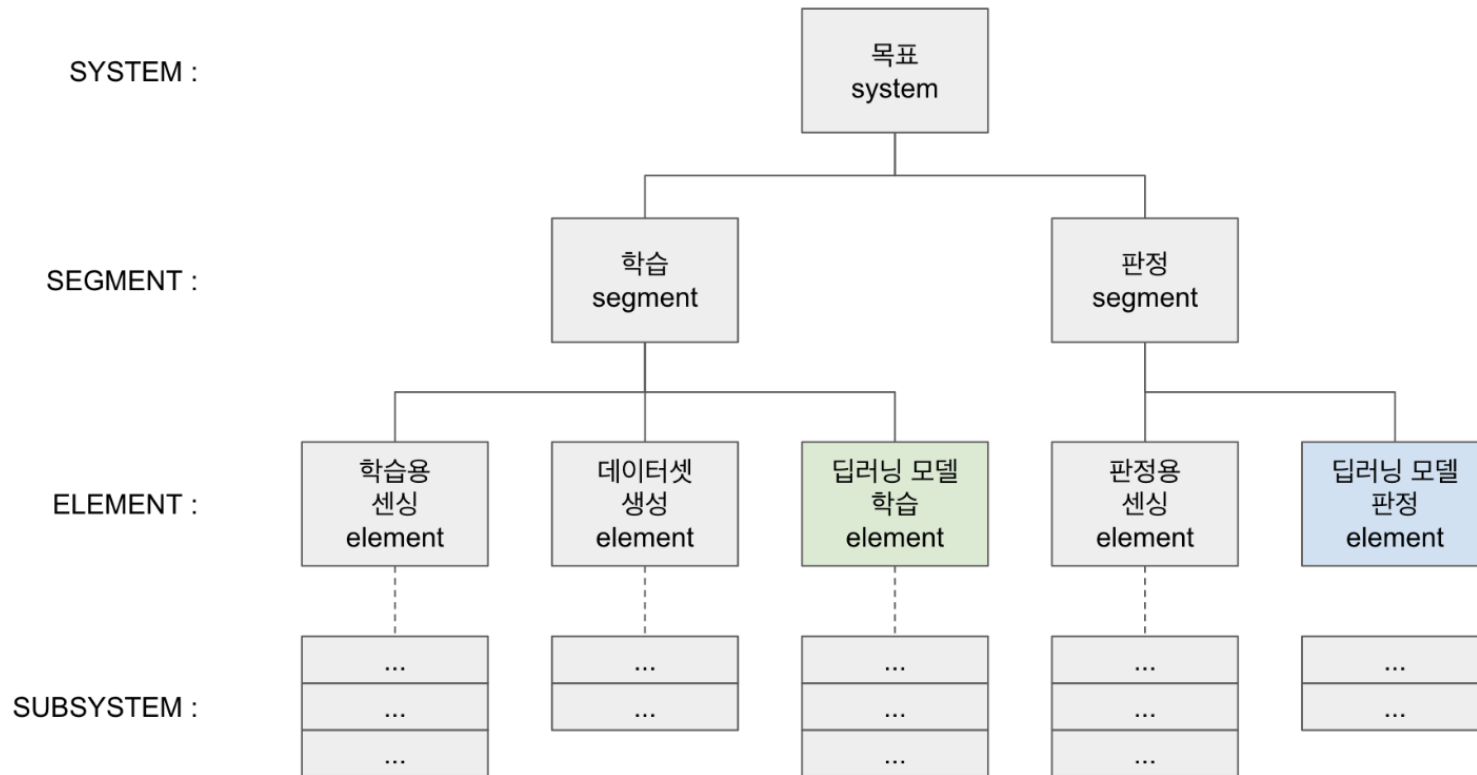
$$(Recall) = \frac{TP}{TP + FN}$$

$$(F1-score) = 2 \times \frac{1}{\frac{1}{Precision} + \frac{1}{Recall}} = 2 \times \frac{Precision \times Recall}{Precision + Recall}$$

		실제 정답	
		True	False
분류 결과	True	True Positive	False Positive
	False	False Negative	True Negative

학습 모델 보기/저장하기/불러오기

◆ 실무에서의 딥러닝



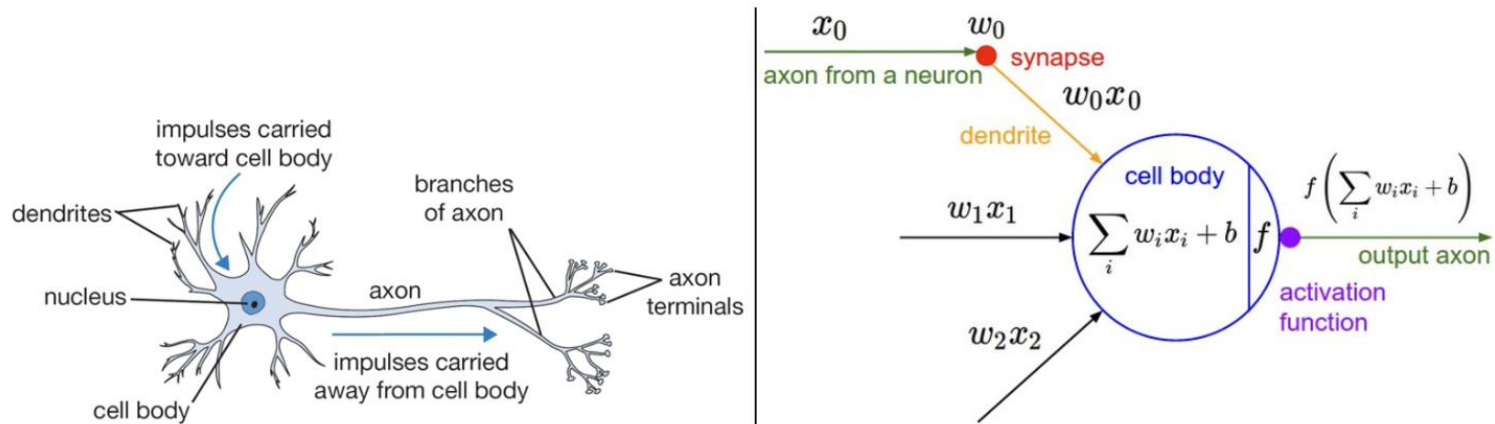
딥러닝

(Part 3)

다층 퍼셉트론 레이어 이야기

◆ 인간의 신경계를 모사한 뉴런 이야기

- axon (축삭돌기) : 팔처럼 몸체에서 뻗어나와 다른 뉴런의 수상돌기와 연결됩니다.
- dendrite (수상돌기) : 다른 뉴런의 축삭 돌기와 연결되며, 몸체에 나뭇가지 형태로 붙어 있습니다.
- synapse (시냅스) : 축삭돌기와 수상돌기가 연결된 지점입니다. 여기서 한 뉴런이 다른 뉴런으로 신호가 전달됩니다.
- x_0, x_1, x_2 : 입력되는 뉴런의 축삭돌기로부터 전달되는 신호의 양
- w_0, w_1, w_2 : 시냅스의 강도, 즉 입력되는 뉴런의 영향력을 나타냅니다.
- $w_0x_0 + w_1x_1 + w_2x_2$: 입력되는 신호의 양과 해당 신호의 시냅스 강도가 곱해진 값의 합계
- f : 최종 합계가 다른 뉴런에게 전달되는 신호의 양을 결정짓는 규칙, 이를 활성화 함수라고 부릅니다.



A cartoon drawing of a biological neuron (left) and its mathematical model (right).

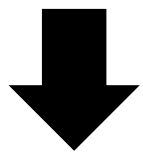
퍼셉트론이란?

- 일반적으로 퍼셉트론은 다음과 같이 식을 변형한다.

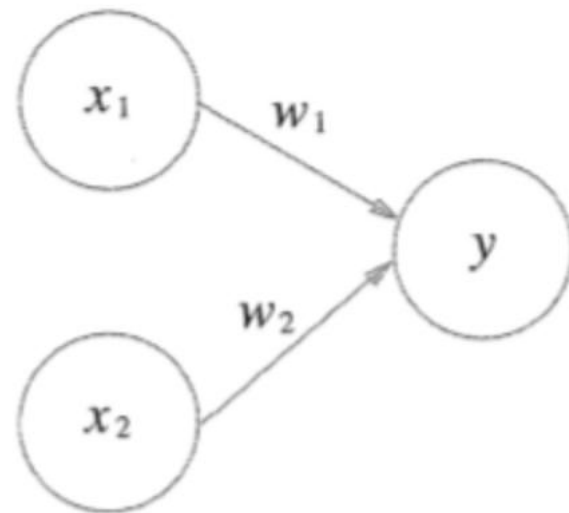
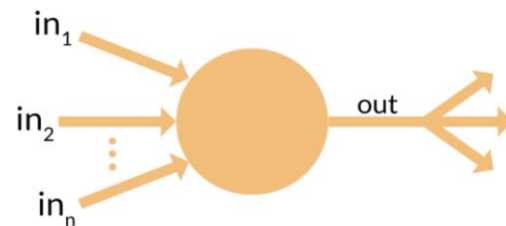
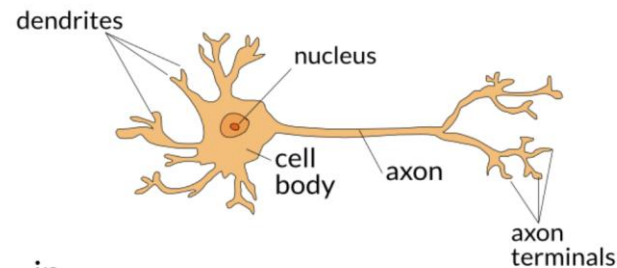
(θ 를 $-b$ (편향 : bias)로 치환)

즉, 입력 신호 * 가중치 + 편향 ≥ 0 이면 1의 값을

$$y = \begin{cases} 0 & (w_1x_1 + w_2x_2 \leq \theta) \\ 1 & (w_1x_1 + w_2x_2 > \theta) \end{cases}$$



$$y = \begin{cases} 0 & (b + w_1x_1 + w_2x_2 \leq 0) \\ 1 & (b + w_1x_1 + w_2x_2 > 0) \end{cases}$$



논리회로

1) AND 게이트를 퍼셉트론으로 표현하려면 ?

-> 진리표 조건에 맞는 w_1, w_2, b 정하기
(0.5, 0.5, -0.7) , (0.5, 0.5, -0.8), (1.0, 1.0, -1.0) 등...

-> 즉 $w_1 * x_1 + w_2 * x_2 + b \geq 0$ 일 때 1
 $w_1 * x_1 + w_2 * x_2 + b < 0$ 일 때 0

x_1	x_2	y
0	0	0
1	0	0
0	1	0
1	1	1

AND 게이트 진리표
OR 게이트

x_1	x_2	y
0	0	1
1	0	1
0	1	1
1	1	0

NAND 게이트

x_1	x_2	y
0	0	0
1	0	1
0	1	1
1	1	1

퍼셉트론 구현

```
: import numpy as np

def AND(x1, x2):
    x = np.array([x1, x2])
    w = np.array([0.5, 0.5])
    b = -0.7
    tmp = np.sum(w*x) + b
    if tmp <= 0:
        return 0
    else:
        return 1

print ("!!!!!!!!!!!!!! AND GATE !!!!!!!!!!!!!!!")
print()
for i in [(0, 0), (1, 0), (0, 1), (1, 1)]:
    y = AND(i[0], i[1])
    print(str(i) + " -> " + str(y))
```

!!!!!!!!!!!!!! AND GATE !!!!!!!!!!!!!!!

(0, 0) -> 0
(1, 0) -> 0
(0, 1) -> 0
(1, 1) -> 1

퍼셉트론의 한계

XOR은 못 찾는데 선형이 안 되서
비선형으로 가야지 그래서 여러층
으로 deep learning = 다층

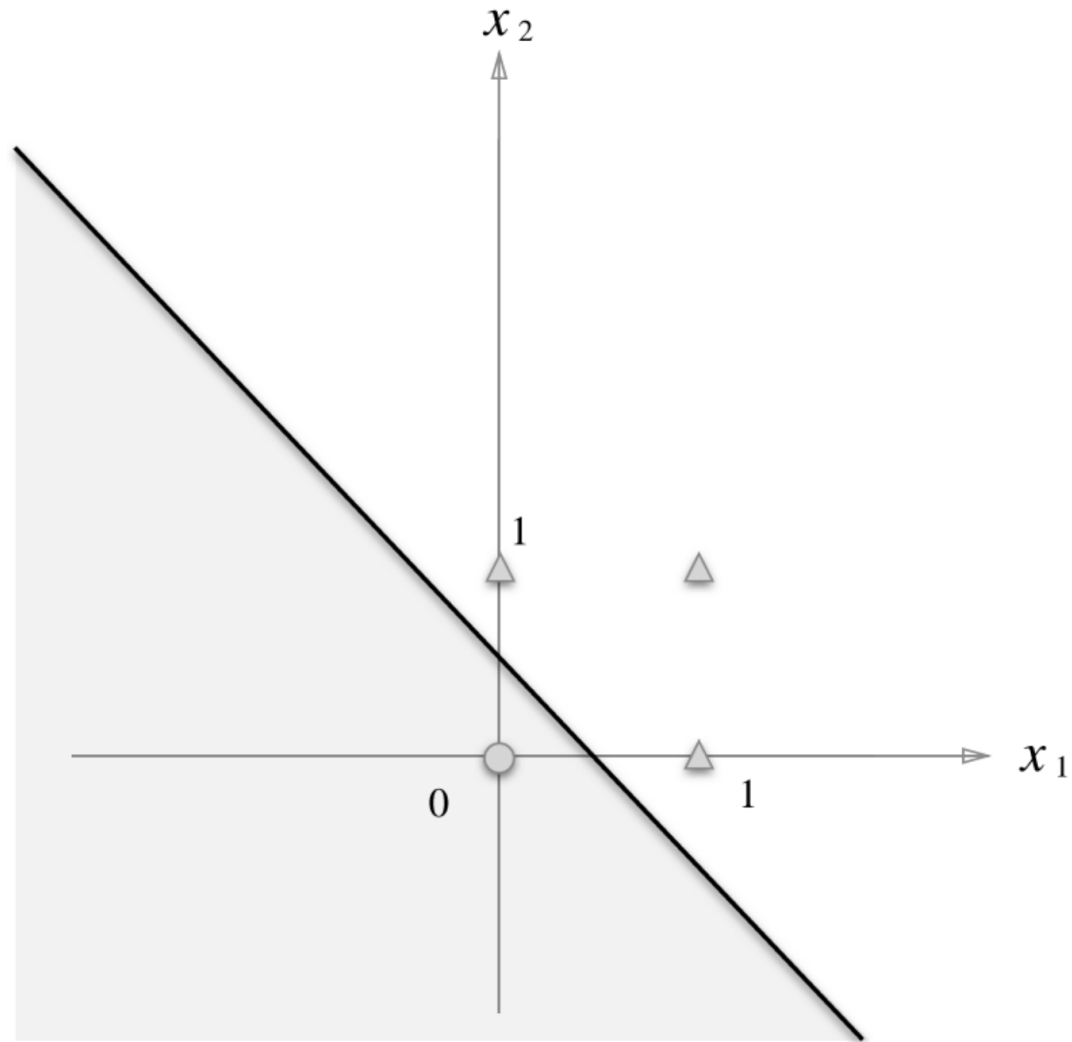
XOR 게이트를 퍼셉트론으로 표현하려면 ?

-> 진리표 조건에 맞는 w_1, w_2, b 정하기

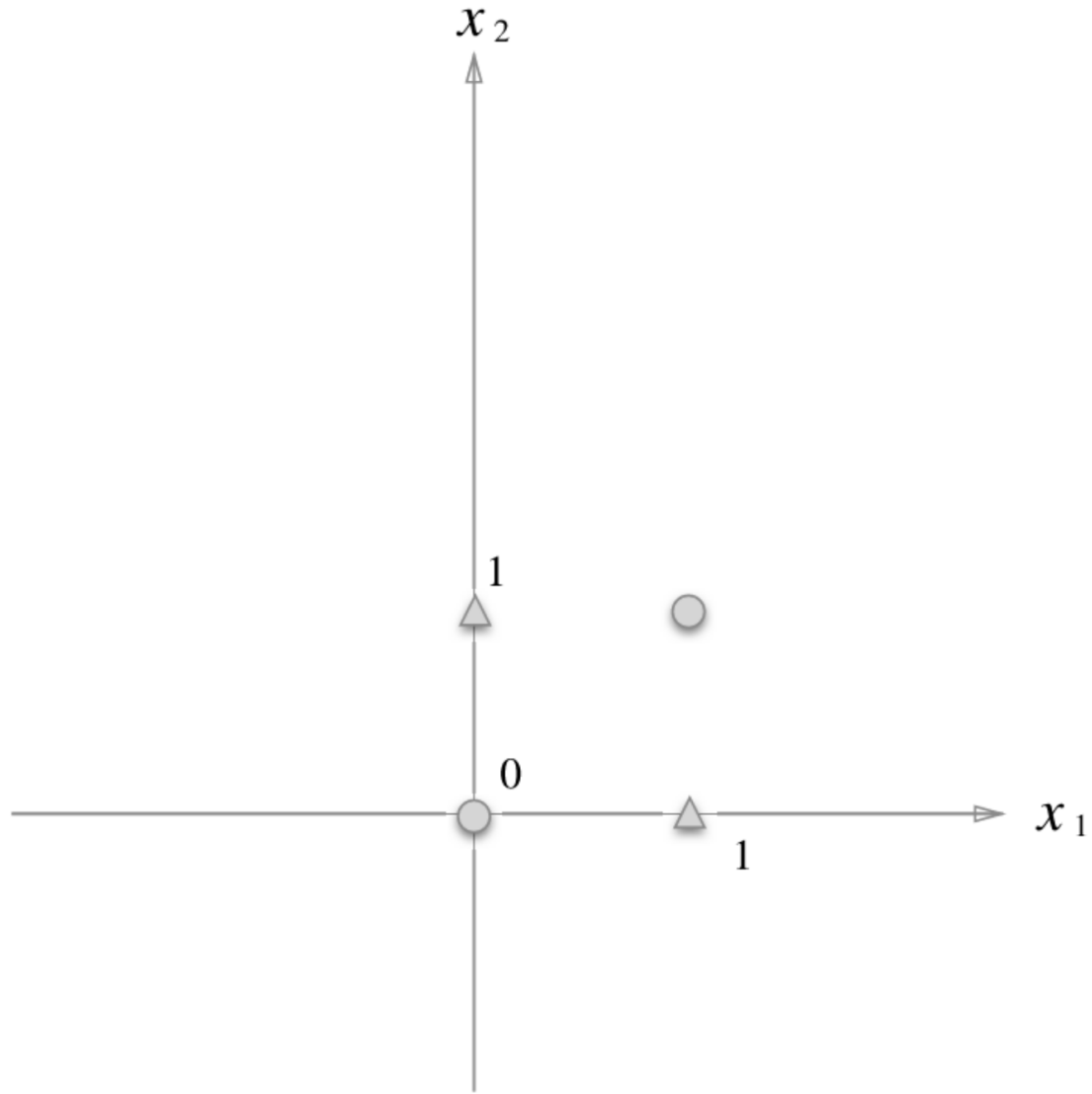
-> 즉 $w_1 \cdot x_1 + w_2 \cdot x_2 + b \geq 0$ 일 때 1
 $w_1 \cdot x_1 + w_2 \cdot x_2 + b < 0$ 일 때 0

x_1	x_2	y
0	0	0
1	0	1
0	1	1
1	1	0

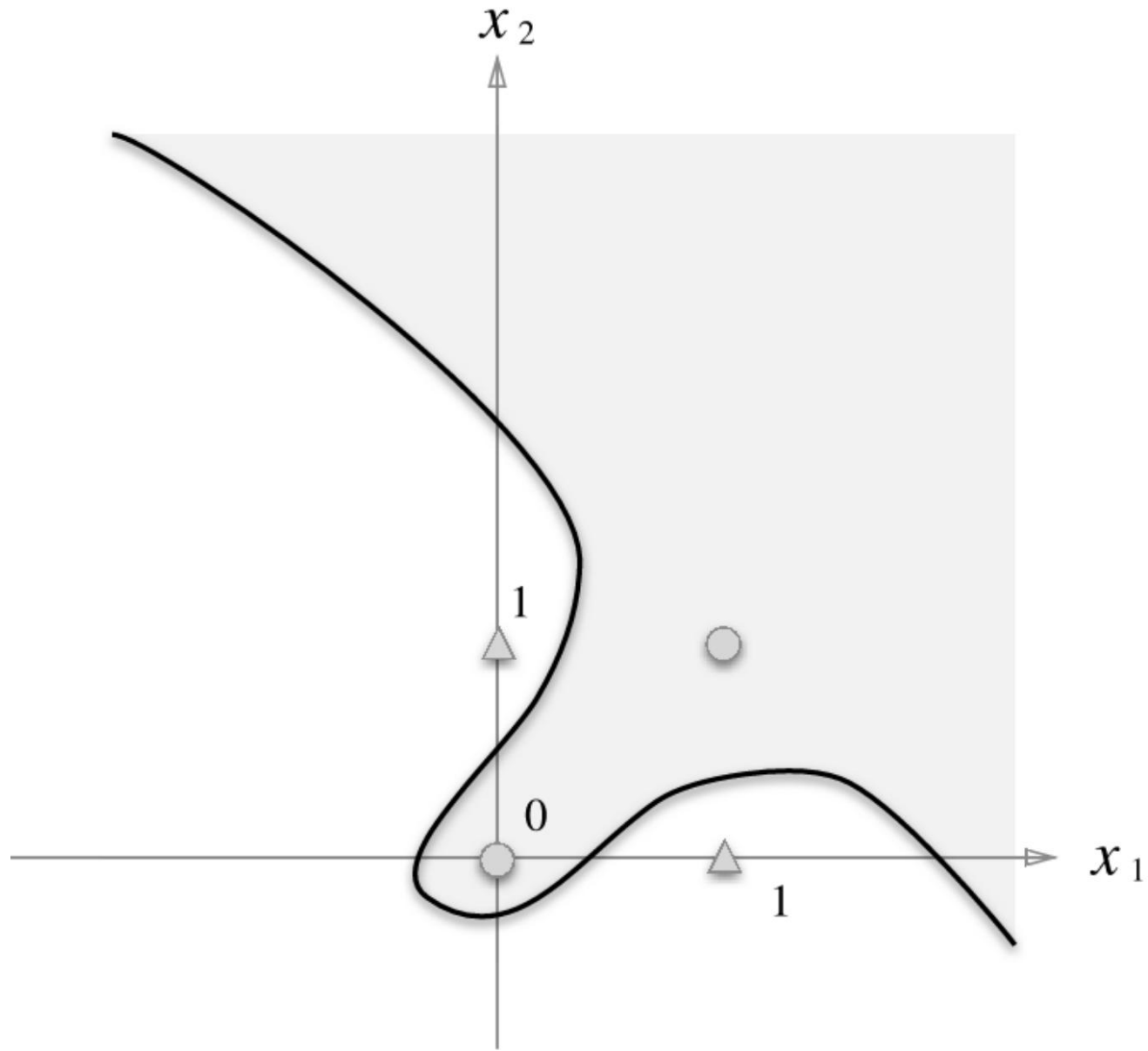
퍼셉트론의 한계



퍼셉트론의 한계



퍼셉트론의 한계



다층 퍼셉트론



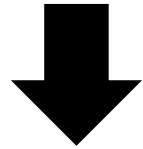
AND



NAND

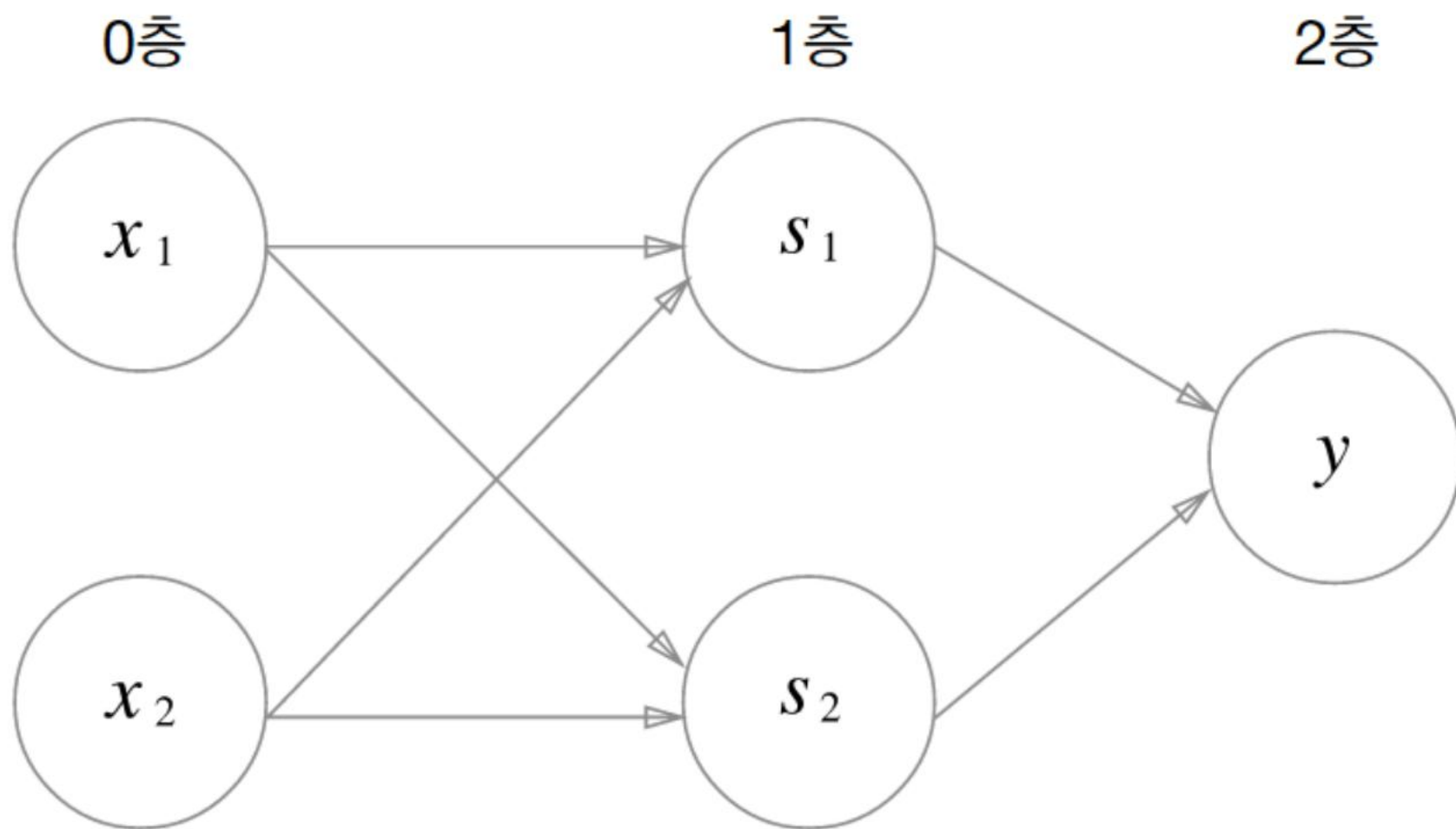


OR



?

다층 퍼셉트론



다층 퍼셉트론

$$\text{XOR}(x_1, x_2) = \text{AND}(\text{NAND}(x_1, x_2), \text{OR}(x_1, x_2))$$

0 층: 입력값 x_1, x_2

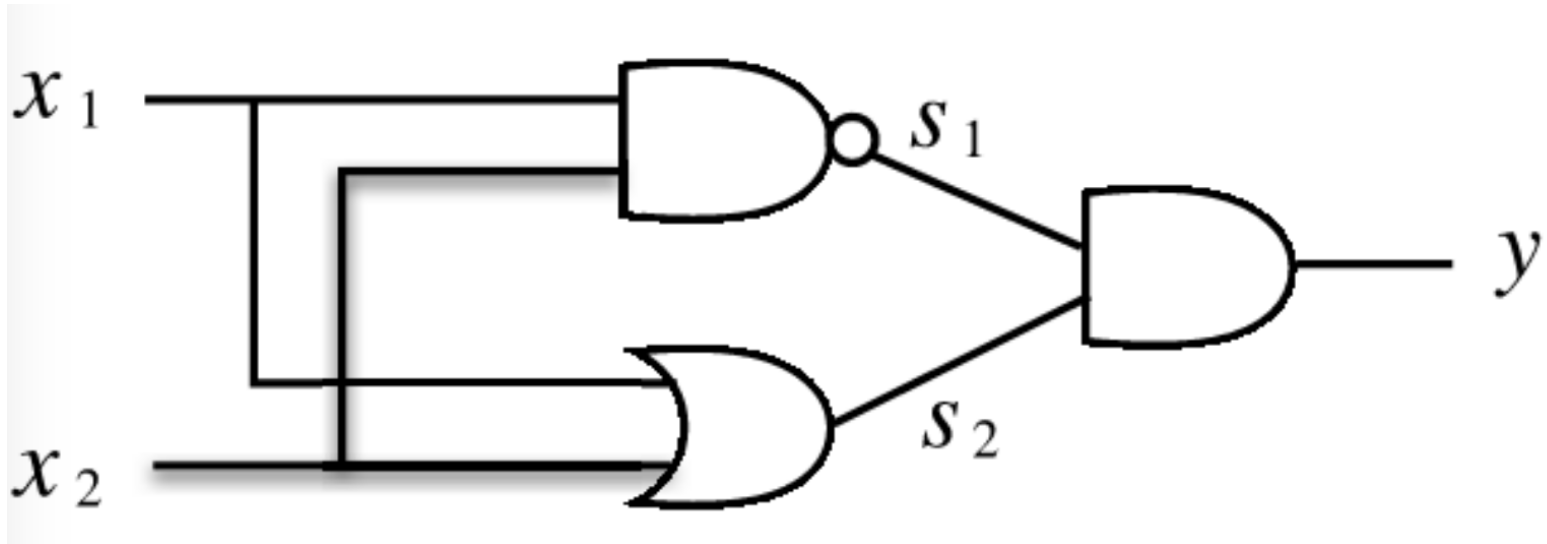
1 층: NAND, OR 게이트 결과

2 층: AND 게이트 결과

- 0층의 두 뉴런이 입력 신호를 받아 1층의 뉴런으로 신호를 보낸다.

- 1층의 뉴런이 2층의 뉴런으로 신호를 보내고, 2층의 뉴런은 이 신호를 바탕으로 y 를 출력한다.

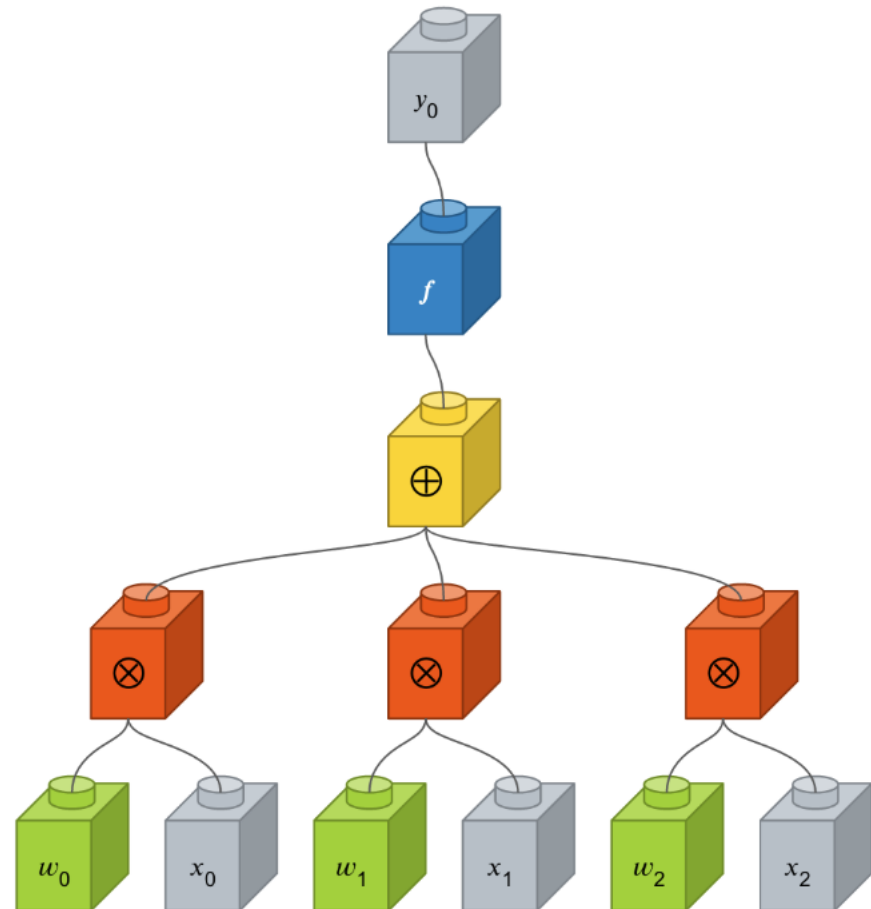
x_1	x_2	s_1	s_2	y
0	0	1	0	0
1	0	1	1	1
0	1	1	1	1
1	1	0	1	0



다층 퍼셉트론 레이어 이야기

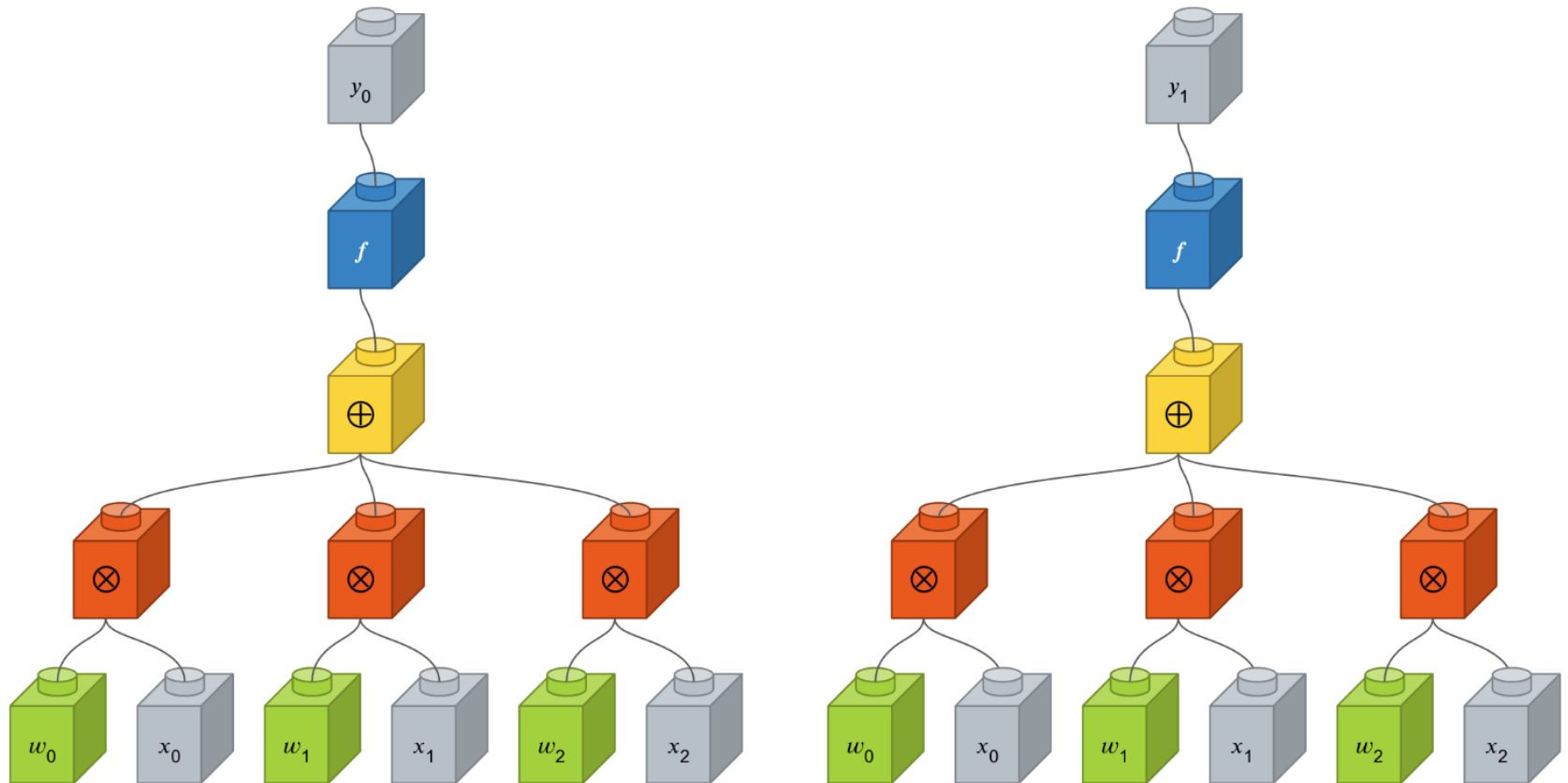
◆ 인간의 신경계를 모사한 뉴런 이야기

- 녹색 : 시냅스 강도 $\equiv$ 가중치
- 노란색, 빨간색 : 연산자
- 파란색 : 활성화 함수 $\equiv \sigma$
- 회색 x : 신호, y : 결과



다층 퍼셉트론 레이어 이야기

◆ 인간의 신경계를 모사한 뉴런 이야기

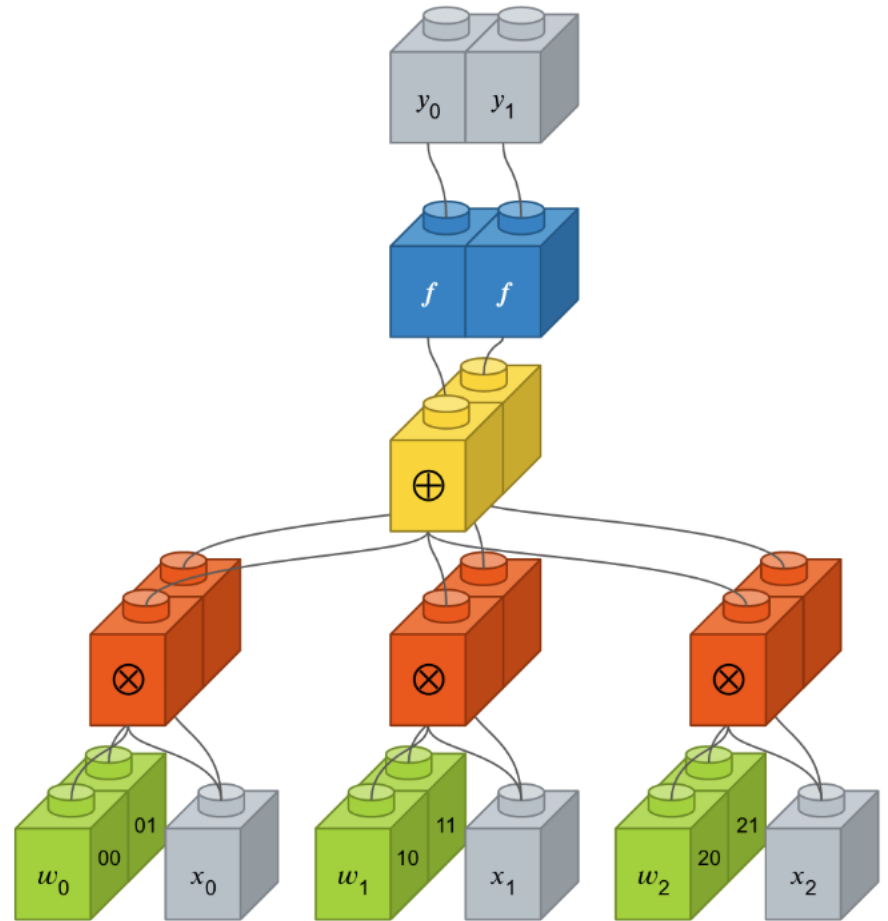


다층 퍼셉트론 레이어 이야기

◆ 인간의 신경계를 모사한 뉴런 이야기

– 총 연결의 수

$$: 3(\text{신호}) \times 2(\text{녹색}) = 6$$



다층 퍼셉트론 레이어 이야기

◆ 입출력을 모두 연결해주는 Dense 레이어

- 가중치가 높을수록 해당 입력 뉴런이 출력 뉴런에 미치는 영향이 크고, 낮을수록 미치는 영향이 적다.

```
Dense(8, input_dim=4, init='uniform', activation='relu'))
```

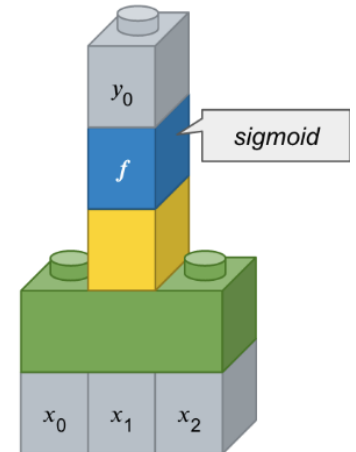
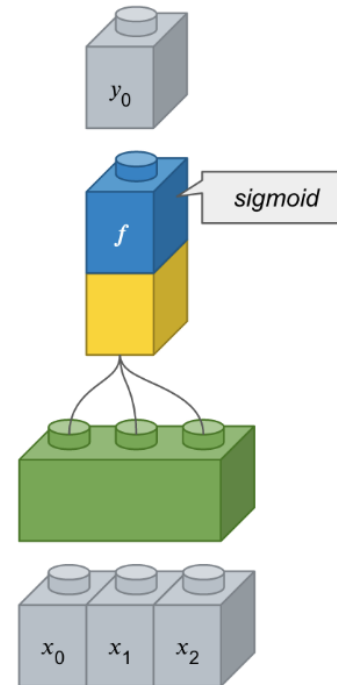
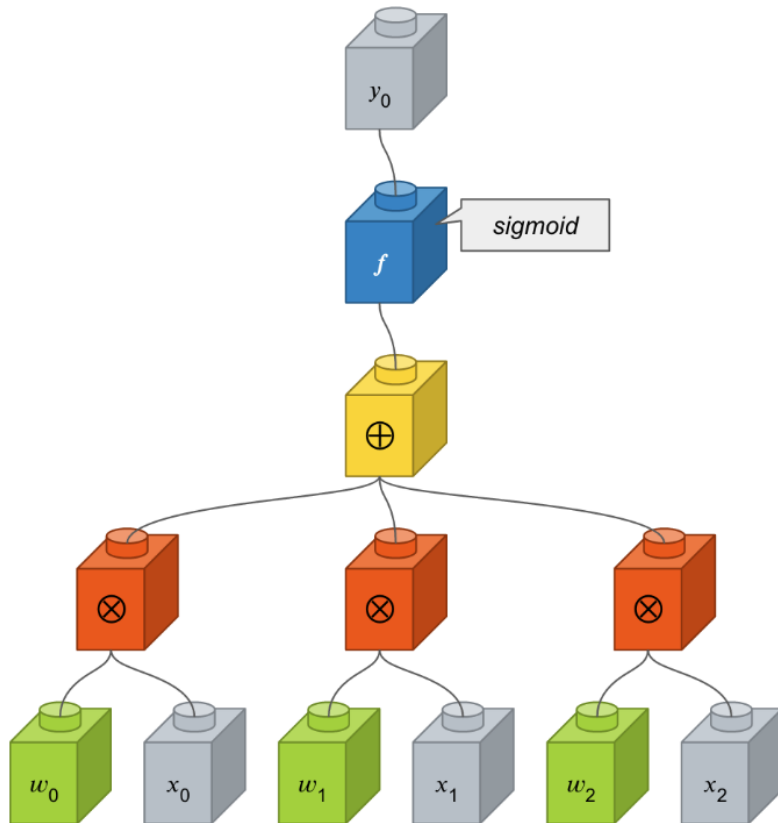
- 첫번째 인자 : 출력 뉴런의 수를 설정합니다.
- input_dim : 입력 뉴런의 수를 설정합니다.
- init : 가중치 초기화 방법 설정합니다.
 - 'uniform' : 균일 분포
 - 'normal' : 가우시안 분포
- activation : 활성화 함수 설정합니다.
 - 'linear' : 디폴트 값, 입력뉴런과 가중치로 계산된 결과값이 그대로 출력으로 나옵니다.
 - 'relu' : rectifier 함수, 은닉층에 주로 쓰입니다.
 - 'sigmoid' : 시그모이드 함수, 이진 분류 문제에서 출력층에 주로 쓰입니다.
 - 'softmax' : 소프트맥스 함수, 다중 클래스 분류 문제에서 출력층에 주로 쓰입니다.

다층 퍼셉트론 레이어 이야기

◆ 입출력을 모두 연결해주는 Dense 레이어

`Dense(1, input_dim=3, activation='sigmoid')`

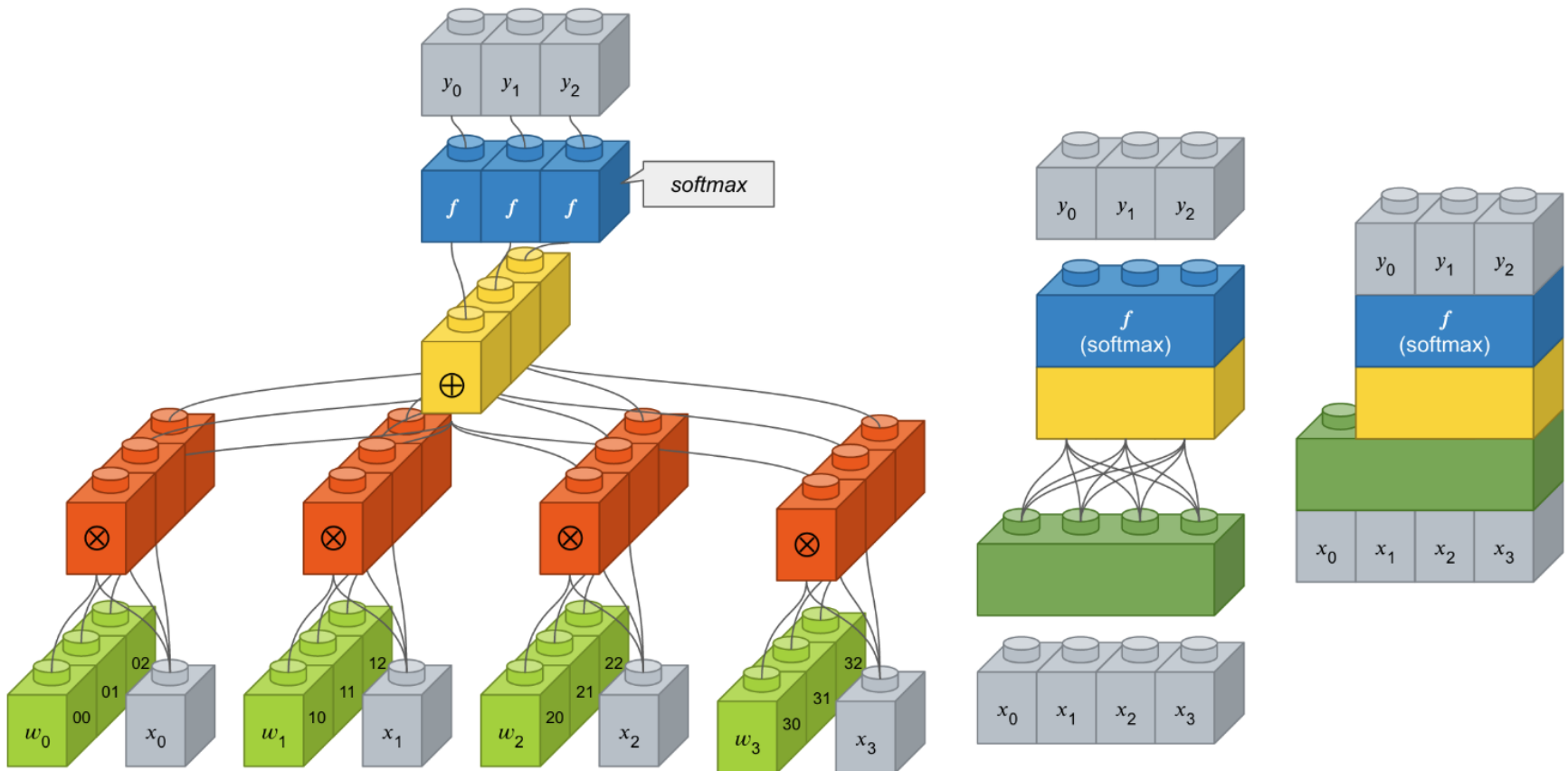
(출력, 입력, 통과할지말지어떤거로)



다층 퍼셉트론 레이어 이야기

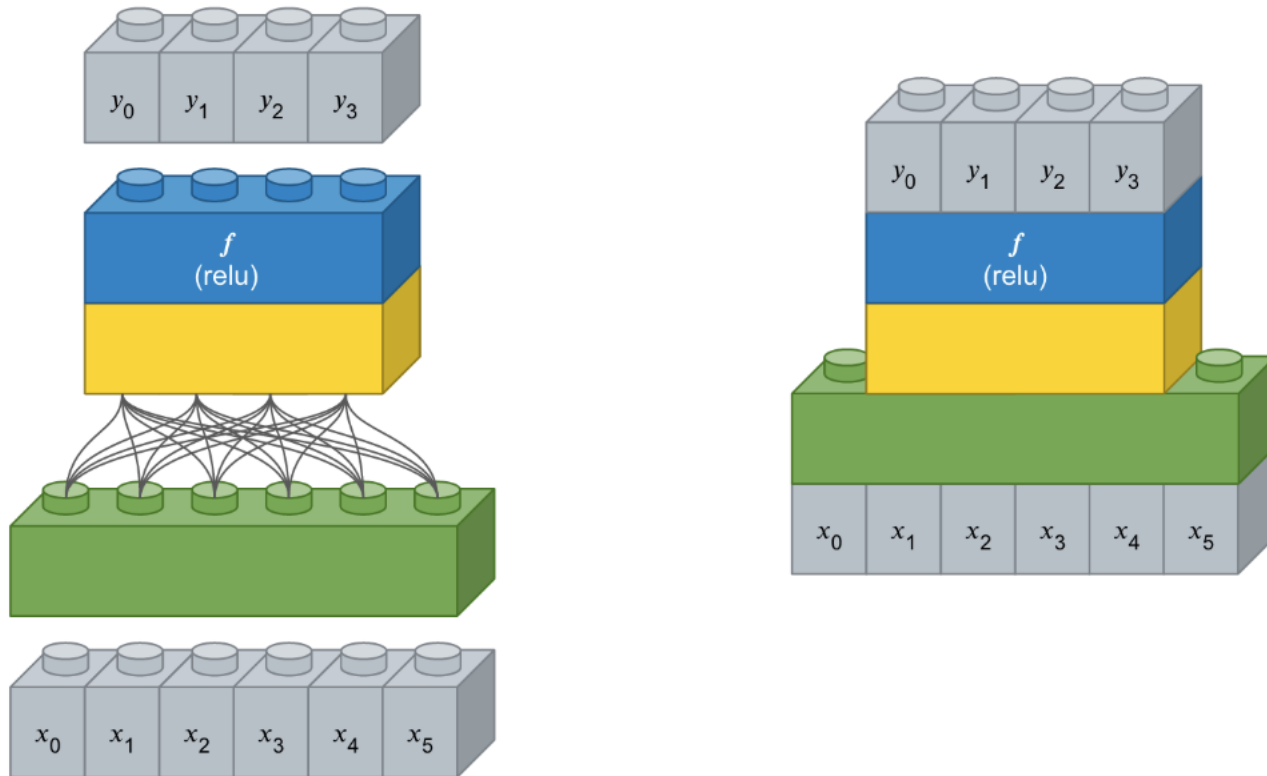
◆ 입출력을 모두 연결해주는 Dense 레이어

`Dense(3, input_dim=4, activation='softmax')`



다층 퍼셉트론 레이어 이야기

◆ 입출력을 모두 연결해주는 Dense 레이어



다층 퍼셉트론 신경망 모델 만들어보기

◆ 피마족 인디언 당뇨병 발병 데이터셋

- 인스턴스 수와 속성 수가 예제로 사용하기에 적당합니다.
- 모든 특징이 정수 혹은 실수로 되어 있어서 별도의 전처리 과정이 필요없습니다.
- 인스턴스 수 : 768개
- 속성 수 : 8가지
- 클래스 수 : 2가지

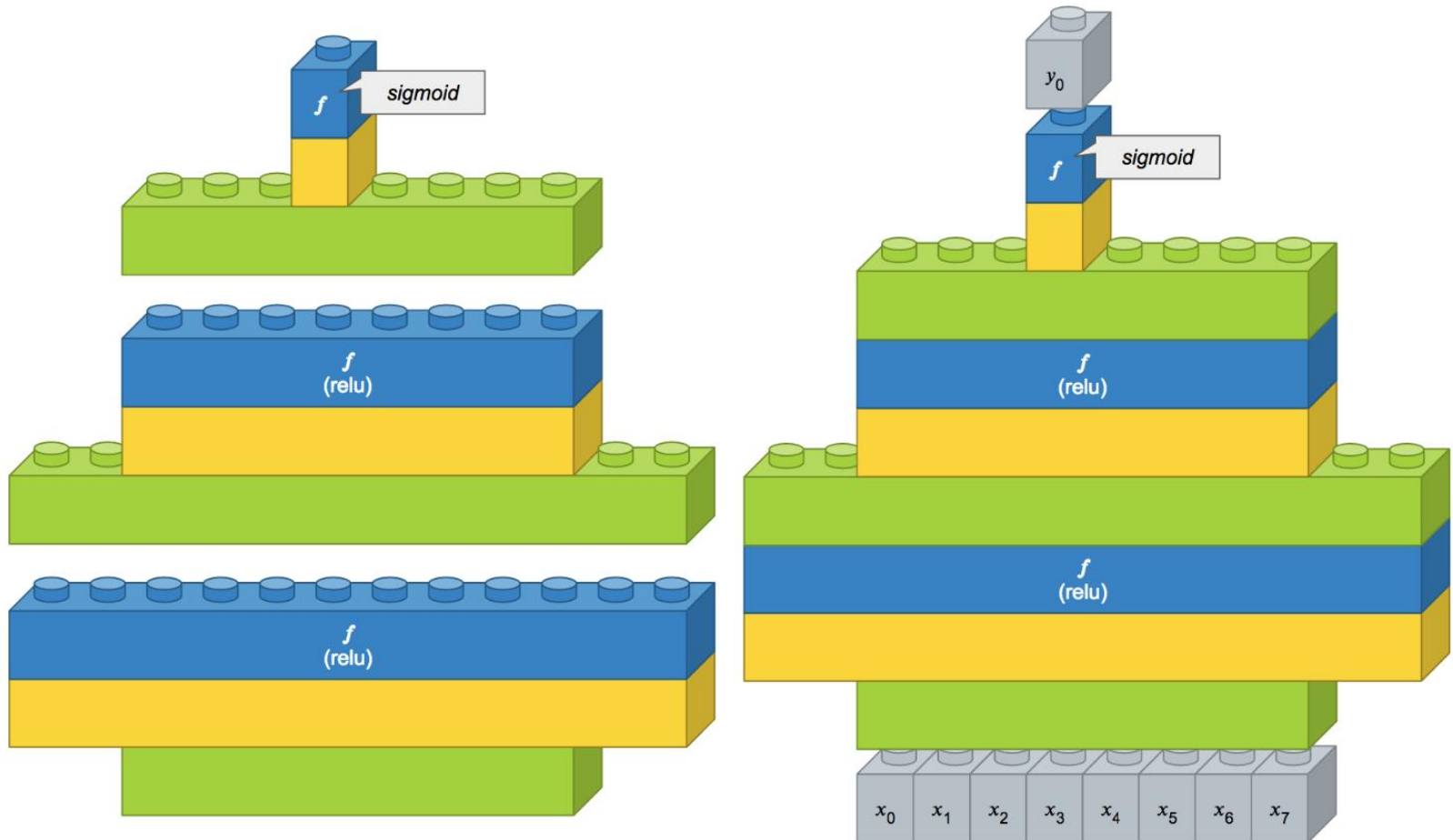
8가지 속성(1번~8번)과 결과(9번)의 상세 내용은 다음과 같습니다.

1. 임신 횟수
2. 경구 포도당 내성 검사에서 2시간 동안의 혈장 포도당 농도
3. 이완기 혈압 (mm Hg)
4. 삼두근 피부 두껍 두께 (mm)
5. 2 시간 혈청 인슐린 (mu U/ml)
6. 체질량 지수
7. 당뇨 직계 가족력
8. 나이 (세)
9. 5년 이내 당뇨병이 발병 여부

양성인 경우가 268개(34.9%)
음성인 경우가 500개(65.1%)

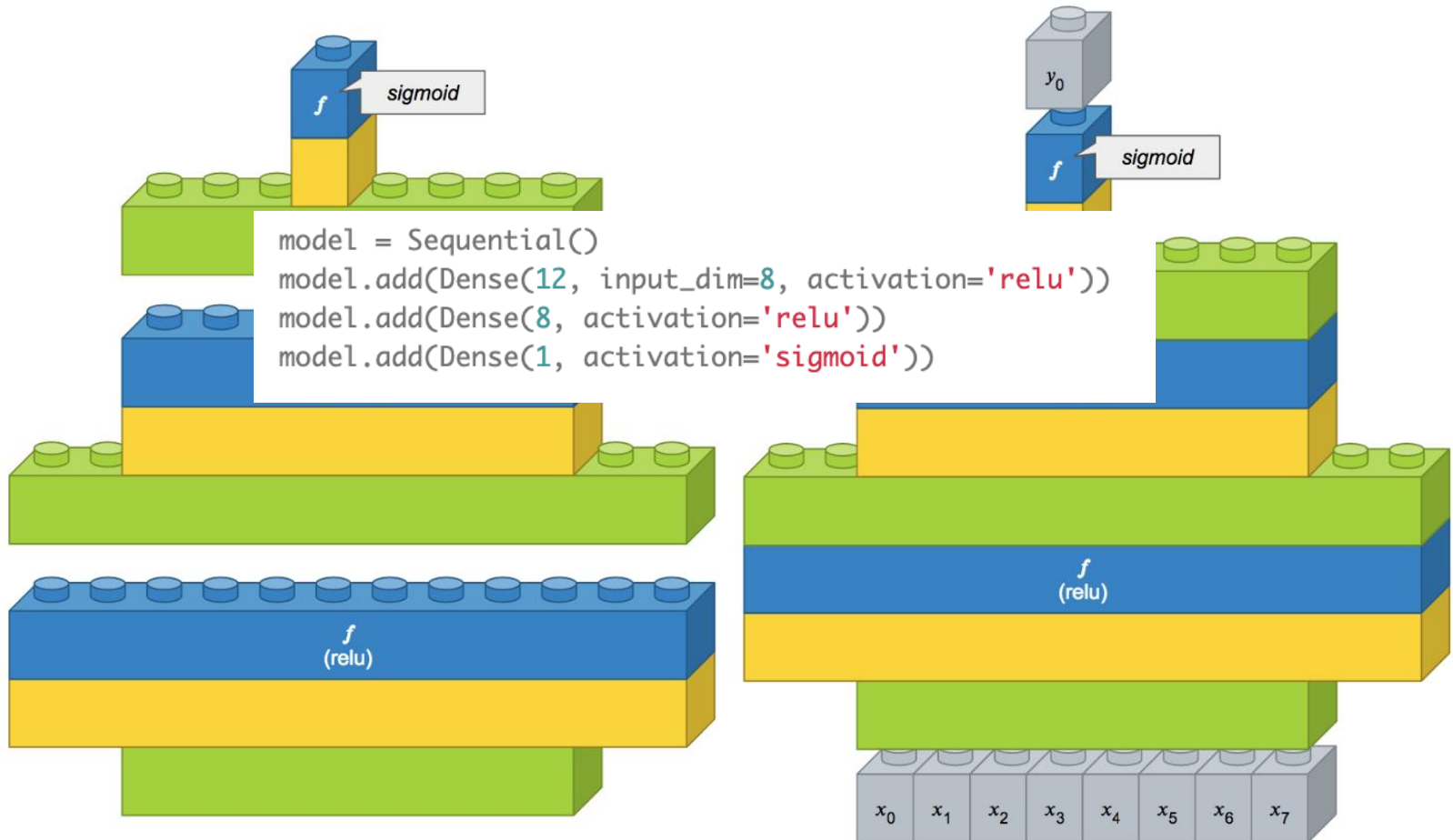
다층 퍼셉트론 신경망 모델 만들어보기

◆ 피마족 인디언 당뇨병 발병 데이터셋



다층 퍼셉트론 신경망 모델 만들어보기

◆ 피마족 인디언 당뇨병 발병 데이터셋



컨볼루션 신경망 레이어 이야기

◆ 필터로 특징을 뽑아주는 컨볼루션 레이어

```
Conv2D(32, (5, 5), padding='valid', input_shape=(28, 28, 1), activation='relu')
```

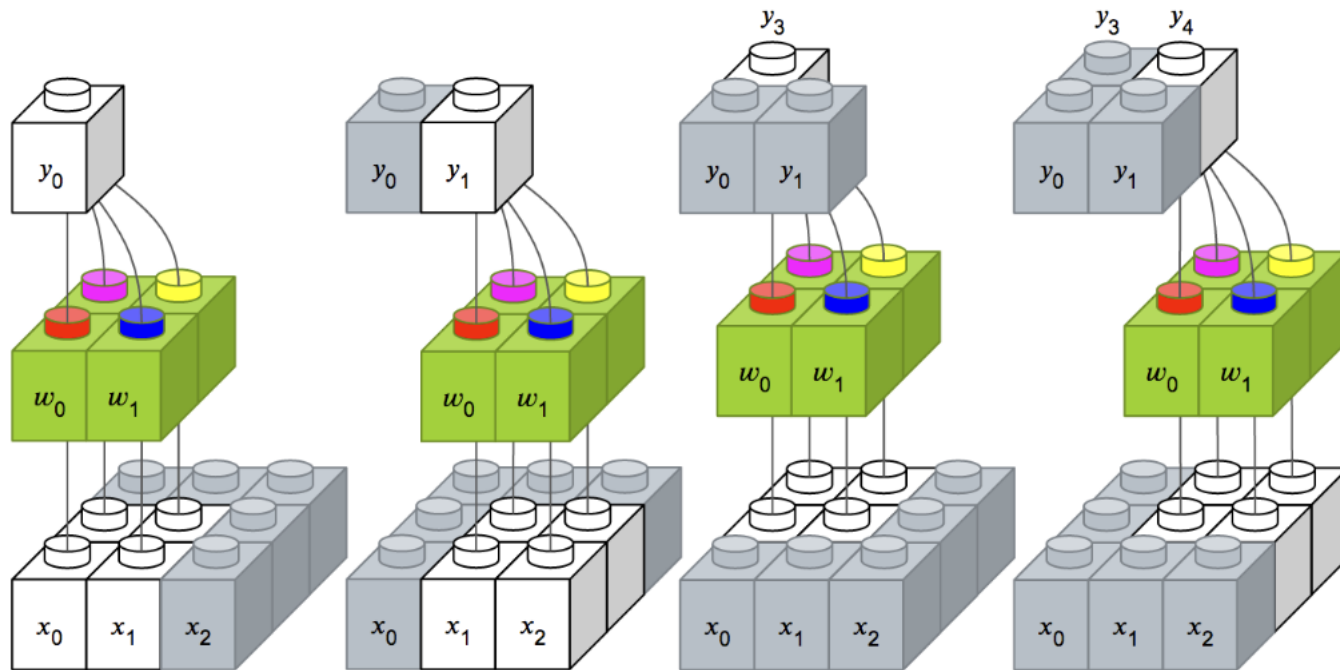
- 첫번째 인자 : 컨볼루션 필터의 수 입니다.
- 두번째 인자 : 컨볼루션 커널의 (행, 열) 입니다.
- padding : 경계 처리 방법을 정의합니다.
 - 'valid' : 유효한 영역만 출력이 됩니다. 따라서 출력 이미지 사이즈는 입력 사이즈보다 작습니다.
 - 'same' : 출력 이미지 사이즈가 입력 이미지 사이즈와 동일합니다.
- input_shape : 샘플 수를 제외한 입력 형태를 정의 합니다. 모델에서 첫 레이어일 때만 정의하면 됩니다.
 - (행, 열, 채널 수)로 정의합니다. 흑백영상인 경우에는 채널이 1이고, 컬러(RGB)영상인 경우에는 채널을 3으로 설정합니다.
- activation : 활성화 함수 설정합니다.
 - 'linear' : 디폴트 값, 입력뉴런과 가중치로 계산된 결과값이 그대로 출력으로 나옵니다.
 - 'relu' : rectifier 함수, 은닉층에 주로 쓰입니다.
 - 'sigmoid' : 시그모이드 함수, 이진 분류 문제에서 출력층에 주로 쓰입니다.
 - 'softmax' : 소프트맥스 함수, 다중 클래스 분류 문제에서 출력층에 주로 쓰입니다.

컨볼루션 신경망 레이어 이야기

◆ 필터로 특징을 뽑아주는 컨볼루션 레이어

`Conv2D(1, (2, 2), padding='valid', input_shape=(3, 3, 1))`

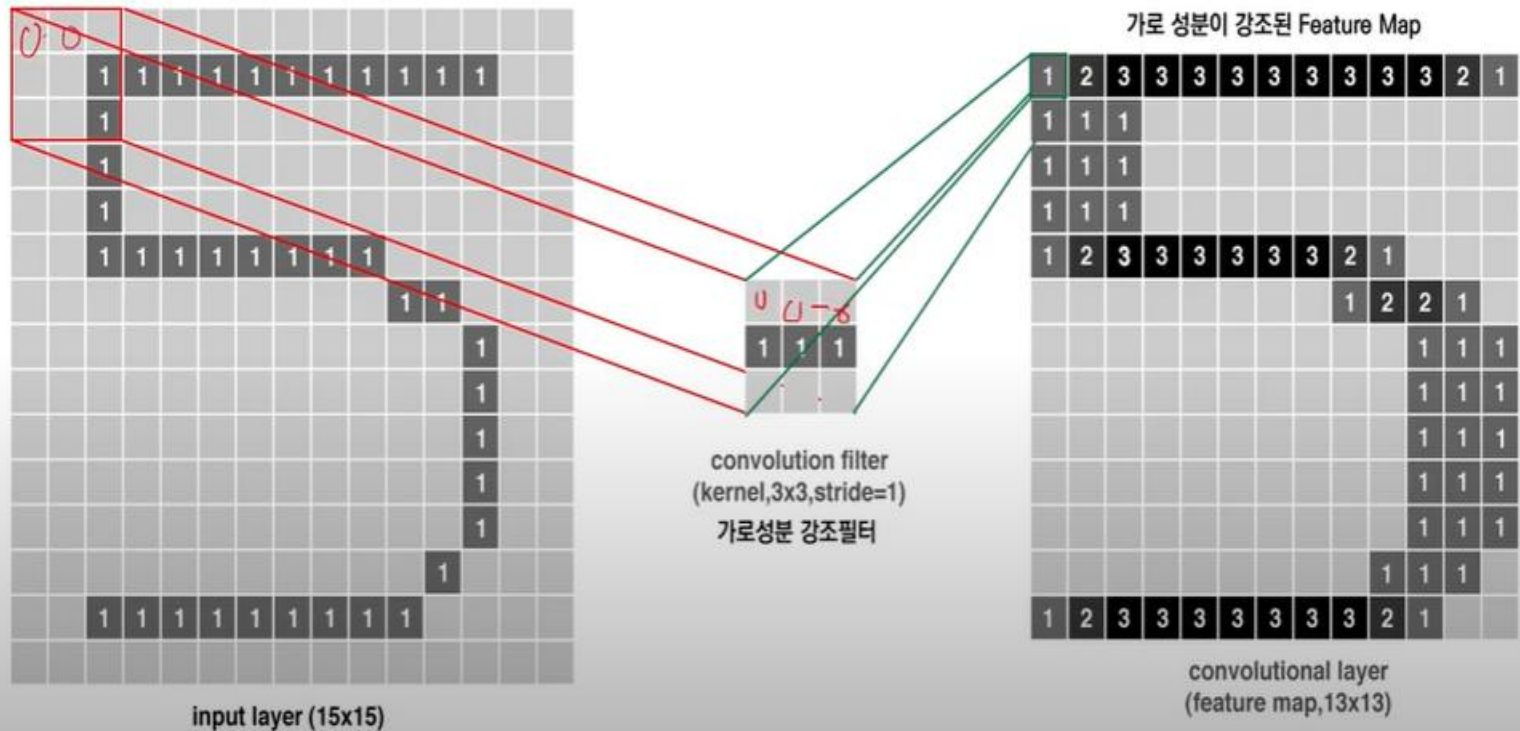
입력 이미지는 채널 수가 1, 너비가 3 픽셀, 높이가 3 픽셀이고, 크기가 2 x 2'



컨볼루션 신경망 레이어 이야기

컨볼루션 필터(커널)의 적용

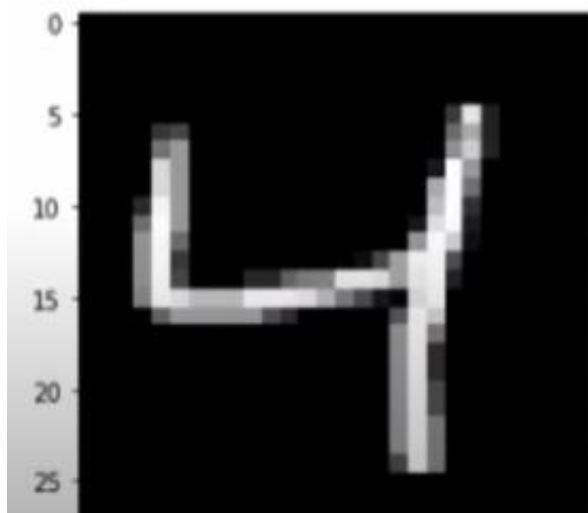
Application of Convolution Filter [Kernal]



컨볼루션 신경망 레이어 이야기

특정한 패턴의 특징이
어디서 나타나는지를 확인하는 도구

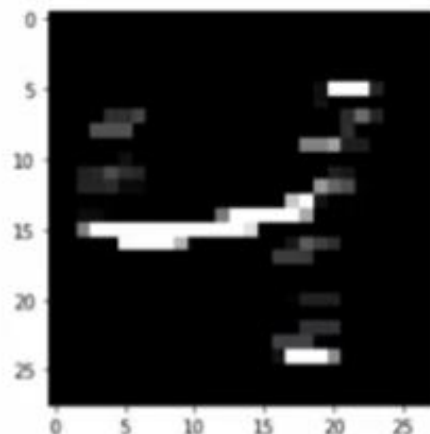
“Convolution”



필터



특징맵 feature map



컨볼루션 신경망 레이어 이야기

풀링; 최대/평균/확률 풀링

Pooling; Max/Mean/Stochastic Pooling

1	2	3	3	3	3	3	3	3	3	3	2	1
1	1	1										
1	1	1										
1	1	1										
1	2	3	3	3	3	3	3	2	1			
								1	2	2	1	
										1	1	1
										1	1	1
										1	1	1
										1	1	1
										1	1	1
										1	1	1
1	2	3	3	3	3	3	3	2	1			

최대 풀링 max pooling

nxn

일정한 크기(nxn)내에 있는 데이터의 값 중 가장 큰 값을 대푯값으로 설정하는 것.

평균 풀링 mean pooling

nxn

일정한 크기(nxn)내에 있는 데이터의 모든 값들의 평균을 대푯값으로 설정하는 것.

확률 풀링 stochastic pooling

nxn

일정한 크기(nxn)내에 있는 데이터의 값들 중 하나를 확률에 근거하여 대푯값으로 설정하는 것.

max-pooled feature map(2×2 , stride=1)

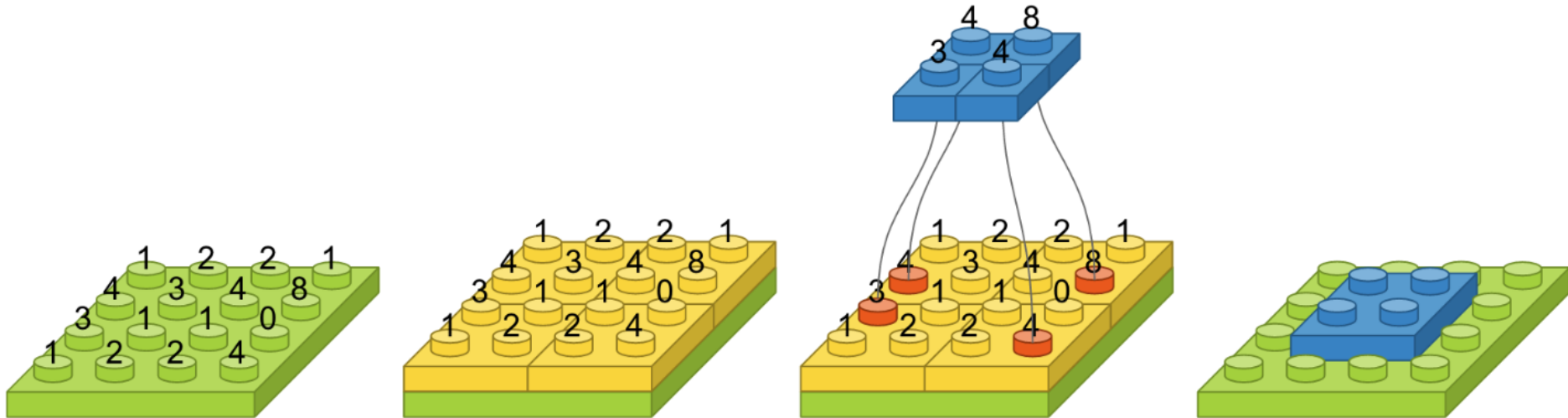
2	3	3	3	3	3	3	3	3	3	3	2
1	1	1									
1	1	1									
2	3	3	3	3	3	3	3	2	1		
1	2	3	3	3	3	3	3	2	2	2	1
								2	2	2	1
									1	1	1
									1	1	1
									1	1	1
									1	1	1
									1	1	1
									1	1	1
2	3	3	3	3	3	3	3	3	2	1	1

컨볼루션 신경망 레이어 이야기

◆ 사소한 변화를 무시해주는 맥스풀링(Max Pooling) 레이어

`MaxPooling2D(pool_size=(2, 2))`

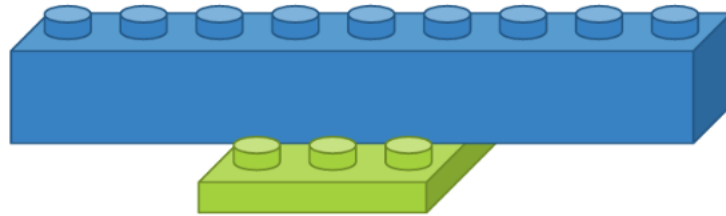
- `pool_size` : 수직, 수평 축소 비율을 지정합니다. (2, 2)이면 출력 영상 크기는 입력 영상 크기의 반으로 줄어듭니다.



컨볼루션 신경망 레이어 이야기

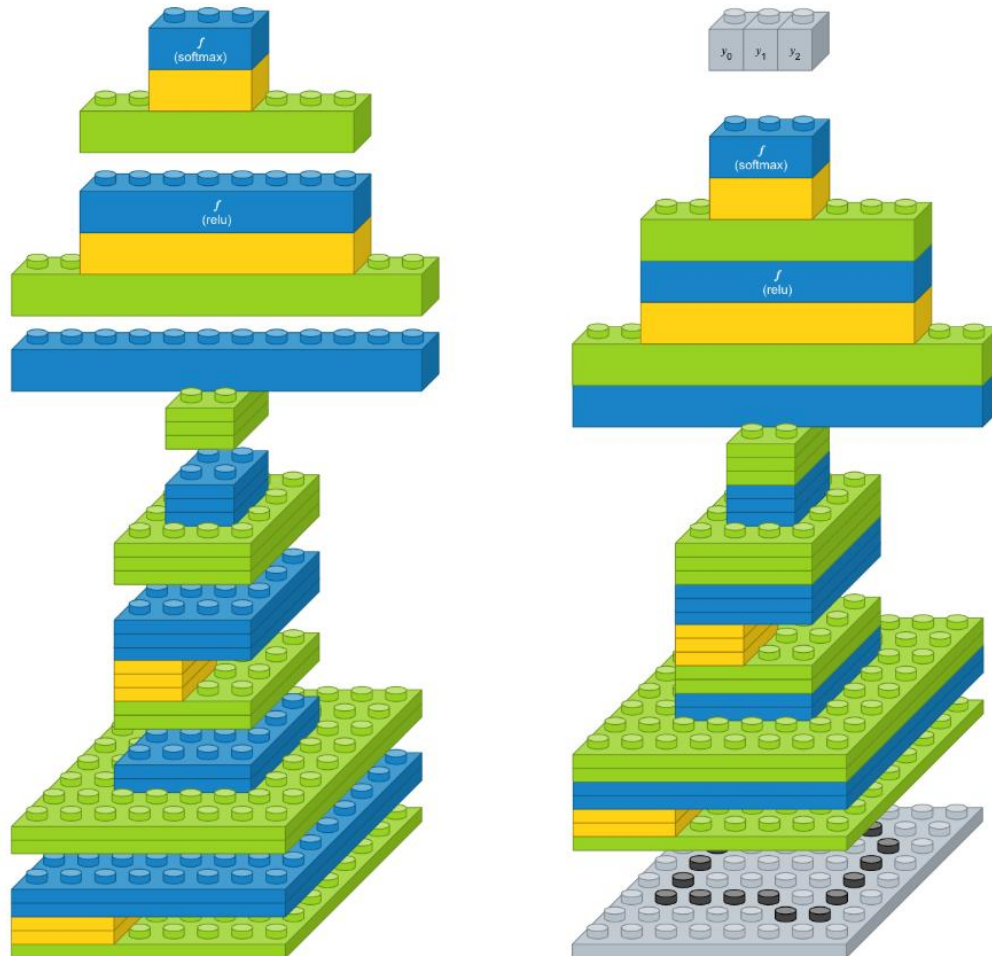
- ◆ 영상을 일차원으로 바꿔주는 플래튼(Flatten) 레이어
 - 크기가 3 x 3인 영상을 1차원으로 변경

Flatten()



컨볼루션 신경망 레이어 이야기

◆ 한번 쌓아보기



컨볼루션 신경망 레이어 이야기

◆ 한번 쌓아보기

```
model = Sequential()

model.add(Conv2D(2, (3, 3), padding='same', activation='relu', input_shape=(8, 8, 1)))
model.add(MaxPooling2D(pool_size=(2, 2)))
model.add(Conv2D(3, (2, 2), padding='same', activation='relu'))
model.add(MaxPooling2D(pool_size=(2, 2)))
model.add(Flatten())
model.add(Dense(8, activation='relu'))
model.add(Dense(3, activation='softmax'))
```

컨볼루션 신경망 모델 만들어보기

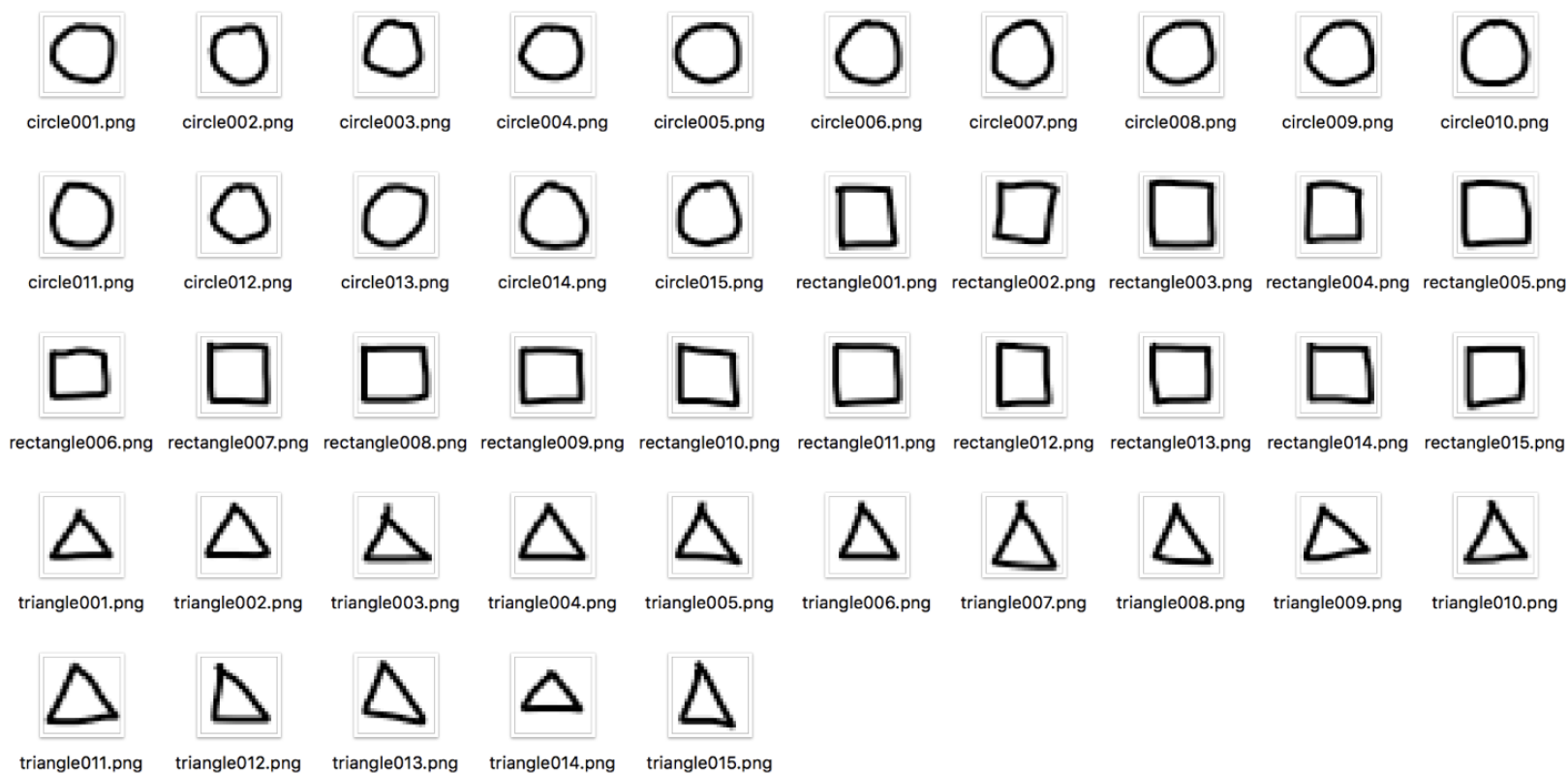
◆ 모델 구성하기

- 컨볼루션 레이어 : 입력 이미지 크기 24×24 , 입력 이미지 채널 3개, 필터 크기 3×3 , 필터 수 32개, 활성화 함수 'relu'
- 컨볼루션 레이어 : 필터 크기 3×3 , 필터 수 64개, 활성화 함수 'relu'
- 맥스풀링 레이어 : 풀 크기 2×2
- 플래튼 레이어
- 댄스 레이어 : 출력 뉴런 수 128개, 활성화 함수 'relu'
- 댄스 레이어 : 출력 뉴런 수 3개, 활성화 함수 'softmax'

컨볼루션 신경망 모델을 위한 부풀리기

◆ 현실적인 문제

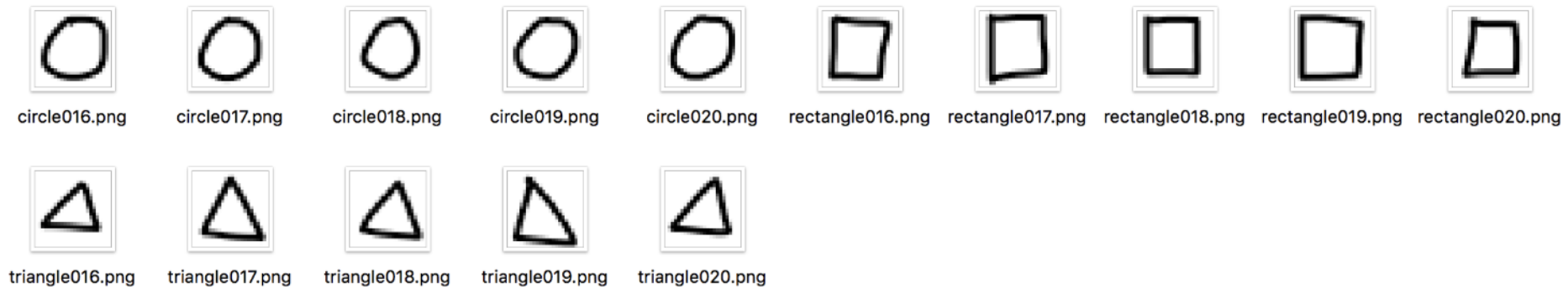
훈련셋



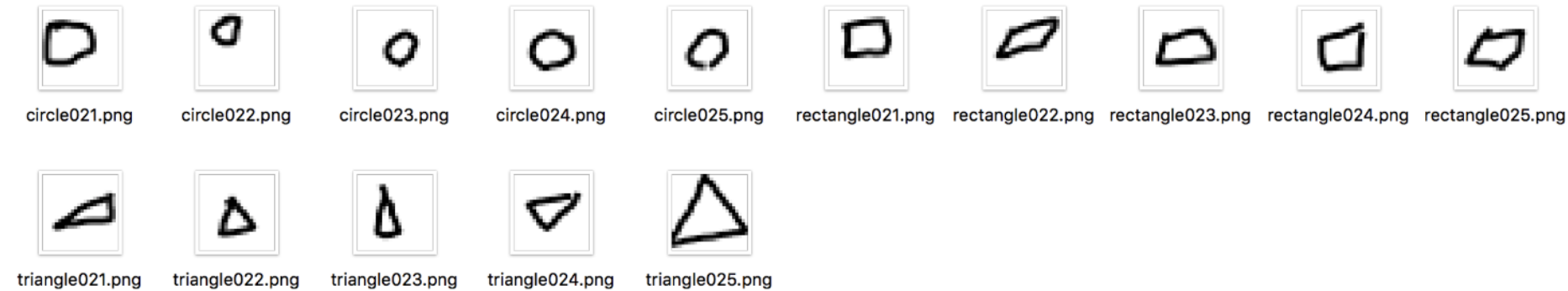
컨볼루션 신경망 모델을 위한 부풀리기

◆ 현실적인 문제

시험셋



도전 시험셋



컨볼루션 신경망 모델을 위한 부풀리기

◆ 현실적인 문제

– 교재 140



tri_0_914.png



tri_0_1137.png



tri_0_1850.png



tri_0_1862.png



tri_0_2127.png



tri_0_2194.png



tri_0_2252.png



tri_0_2271.png



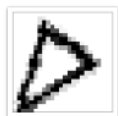
tri_0_2773.png



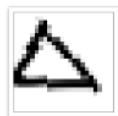
tri_0_3182.png



tri_0_3307.png



tri_0_3693.png



tri_0_4311.png



tri_0_4487.png



tri_0_4604.png



tri_0_5594.png



tri_0_6022.png



tri_0_6069.png



tri_0_6263.png



tri_0_6409.png



tri_0_6554.png



tri_0_6641.png



tri_0_7123.png



tri_0_7322.png



tri_0_7687.png



tri_0_7807.png



tri_0_8266.png



tri_0_8499.png



tri_0_8581.png



tri_0_8904.png

레시피 따라해보기

◆ MNIST 손글씨 인식

- 다층 퍼셉트론
- 컨볼루션 레이어
- 깊은 컨볼루션 레이어


```
train_datagen = ImageDataGenerator(rescale=1./255,  
    rotation_range=10,  
    width_shift_range=0.2,  
    height_shift_range=0.2,  
    shear_range=0.7, #0.7라이안 밀림  
    zoom_range=[0.9,2.2], # 0.9배~2.2배  
    horizontal_flip=True, # 수평방향으로 뒤집기  
    vertical_flip=True, # 수직방향으로 뒤집기  
    fill_mode='nearest')#이미지를 회전,  
    #이동하거나 축소할 때 공간을 채우는 방식
```

```
train_generator = train_datagen.flow_from_directory(  
    'data/handwriting/hand_test/train',  
    target_size=(24,24),  
    batch_size=3,  
    class_mode = 'categorical')
```

```
test_datagen = ImageDataGenerator(rescale=1./255)
```

```
test_generator = test_datagen.flow_from_directory(  
    'data/handwriting/hand_test/test',  
    target_size=(24,24),  
    batch_size=3,  
    class_mode = 'categorical')
```